Tell us about your PDF experience.

# AI Workload Documentation

AI workloads use non-deterministic behavior, making them more adaptable to complex situations compared to traditional workloads. They combine code and data into models, creating unique experiences. This guidance focuses on the architectural challenges of designing AI workloads, including non-deterministic functionality, data and application design, and operations.

## Get started

OVERVIEW

What is an AI workload?

CONCEPT

Design methodology

Design principles

Workload personas

## Technical design areas

CONCEPT

Application design

Application platform

Training data design

Grounding data design

Data platform

## Operational design areas

CONCEPT

Workload operations

MLOps and GenAIOps

Testing and evaluation

Responsible AI

## Reference examples

ARCHITECTURE

Baseline Microsoft Foundry chat reference architecture

## Related services

ARCHITECTURE

Azure Machine Learning service guide

## Assessment

HOW-TO GUIDE

AI assessment tool

# AI workloads on Azure

Article • 01/08/2025

This article addresses architectural challenges of designing AI workloads. It focuses on nondeterministic functionality, data and application design, and operations. The recommendations are based on Azure Well-Architected Framework principles and include insights from successful Azure implementations.

These articles are meant for *workload owners* and *technical stakeholders* like architects, development leads, and IT leaders. Specialized AI and data roles, such as data scientists, should also be aware of this guidance because collaboration across various roles and teams is a key aspect.

> **Note**
>
> Azure offers various AI services that you can integrate into your workload or build around it. Depending on your business needs, you can choose between fully managed software as a service (SaaS) solutions, platform as a service (PaaS) solutions, or build your own AI solution. *Specific Azure services and their capabilities are not covered here*. We recommend that you refer to the respective product documentation for that information.
>
> Also, certain AI workloads aren't in scope, such as:
>
> - Workloads that are realized through low-code and no-code offerings, such as Copilot Studio.
> - Workloads that require high-performance computing.
> - Workloads that don't implement generative or discriminative AI use cases.

## What is an AI workload?

In the context of Well-Architected Framework, an AI workload meets the needs of predictive, discriminatory, or generative tasks. It focuses on ethical functionality, adapting to fast-evolving AI technologies, and staying relevant and explainable. Apply the Well-Architected Framework pillars at every decision point to ensure that the system is reliable, secure, efficient, and cost-effective.

AI workloads are different from traditional ones because they replace deterministic functionality in parts of the workload with nondeterministic behavior that solves for situations where fixed outcomes are impractical. Instead, they combine code and data

into an entity, or a *model*, to enable unique experiences that traditional systems can't provide.

Before you begin your design strategies, consider these key points first.

## Familiarize yourself with the broad categories of models

- **Generative AI**: Uses machine learning to autonomously create new content. It includes language models that can be customized with user data or used as services like Azure OpenAI Service. For example, GPT, a type of language model, specializes in mimicking human conversational language and is ideal for chat and natural language experiences.

  Use cases: Generative AI can produce articles, stories, and art. It can also generate synthetic data to balance datasets and make chatbots more humanlike.

- **Discriminative AI**: Uses explicit programming to perform specific tasks based on rules and algorithms. It's divided into:
  - **Model-based**: Predictive systems find patterns based on training performed from prior observations to make predictions but can't create new content or adapt on their own.
  - **Non-model-based**: Autonomous agents follow predefined rules to interact with systems, like video game characters.

  Use case: Discriminative AI is used for predictive analytics, recommendation systems, and fraud detection.

This series of articles covers different AI workloads and focuses on specific types like language models when needed.

> **ⓘ Important**
>
> When you choose between generative and discriminative models, think about the task you need to accomplish. Generative models create new data. Discriminative models classify existing data based on features. For classification or regression tasks, pick models that fit the job. For example, a language model that can classify might be more versatile than one that only classifies.

## Evaluate your build versus buy options

If generic responses are acceptable, a prebuilt model or an AI service-based solution that uses opaque processing should be sufficient for your workload. But if you need

data specific to your business or have compliance requirements, create a custom model.

When you choose between a custom model, a prebuilt model, or a service, consider these factors:

- **Data control**: Custom models give you more control over sensitive data. Prebuilt models are easier for general tasks.
- **Customization**: Custom models are better for unique needs. Prebuilt models might lack flexibility.
- **Cost and upkeep**: Custom models need ongoing maintenance and resources. Prebuilt models usually have lower initial costs and less infrastructure burden.
- **Performance**: Prebuilt services offer optimized infrastructure and scalability. They're ideal for low-latency or high-scalability needs.
- **Expertise**: Custom models require a skilled team. Prebuilt models are often quicker to deploy and easier to use if expertise is limited.

> **ⓘ Important**
>
> Creating and maintaining your own model requires a lot of resources, time, and expertise. It's important to research thoroughly before you decide. Usually, choosing a prebuilt model or a managed service is a better option.

## What are the common challenges?

- **Compute costs**: AI functions can be expensive because of high compute needs, and compute needs can vary according to your workload design. Understand your requirements and choose the right service to manage costs.
- **Security and compliance requirements**: Off-the-shelf solutions might not meet your security and compliance needs. Research options to avoid unnecessary burdens.
- **Volume of data**: Handling large data volumes in various formats comes with challenges of protecting sensitive information and efficient processing. Optimizing storage, processing, and transfer costs should be an ongoing activity.
- **Model decay**: Models can degrade over time, which leads to inaccurate results. Testing AI systems is challenging because of their randomness.
- **Skill challenges**: New AI workloads might need specialized roles and new operations processes that require extensive training.
- **Pace of AI innovation**: Adopting the latest technologies can be tempting to stay at the cutting edge. Evaluate new technologies carefully to ensure that they improve user experience and don't just add complexity for the sake of being up to date.

- **Ethical requirements**: Clearly determine whether your use case is an ethical target for AI. Maintaining ethical standards is necessary throughout the planning and implementation phases to ensure that you're building a responsible system.

# How to use this guidance

✓ Start with Design methodology, which outlines the rationale and recurring themes across technical and operational areas. This systematic approach helps to define requirements and design strategies. Revisit this methodology when you face uncertain choices to stay aligned with the workload's overall goals. It also provides a framework for collaborating with stakeholders to justify technical decisions and incorporate customer feedback for continuous improvement.

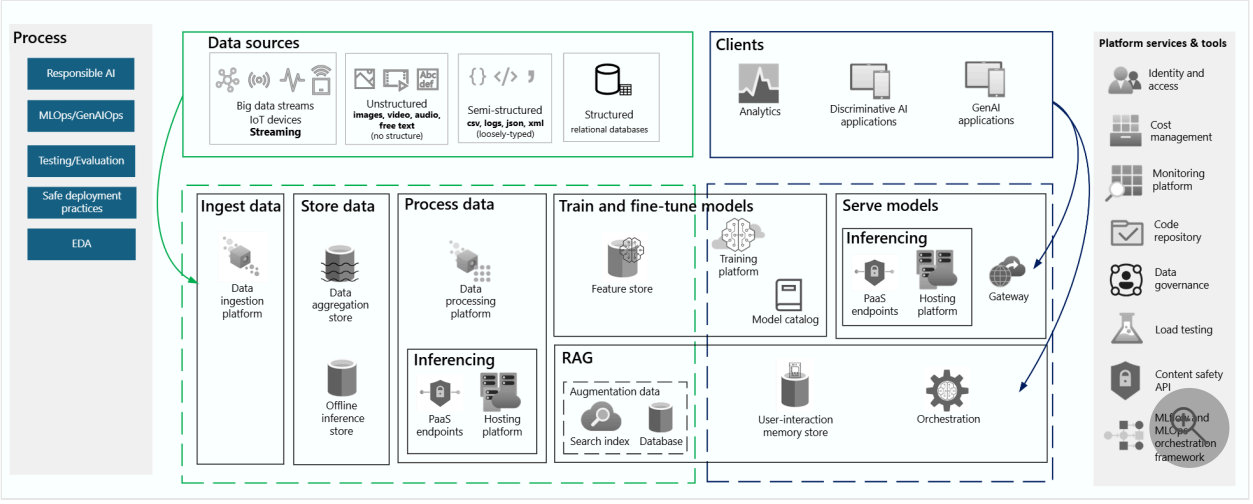
✓ Proceed to Design principles to see how the design methodology aligns with the core Well-Architected Framework pillars. Consider growth evolution. Evaluate the underlying principles for all pillars collectively, including the tradeoffs.

✓ Focus on the *design areas* that have the biggest effect on your solution. Each area includes considerations and recommendations to guide you through the design decisions.

✓ Use the Assessment Review Tool to evaluate the readiness of your optimized AI workload in production.

# Typical architecture pattern and design areas

The following diagram illustrates how data flows through the system from initial collection to final user interaction.




The architecture highlights the integration of different components to enable efficient data processing, model optimization, and real-time application deployment in AI-driven

solutions. It includes modules such as data sources, data processing, model training, model deployment, and user interfaces.

The following table describes some key design areas related to that pattern.

Expand table

| Design areas |
| --- |
| Application design: Learn about considerations unique to AI workloads that might have a significant effect on your existing application design standards. |
| Application platform: Determine the best platforms to use to support AI workload functions, like model hosting, model training, and inferencing. |
| Training data design: Design strategies for data ingestion, preprocessing, retention, and governance topics to handle your model training data. |
| Grounding data design: Design strategies to optimize searchability and retrieval, while meeting security and compliance requirements for your grounding data. |
| Data platform: Determine the best hosting platform to handle the large amounts and potentially many formats of data that your workload uses. |
| Machine learning operations and Generative AI operations: Establish modern DevOps practices to support your machine learning or generative AI functions and systems. |
| Workload operations: Modernize your operational practices with new approaches and add specialized roles and training. |
| Testing and evaluation: Develop testing and evaluation strategies to measure characteristics like accuracy, precision, sensitivity, and specificity through metrics that are targeted for AI workloads. |
| Workload personas: Understand how personas are involved in the full lifecycle of your AI workload to help ensure that your team is fully capable of building and supporting it. |
| Responsible AI: Pay special attention to the user experience and ethical implications of releasing your AI solution to the public. AI brings incredible opportunities for new products and services, but it also carries a considerable degree of risk. |

**Tip**

Every architectural decision involves a range of considerations and a set of acknowledged compromises that balance different aspects of the framework. These tradeoffs are indicated by this icon  .

## Next step

Design methodology

## Feedback

Was this page helpful? Yes No

# Design methodology for AI workloads on Azure

Article • 01/09/2025

When you design AI workloads, the code and data that you integrate replaces deterministic behavior. This shift facilitates tasks like prediction, classification, and other functional goals. AI workload architecture can often be complex and must align with business constraints. The Azure Well-Architected Framework provides a solid foundation for architectural excellence, but you should also consider AI-specific design principles.

This article presents a design methodology based on AI principles. The AI principles systematically guide the design and optimization of solutions. Another benefit of the methodology is collaboration with product owners and stakeholders to justify technical decisions. If you need help making decisions, refer back to this methodology to **align your design direction with high-level AI principles**.

If you design a capability or introduce an improvement, evaluate the change from the methodology perspective. Does your change affect the user experience? Is your change flexible enough to adapt to future innovations? Does it disrupt the experimentation flow?

## Design with an experimental mindset

Design with an experimental mindset so that you can **achieve relevancy through iterative and statistically driven processes** based on real-world use cases.

Experimentation in AI involves continuous adjustments with results that you can measure against quality targets after each iteration. Do an experimentation loop during the initial model evaluation and ongoing refinement. The inner loop refines a model's predictive power in a development environment. The outer loop monitors production usage and can trigger further refinement or data preparation. Both loops rely on continuous monitoring and evaluation to identify improvements.

Not every experiment succeeds. Consider worst-case scenarios, and have contingency plans for failed experiments.

## Design responsibly

When users interact with your AI system, they place their **trust in its ethical functionality**, even if they don't understand the underlying logic and decision making of

the AI model. This trust holds you accountable for designing a system that prevents unethical behaviors, such as manipulation, content toxicity, IP infringement, and fabricated responses. You should embed responsible AI principles in systems operations and in the culture of your team. The practices must extend throughout the lifecycle of user interaction—from the users' initial intent to use the system, during sessions, and even during disruptions caused by system errors.

Content moderation is a key strategy in responsible design of generative AI. Content moderation evaluates requests and responses in real-time to help ensure safety and appropriateness. As part of the experimentation loops, strive to make algorithms fair and inclusive to minimize bias. Bias can enter the system through various channels, including during actual sessions or when you gather feedback.

Ethical data management is central to responsible design. Make informed decisions about when to use or avoid relying on user data. Users trust you to ensure that personal information is either removed from the system or retained only with their consent. If retention is inevitable, ensure that you use trusted technology to protect data, which helps ensure privacy and security.

## Design for explainability

AI model results must be explainable and justifiable. You should be able to **trace the origins of the data, the inference processes, and the journey of data** from its source to the serve layer. In discriminative AI, decisions can be justified with each step. In generative models, explainability can be complex. Document the decision-making process, both manually and through technical capabilities.

Explainable results help ensure the system's transparency and accountability to gain users' trust.

## Stay ahead of model decay

Model decay is a unique challenge in AI that affects design decisions. The **quality of AI model outputs can deteriorate** over time without any changes to the code. Sometimes deterioration can even occur suddenly because of changes in data or external factors.

This deterioration affects various aspects of the system. These aspects include data-ingestion speed, data quality, monitoring needs, evaluation processes, and reaction times to correct problems. Implement early detection through a combination of automated processes for continuous monitoring and model evaluation. Take advantage of user feedback to help identify model decay.

Regardless of the methods that you use to identify model decay, the operations team must engage data scientists to promptly research and address potential decay problems.

## Design for adaptability

AI is advancing at a rapid pace in terms of technological advancement and adoption. Be aware that **what you build today might become obsolete quickly.** Keep this consideration in mind when you make design decisions and create processes.

AI advancement emphasizes the need for **agility and flexibility**. Recognize that some components might have a limited lifespan. Adopt a *pause-and-think* approach that focuses on the research of model discovery, programming libraries and frameworks, and processing technologies.

## Next step

Learn about design principles to build and operate AI workloads on Azure.

Design principles

## Feedback

**Was this page helpful?** Yes No

# Design principles for AI workloads on Azure

Article • 01/09/2025

This article outlines the core principles for AI workloads on Azure, with a focus on the AI aspects of an architecture. It's essential to consider all Azure Well-Architected Framework pillars collectively, including their trade-offs. Apply each pillar to the functional and nonfunctional requirements of the workload.

## Reliability

When you run AI workloads on Azure, you need to consider many of the same reliability requirements that you consider for other types of workloads. However, specific considerations for model training, hosting, and inferencing are particularly important and are the focus of this article. It's important integrate these practices with standard design best practices for cloud applications, which also apply to AI workloads.

Review the reliability design principles to get a foundational understanding of the concepts described here.

Expand table

| Design principle | Considerations |
| --- | --- |
| Mitigate single points of failure. | Relying on a single instance for critical components can lead to significant problems. To prevent these kinds of problems, build redundancy into all critical components. Use platforms that have built-in fault tolerance and high-availability features, and implement redundancy by deploying multiple instances or nodes. |
| Conduct failure mode analysis.<br><br>Take advantage of well-known design patterns. | Regularly analyze potential failure modes in your system components, and build resilience against these failures. Use well-known design patterns to isolate parts of your system and prevent cascading failures. For example, the Bulkhead pattern can help isolate failures, and retry mechanisms and circuit breakers can handle transient errors like throttling requests. |
| Balance reliability objectives across dependent components. | Ensure that the inferencing endpoint, or model, and the data store are aligned in terms of reliability. Application components must remain available even under unexpected conditions, such as a surge in concurrent users. Additionally, the workload should be able to fail over to backup systems in case of failures. If one component fails, it affects the reliability of the other. |

| Design principle | Considerations |
| --- | --- |
| | Similarly, service layer APIs, which are critical production resources, should adhere to the same reliability standards as other high-criticality workload flows. If these APIs require high availability, the hosting platform must support availability zones or a multiregion design to ensure continuous operation and resilience. |
| Design for operational reliability. | Maintain the reliability of model responses by ensuring that updates are frequent and timely. Decide whether to use recent data or slightly older data, balancing the need for up-to-date information with the practicality of update frequency. You should automate online training because of its frequency and reliability requirements. Offline training can be justified through a cost-benefit analysis. You can use cheaper resources for offline training. |
| Design for a reliable user experience. | Use load testing to evaluate how your workload handles stress, and design your user interface to manage user expectations for response times. Implement UI elements that inform users of potential wait times, and adopt asynchronous cloud design principles to address intermittence, latency, or failures. |

# Security

Models often use sensitive production data to produce relevant results. You must therefore implement robust security measures in all layers of the architecture. These measures include implementing secure application design early in the lifecycle, encrypting data both at rest and in transit, and adhering to industry compliance standards. Regular security assessments are crucial for identifying and mitigating vulnerabilities. Advanced threat detection mechanisms help ensure prompt responses to potential threats.

Security principles are fundamental for safeguarding data integrity, design integrity, and user privacy in AI solutions. As a workload owner, you're responsible for building trust and protecting sensitive information to help ensure a safe user experience.

Expand table

| Design principle | Considerations |
| --- | --- |
| Earn user trust. | Integrate content safety at every stage of the lifecycle by using custom code, tools, and effective security controls.<br><br>Remove unneeded personal and confidential information at all data storage points, including aggregated data stores, search |

| Design principle | Considerations |
| --- | --- |
| | indexes, caches, and applications. Maintain this level of security consistently throughout the architecture.<br><br>Be sure to implement thorough content moderation that inspects data in both directions and filters out inappropriate and offensive content. |
| Protect data at rest, in transit, and in use, according to the sensitivity requirements of the workload. | At a minimum, use platform-level encryption with Microsoft-managed or customer-managed keys on all data stores in the architecture. Determine where you need higher levels of encryption, and use double encryption if you need to.<br><br>Ensure that all traffic uses HTTPS for encryption. Determine the TLS termination points at each hop.<br><br>The model itself needs to be protected to prevent attackers from extracting sensitive information that's used during training. The model requires the highest level of security measures. Consider using homomorphic encryption that allows inferencing on encrypted data. |
| Invest in robust access management for identities (user and system) that access the system. | Implement role-based access control (RBAC) and/or attribute-based access control (ABAC) for both control and data planes.<br><br>Maintain proper identity segmentation to help protect privacy. Only allow access to content that identities are authorized to view. |
| Protect the integrity of the design by implementing segmentation. | Provide private networking for access to centralized repositories for container images, data, and code assets.<br><br>When sharing infrastructure with other workloads creates segmentation, for example, when you host your inference server in Azure Kubernetes Service, isolate the node pool from other APIs and workloads. |
| Conduct security testing. | Develop a detailed test plan that includes tests for detecting unethical behavior whenever changes are introduced to the system.<br><br>Integrate AI components into your existing security testing. For instance, incorporate the inferencing endpoint into your routine tests alongside other public endpoints.<br><br>Consider conducting tests on the live system, such as penetration tests or red team exercises, to identify and address potential vulnerabilities effectively. |

| Design principle | Considerations |
| --- | --- |
| Reduce the attack surface by enforcing strict user access and API design. | Require authentication for all inferencing endpoints, including system-to-system calls. Avoid anonymous endpoints.<br><br>Prefer constrained API design at the cost of client-side flexibility. |

> **Trade-off.** Implementing the highest levels of security incurs trade-offs in cost and accuracy because the ability to analyze, inspect, or log the encrypted data is limited. Content safety checks and achieving explainability can also be challenging in highly secured environments.

The following design areas provide details on the preceding principles and considerations:

- Responsible AI
- Testing
- Operations
- MLOps and GenAIOps

# Cost Optimization

The goal of the Cost Optimization pillar is to maximize investment, not necessarily to reduce costs. Every architectural choice creates both direct and indirect financial impacts. Understand the costs that are associated with various options, including build versus buy decisions, technology selections, billing models, licensing, training, and operational expenses. It's important to maximize your investment in the selected tier and continuously reassess billing models. Continuously evaluate costs that are associated with changes in architecture, business needs, processes, and team structure.

Review the Cost Optimization principles, focusing on rate and usage optimization.

Expand table

| Design principle | Considerations |
| --- | --- |
| Determine the cost drivers by doing a comprehensive cost modeling exercise. | Consider the key factors of the data and application platform:<br>- **Volume of data**. Estimate the amount of data you'll be indexing and processing. Larger volumes can increase storage and processing costs.<br>- **Number of queries**. Predict the frequency and complexity of queries. Higher query volumes can increase costs, especially if the queries require significant computational resources. |

| Design principle | Considerations |
| --- | --- |
| | - **Throughput**. Assess the expected throughput to determine which performance tier you need. Higher throughput demands can lead to higher costs.<br>- **Dependency costs**. Understand that there might be hidden costs in dependencies. For example, when you calculate the cost of indexing, include the cost of the skill set, which is part of data processing but outside the index scope. |
| Pay for what you intend to use. | Choose a technology solution that meets your needs without incurring unnecessary expenses. If you don't need advanced features, consider less expensive options and open-source tools.<br><br>Factor in the frequency of usage. Prefer elastic compute options for orchestration tools to minimize utilization costs, because they're *always-on*. Avoid serverless compute for full-time operations because it can escalate costs.<br><br>Don't pay for higher tiers without using their full capacity. Ensure that your chosen tier aligns with your production usage patterns to optimize spending. Use standard pricing for regular tasks and spot pricing for highly interruptible training. To reduce costs, use GPU-based compute only for AI workload functions.<br><br>Extensively test and benchmark your training and fine-tuning to find the SKU that best balances performance and cost. |
| Use what you pay for. Minimize waste. | Monitor utilization metrics closely. AI workloads can be expensive, and costs can escalate quickly if resources aren't shut down, scaled down, or deallocated when they're not being used.<br><br>Optimize the system for *write once, read many* to avoid overspending on data storage. Training data doesn't require the instant responsiveness of a production database.<br><br>Stress testing an Azure OpenAI Service inference endpoint can be expensive because every call incurs charges. To reduce costs, use unused PTUs of OpenAI Service in a test environment or simulate the inference endpoint instead.<br><br>Tasks like exploratory data analysis (EDA), model training, and fine-tuning are typically not run full time. For these tasks, prefer a platform that can be stopped when it's not in use. For example, EDA jobs are typically interactive, so users need to be able to start VMs and stop them as they run jobs.<br><br>Assign cost accountabilities to operations teams. These teams should ensure that costs stay within expected parameters by actively monitoring and managing resource utilization. |

| Design principle | Considerations |
| --- | --- |
| Optimize operational costs. | Online training can be expensive because of its frequency requirements. Automating this process helps maintain consistency and minimizes cost from human error. Additionally, using slightly older data for training, and delaying updates when possible, can further reduce expenses without significantly affecting model accuracy.<br><br>For offline training, evaluate whether cheaper resources can be used, such as offline hardware.<br><br>In general, delete data from feature stores to reduce clutter and storage costs for features. |

 **Trade-off: Cost Optimization and Performance Efficiency.** Balancing cost with performance in database management involves trade-offs. Efficient index design speeds up queries but can increase costs because of metadata management and index size. Similarly, partitioning large tables can improve query performance and reduce storage load, but it incurs additional costs. Conversely, techniques that avoid excessive indexing can reduce costs but might affect performance if they're not managed properly.

 **Trade-off: Cost Optimization and Operational Excellence.** There are trade-offs to consider when you decide which of the two primary approaches to use for model training. Training in the development environment with full production data can reduce computational costs because the model is trained only once and only the artifact is promoted. However, this approach requires stringent security measures for handling production data in lower environments, which can be complex and resource intensive. Conversely, training the model in each environment enhances stability and reliability because of thorough code review and testing, but this model increases computational costs because training runs occur multiple times.

# Operational Excellence

The primary objective of Operational Excellence is to deliver capabilities efficiently throughout the development lifecycle. Achieving this goal involves establishing repeatable processes that support the design methodology of experimentation and yield results to improve model performance. Operational Excellence is also about maintaining the accuracy of models over time, implementing effective monitoring

practices and governance to minimize risks, and developing change management processes to adapt to model drift.

Although all Operational Excellence principles apply to AI workloads, prioritize automation and monitoring as your foundational operational strategies.

Expand table

| Design principle | Considerations |
| --- | --- |
| Foster a continuous learning and experimentation mindset throughout the lifecycles of application development, data handling, and AI model management. | Workload operations should build on industry-proven methodologies like DevOps, DataOps, MLOps, and GenAIOps.<br><br>Early collaboration between operations, application development, and data teams is essential to establish a mutual understanding of acceptable model performance. Operations teams provide quality signals and actionable alerts. Application and data teams help diagnose and resolve problems efficiently. |
| Choose technologies that minimize operational burden. | When you choose platform solutions, prefer platform as a service (PaaS) over self-hosted options to simplify design, automate workflow orchestration, and make day-2 operations easier. |
| Create an automated monitoring system that supports alerting functionality, including logging and auditability. | Given the nondeterministic nature of AI, it's important to establish quality measurements early in the lifecycle. Work with data scientists to define quality metrics. Visualize ongoing insights in comprehensive dashboards.<br><br>Track experiments by using tools that can capture details like code versions, environments, parameters, runs, and results.<br><br>Implement actionable alerts that provide just enough information to enable operators to respond quickly. |
| Automate the detection and mitigation of model decay. | Use automated tests to evaluate drift over time. Ensure that your monitoring system sends alerts when responses start deviating from the expected results and start doing so on a regular basis. Use tools that can automatically discover and update new models.<br><br>As needed, adapt to new use cases by adjusting data processing and model training logic. |
| Implement safe deployments. | Choose between side-by-side deployments or in-place updates to minimize downtime. Implement thorough testing before deployment to ensure the model is correctly |

| Design principle | Considerations |
| --- | --- |
| | configured and meets targets, user expectations, and quality standards. Always plan for emergency operations, regardless of the deployment strategy. |
| Continuously evaluate the user experience in production. | Enable workload users to provide feedback about their experience. Get consent to share part or all of their conversations in an associated log for troubleshooting. Consider how much of a user interaction is viable, compliant, safe, and useful to capture, and use the data diligently to evaluate the performance of your workload with real user interactions. |

The following design areas provide details on the preceding principles and considerations:

- Operations
- MLOps and GenAIOps
- Testing

# Performance Efficiency

The goal of AI model performance evaluation is to determine how effectively a model accomplishes its intended tasks. Accomplishing this goal involves assessing various metrics, like accuracy, precision, and fairness. Additionally, the performance of the platform and application components that support the model is crucial.

Model performance is also influenced by the efficiency of operations like experiment tracking and data processing. Applying Performance Efficiency principles helps measure performance against an acceptable quality bar. This comparison is key to detecting degradation or decay. To maintain the workload, including AI components, you need automated processes for continuous monitoring and evaluation.

Expand table

| Design principle | Considerations |
| --- | --- |
| Establish performance benchmarks. | Conduct rigorous performance testing across different architectural areas, and set acceptable targets. This ongoing assessment should be part of your operational processes, not a one-time test.<br><br>Benchmarking applies to prediction performance. Start with a baseline to understand initial model performance and |

| Design principle | Considerations |
| --- | --- |
| | continuously re-evaluate performance to ensure that it meets expectations. |
| Evaluate resource needs for meeting performance targets. | Understand load characteristics to choose the right platform and right-size resources. Factor in this data for capacity planning to operate at scale.<br><br>For example, use load testing to determine the optimal compute platform and SKU. High-performance GPU-optimized compute is often needed for model training and fine-tuning, but general-purpose SKUs are suitable for orchestration tools.<br><br>Similarly, choose a data platform that efficiently handles data ingestion, manages concurrent writes, and maintains individual write performance without degradation.<br><br>Different inferencing servers have varying performance characteristics. These characteristics affect model performance at runtime. Select a server that meets baseline expectations. |
| Collect and analyze performance metrics, and identify bottlenecks. | Evaluate telemetry from the data pipeline to ensure that performance targets for throughput and read/write operations are met.<br><br>For the search index, consider metrics like query latency, throughput, and the accuracy of the results. As data volume increases, error rates shouldn't increase.<br><br>Analyze telemetry from code components, like the orchestrator, which collects data from service calls. Analyzing that data can help you understand time spent on local processing versus network calls and identify potential latency in other components.<br><br>Assess the UI experience by using engagement metrics to determine whether users are positively engaged or frustrated. Increased time on a page can indicate either. Multimodal capabilities, like voice or video responses, can cause significant latency, leading to longer response times. |
| Continuously improve benchmark performance by using quality measurements from production. | Implement automated collection and analysis of performance metrics, alerts, and model retraining to maintain model efficacy. |

 **Trade-off**. When you consider platform capabilities, you need to balance cost and performance. For example, GPU SKUs are expensive, so continuously monitor for under-utilization and right-size resources as needed. After adjustments, test

> resource utilization to ensure a balance between the cost of platform resources and their operations and the expected performance, as indicated by the baseline. For Cost Optimization strategies, see Recommendations for optimizing component costs.

The following design areas provide details on the preceding principles and considerations:

- Testing
- MLOps and GenAIOps

## Next step

Design area: Application design

## Feedback

Was this page helpful? Yes No

# Application design for AI workloads on Azure

10/08/2025

This article describes common design areas and factors to consider when you make decisions about technology and approach when you create an application that has AI functions. The guidance applies to both traditional machine learning and generative AI workloads. It covers frameworks like TensorFlow, Keras, and PyTorch, along with various serving and hosting options that include batch, stream, and online processing.

There are many choices to consider when you design intelligent capabilities into your workload. Your unique functional and nonfunctional requirements help you determine whether you need basic inferencing capabilities or more dynamic problem-solving capabilities. These requirements guide your high-level design decisions. Consider these choices as you move from high-level design areas to lower-level design areas.

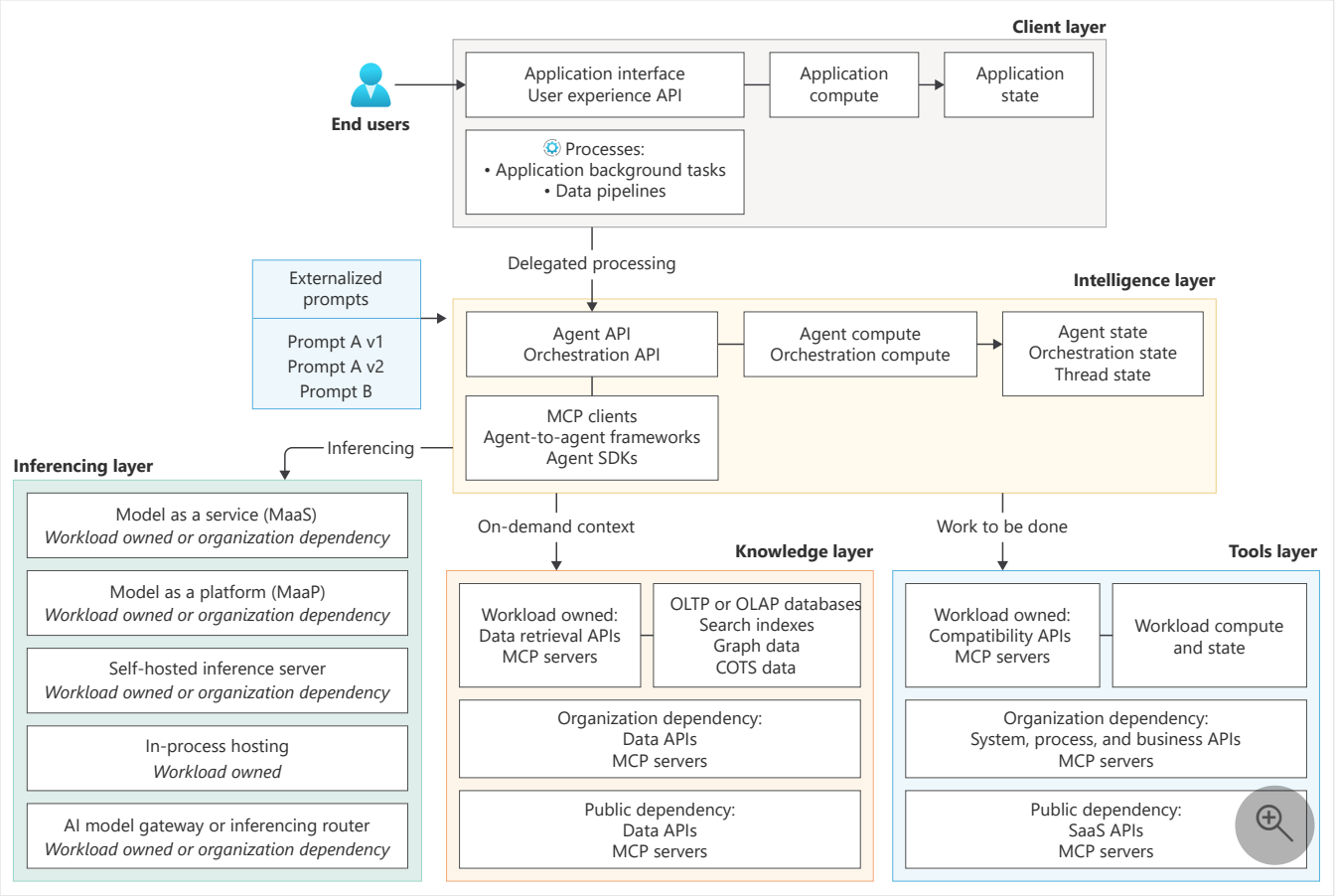
As discussed in the Get started article, whether to build your own model or use a prebuilt model is one of the first important decisions to make. When you use a prebuilt model, consider the following points:

- **Catalog sources:** Explore repositories like the Hugging Face Model Hub, TensorFlow Hub, and the Azure AI Foundry portal model catalog to find pretrained models. These platforms provide an extensive catalog of models for various tasks.
- **Licensing:** Make sure that the model's licensing terms fit your security, compliance, and application goals, especially if you plan to distribute the application or integrate it with other services.
- **Key components:** Look at the model's architecture, training data, and performance to determine whether it's fine-tuned for your task or domain.

For guidance about how to choose a hosting platform, see Considerations for the model hosting and inferencing platform.

## Application layer architecture

When you design intelligent capabilities, establish clear boundaries in your design across the following five key layers.




- **Client layer:** The user interface and client applications where users or processes experience your workload's intelligent capabilities. Keep this layer thin and delegate most capabilities to other layers.
- **Intelligence layer:** Routing, orchestration, and agent capabilities that coordinate AI operations. This layer includes model routing, conversation management, and intelligent decision-making.
- **Inferencing layer:** Running trained models to generate predictions or decisions based on data. This layer handles model loading and runtime invocation, input preprocessing and output postprocessing, along with serving predictions via APIs or embedded systems.
- **Knowledge layer:** Grounding data, knowledge graphs, and retrieval services that provide relevant context and information to the intelligence layer. This layer enforces data access policies and authorization.
- **Tools layer:** Business APIs, external services, and action capabilities that the intelligence layer can invoke. This layer should use standardized interfaces and enforce its own security policies.

Each layer enforces its own policies, identities, and caching strategies to achieve their own localized reliability, security, and performance requirements. The layers also enable focused development, testing, and troubleshooting.

# Recommendations

The following table summarizes the recommendations provided in this article.

Expand table

| Recommendation | Description |
| --- | --- |
| Prioritize security and Responsible AI controls. | Implement traditional application security plus AI-specific safety measures as a primary design driver. Enforce provider safety systems, input/output filtering, identity-bound rate limiting and quotas, and token/prompt caps. Security and safety controls must be verified and can't be assumed from managed services. |
| Keep intelligence away from the client. | Design back-end services to handle cross-cutting concerns like rate limiting, failover operations, and AI processing logic. Abstract behavior and intelligence away from the client to future-proof your design and improve maintainability. |
| Block direct access to data stores. | Code in AI systems shouldn't directly access your data stores. Route all data requests through an API or similar data access abstraction that enforces authorization and propagates user or tenant context into retrieval and filtering. Pass forward user identity so that data level security can be applied. |
| Abstract your models and tools. | Use abstraction layers to decouple your application from specific models, tools, and technologies. Implement standardized interfaces and protocols to provide flexibility as technologies evolve, which makes your design more maintainable and future-proof. |
| Isolate behaviors and actions. | Design clear boundaries across client, intelligence (routing/orchestration/agents), knowledge (grounding data), and tools (business APIs) layers. Each layer should enforce its own policies, identities, and caching strategies to reduce blast radius and focus development efforts. |
| Prioritize prebuilt solutions. | Use software as a service (SaaS) or platform as a service (PaaS) to handle workload functions when they meet security, safety, compliance, and quota needs. Implement compensating controls through gateways that enforce authentication, quotas, safety, and logging. Use prebuilt and pretrained models where possible to minimize the operational and development burden for your workload and operations teams. |

# Distinguish between inferencing and intelligent applications

When you design your application architecture, first determine whether you're building an inferencing-focused application or an intelligence-focused application because this distinction drives design decisions.

## Inferencing applications

Inferencing applications perform single-step operations like classification, translation, or summarization. These applications have simpler architectures.

**Typical architecture:** The client communicates with an AI gateway that provides authentication, quotas, safety, and routing. That gateway calls into the model serving layer such as Azure AI Foundry, Azure Kubernetes Service (AKS), or managed online endpoints. Where practical, the results might be cached for future inferencing calls before being returned to the client.

Inferencing applications typically feature direct model calls, require minimal orchestration, focus on performance and throughput, and don't require complex state management.

## Intelligent applications

Intelligent applications perform planning, coordination, and multi-step reasoning and are typically handled through agents and agent orchestration. They require more concepts to be addressed in their architectures.

**Typical architecture:** The client invokes an agent or agent orchestrator. Based on the design or autonomous capabilities, this layer invokes a tools layer such as Model Context Protocol (MCP) servers or custom APIs. The agent might need to call into grounding knowledge services such as a search index, database, or graph. Models might be invoked at multiple points during this process. Caching can also occur at multiple levels to optimize the process.

Intelligent applications typically feature agentic patterns and model routing with complex workflow coordination, integrate with multiple data sources and tools, and require conversation and context management.

# Fundamental AI application design guidelines

When you design AI applications, apply the following foundational guidelines to create resilient and maintainable systems.

## Start with business outcomes

Before you select technologies, clearly define the business problem you're solving. This definition guides technology choices and architecture design. Consider the following factors:

- **Success metrics:** Define measurable outcomes that demonstrate value, such as accuracy improvements, cost reductions, or user satisfaction scores.

- **User experience requirements:** Understand how users and processes will interact with AI capabilities and what response times that they expect.
- **Regulatory constraints:** Identify compliance requirements that might affect your design choices, such as data residency or explainability requirements.

## Design for security from day one

AI workloads demand a security-first approach from the outset. Implement layered security across all components by using identity propagation and auditable controls. For more information, see Security design strategies.

## Design for observability from day one

AI applications require monitoring beyond traditional application metrics. Build observability into your design from the start:

- **Model performance tracking:** Monitor accuracy drift, inference latency, and prediction confidence scores.
- **Data quality monitoring:** Track input data distribution changes that might affect model performance.
- **User interaction analysis:** Understand how users interact with AI features to identify improvement opportunities.

## Plan for abstraction and future flexibility

Design your system with abstraction layers that allow you to adapt as AI technologies evolve:

- **Model abstraction:** Abstract models behind consistent interfaces to enable swapping or upgrading without application changes.
- **Standardized protocols:** Prefer open, documented interfaces and formats such as OpenAPI for tools, ONNX for model portability, and OpenTelemetry for telemetry.
- **Capability-based design:** Design around capabilities rather than specific technologies to maintain flexibility.

## Externalize prompts and configuration

Treat prompts as configuration that should be externalized according to The Twelve-Factor App design principles:

- **Version control:** Keep prompts in version control with clear deployment tracking to correlate telemetry and safety results with specific prompt versions.
- **Role separation:** Enable business analysts and domain experts to tune prompts without code changes.
- **Safe deployment:** Implement controlled rollout processes for prompt changes and treat them as significant configuration updates.

## Plan for model deprecation

Foundation models eventually reach end-of-life and are retired by your model hosting platform. The models are replaced by models that perform better, are more cost effective, have updated training knowledge, and support new capabilities. Design abstractions that minimize the impact of your workload's future model transitions.

- **Provider abstraction:** Use abstraction layers that allow switching between model providers without application changes.
- **Version management:** Implement versioning strategies that support gradual migration between model versions.
- **Fallback strategies:** Design fallback mechanisms for when preferred models become unavailable.

For architecture design techniques that address model life cycle concerns, see Design to support foundation model life cycles.

## Containerize components

To ensure that your independently deployable components are self-contained and to improve deployment consistency, consider containerization as part of your design strategy. The following components should be containerized:

- **Microservices:** Containerize individual microservices that handle specific functions of the application, like data processing, model inference, and user authentication. This approach enables independent deployment and scaling and facilitates more efficient updates and maintenance.
- **AI models:** Containerize AI models to ensure that all dependencies, libraries, and configurations are bundled together. This approach isolates the model environment from the host system to prevent version conflicts and help ensure consistent behavior across different deployment environments.

- **Data processing pipelines:** Containerize data processing tasks that precede or follow model inference, like data cleaning, transformation, and feature extraction. This approach enhances reproducibility and simplifies the management of dependencies.
- **Infrastructure services:** Containerize services that provide infrastructure support, like databases and caching layers. This approach helps maintain version consistency and facilitates easier scaling and management of these components.

# Implement multi-layer caching strategies

A multi-layer caching approach can help improve performance and reduce costs in your AI applications. Consider implementing caching at multiple levels of your application stack:

- **Result and answer caching:** Use this approach to reuse responses for identical or semantically similar queries, if appropriate for the situation. In workloads where responses are cacheable, this approach can significantly reduce invocations to your model, which improves performance and reduces model invocation costs.
- **Retrieval and grounding snippet caching:** Cache frequently retrieved knowledge fragments and grounding data to avoid repeated database and search queries or data API operations.
- **Model output caching:** Cache intermediate model outputs that can be reused across requests.

When you design your caching approach in each layer, you need to understand what data is frequently accessed and how the application's users' roles and permissions might influence what data they can access. For caching to be effective, you must expect a healthy cache hit rate. Focus on caching for high-traffic paths like product catalog searches, while monitoring cache evictions.

- **Cache key components:** Cached values need to be tied to specific runtime factors within your workload. Include values such as tenant or user identity, policy context, model version, and prompt version in cache keys to ensure that a cached value is only being returned when appropriate for the request.
- **Time-to-live (TTL) policies:** Set appropriate expiration times based on data freshness requirements and content sensitivity.
- **Invalidation hooks:** Implement cache invalidation triggers for data updates, model changes, and prompt modifications.
- **User privacy protection:** Never cache user-private content unless properly scoped by key and policy. In general, caching works best for data that applies across multiple users.

Don't cache for a single user in most situations. Avoid caching user-specific responses as a valid response for all users. For example, a cached response for "How many hours of paid time off do I have left?" is only ever appropriate for the user that requests the data, even if other users present the same query.

> ⚠ **Risk:** Caching improves performance and reduces costs but introduces security and data freshness risks. These risks include **data leakage**, **stale data**, and **privacy violations**.

# Evaluate the use of orchestration and agents in generative AI solutions

Choose the appropriate coordination approach based on your application's requirements for determinism and exploratory capabilities.

## When to use orchestration

An orchestrator manages a workflow by coordinating the communication among the different components of the AI solution that would otherwise be difficult to manage in complex workloads. We recommend that you build an orchestrator into your design if your workload needs any of the following characteristics:

- **Predictable workflows:** Multi-step processes with well-defined sequences and decision points.
- **Compliance requirements:** Scenarios where you need to ensure that specific steps are performed for regulatory compliance.
- **Performance-critical paths:** Workflows where latency and resource usage must be tightly controlled.
- **Simple coordination:** Straightforward task delegation that doesn't require dynamic reasoning.

## When to use agent collaboration

An **agent** is a way to wrap, extract, and define intelligent behavior in your application. Agents provide context-bound functionality and can work with orchestration frameworks like Semantic Kernel, Autogen, or Microsoft Agent Framework.

Use agent-routed collaboration approaches for exploratory or composed tasks:

- **Multi-step reasoning:** Tasks that require planning and decision-making across multiple steps
- **Tool coordination:** Complex workflows that involve coordinating multiple specialized tools and services
- **Adaptive behavior:** Scenarios where the system needs to adjust its approach based on intermediate results or changing conditions
- **Context management:** Applications that need to maintain conversation state and user context across interactions

### Agent design considerations

When you design agent-based systems, consider isolation and boundaries, communication patterns, and tool abstraction strategies. Design clear boundaries between different agent capabilities to reduce blast radius and improve testability. Use established patterns like topic-queue systems for agent communication. Abstract tool capabilities by using standardized interfaces to enable agent flexibility.

> **ⓘ Important**
>
> Don't automatically add agents between the task to be completed and model calls. Evaluate whether the intelligence being delivered requires the complexity of agent patterns, or if direct model calls are sufficient for your use case. Agent layers add latency, expand the surface area, and complicate testing.

### Agent state persistence

For multi-turn conversations and long-running tasks, design agents that include durable state persistence so that they can retain context across requests, sessions, or deployment cycles. Use secure, shared storage such as Azure Cosmos DB, Azure Managed Redis, or Azure Table Storage to store relevant metadata, conversation history, and task progress. Scope persisted state to the minimal necessary information to reduce privacy risks and token overhead, and implement TTL policies to expire stale data. Ensure that agents can rehydrate state on resumption to continue workflows without repeating completed steps.

## Hybrid approaches

Consider hybrid designs where orchestrators delegate specific subtasks to agents.

- **Structured workflows with flexible steps:** Use orchestration for the overall workflow while allowing agents to handle complex individual steps.
- **Escalation patterns:** Start with deterministic orchestration, and escalate to agent-based reasoning when predetermined logic is insufficient.
- **Domain-specific reasoning:** Use orchestration for cross-domain coordination while employing specialized agents for domain-specific tasks.

>  **Trade-off:** Orchestration provides predictability and control but limits adaptability. Agent collaboration enables dynamic problem-solving but introduces variability and complexity.

# Implement AI gateways for policy enforcement

Use AI gateways in your design to provide gateway offloading, gateway routing, and gateway aggregation capabilities within your workload. Gateways are typically implemented by using platforms like Azure API Management or through custom-coded solutions, such as an Envoy or NGINX implementation. Gateways can be used to proxy requests to model providers, agent endpoints, tools, and knowledge stores.

## Core AI gateway capabilities

An AI gateway can address cross-cutting concerns and serve as a layer of abstraction and indirection from the target system. When you design an AI gateway strategy, consider the following capabilities:

- **Protocol normalization:** For shared AI capabilities across providers, normalize interfaces to a common protocol via the gateway to improve maintainability. For provider-specific features, use the gateway as a reverse proxy without normalization.
- **Authentication and authorization:** Enforce identity-based access control, and propagate user context to downstream services.
- **Rate limiting and quotas:** Implement per-user, per-application, or per-tenant rate limits and token quotas to prevent abuse and manage costs.
- **Request and response filtering:** Apply input validation, prompt filtering, and output safety checks at the gateway level.
- **Token and prompt caps:** Enforce maximum token limits and prompt size restrictions to prevent resource exhaustion by preemptively rejecting requests that represent more

demand on the system than desired or possible.

- **Model routing policies:** Route requests to appropriate models based on user permissions, request characteristics, or cost optimization goals. Gateways are often used to implement spillover between multiple models when capacity or pricing is consumed on one model but is available on another model deployment.
- **Header injection and transformation:** Add required headers, user context, and security tokens for downstream services.
- **Chargeback management:** Allocate charges across departments that share AI workload components, like a single Azure OpenAI instance.

> **Tip**
>
> For an in-depth look at how an AI gateway can be used for cross-cutting concerns and how a gateway introduces extra complexity into your architecture, see Access Azure OpenAI and other language models through a gateway.

## Multi-provider scenarios

AI gateways are especially valuable when you use multiple model providers or shared model hosting platforms. These gateways are often centralized resources within your organization that your workload is encouraged or required to depend on. A sufficiently complex workload might also benefit from building this layer of abstraction within its own design.

- **Unified interface:** Provide a single API surface that abstracts multiple back-end providers.
- **Failover and redundancy:** Route requests to alternate providers when primary services are unavailable.
- **Cost optimization:** Route requests to the most cost-effective provider based on request characteristics.

> **Important**
>
> Don't rely solely on SDK retries and timeouts. Enforce limits, authentication, quotas, and timeouts at the gateway and policy layer for comprehensive control.

# Use AI application design patterns

The industry defines several common design patterns for AI applications. You can use them to simplify your design and implementation. These design patterns include the following types:

- **Model ensembling:** This design pattern combines predictions from multiple models to improve accuracy and robustness by mitigating the limitations of individual models.
- **Microservices architecture:** This design pattern separates components into independently deployable services to enhance scalability and maintainability. It enables teams to work on different parts of the application simultaneously.
- **Event-driven architecture:** This design pattern uses events to trigger actions, which enables decoupled components and real-time processing to make the system more responsive and adaptable to changing data.

## Grounding and knowledge integration patterns

Many AI applications access external knowledge sources to provide accurate, up-to-date information. Treat retrieval as an agent tool or knowledge call behind a service that enforces authorization.

- **Knowledge routing:** Direct queries to appropriate knowledge sources based on query type, domain, or user context.
- **Context layering:** Build systems that can combine multiple knowledge sources and context types to provide comprehensive responses.
- **Dynamic grounding:** Implement systems that can adapt their knowledge sources and grounding strategies based on changing requirements or data availability.
- **Authorization-aware retrieval:** Ensure that grounding services enforce user permissions and tenant context when retrieving information.

For a deeper understanding of designing a purpose-built vector index that supports your workload-specific grounding data for semantic retrieval, see Design and develop a retrieval-augmented generation (RAG) solution.

## Multi-model routing patterns

Using a model router can improve your workload's availability by routing requests to a healthy model when another model is in an unhealthy state, or it can help improve the quality of responses by selecting the best model for a specific task in real time. Model routers add flexibility but introduce extra variability. Use them selectively:

**When to use model routers:** Use a model router when your workload can tolerate added variability and latency, the user experience expects breadth across model types, or you need to balance costs and capabilities across different models.

**When to avoid model routers:** Don't use a model router in scenarios where the workload needs precise answers optimized for specific tasks, when you use fine-tuned models with narrow service-level objectives (SLOs), or when deterministic behavior, including consistent performance, is critical for your use case.

When you add a model router to your application, prefer provider-native routing when available, and monitor performance impact and user satisfaction.

>  **Trade-off:** Model routers provide flexibility and cost optimization but add non-deterministic behavior and complexity to testing and troubleshooting.

If your workload is a multitenant application, see Guide to design a secure multitenant RAG inferencing solution for recommendations about accessing per-tenant grounding data.

## When to use design patterns

Consider using these design patterns for the following scenarios:

- **Complex workflows:** When you have complex workflows or interactions between multiple AI models, patterns like RAG or microservices can help manage complexity and ensure clear communication between components.
- **Scalability requirements:** If the demand on your application fluctuates, a pattern like microservices enables individual components to scale independently to accommodate varying loads without affecting the overall system performance.
- **Data-driven applications:** If your application requires extensive data handling, an event-driven architecture can provide real-time responsiveness and efficient data processing.

> ⓘ **Note**
>
> Smaller applications or proof of concepts (POCs) typically don't benefit from these design patterns. These applications should be designed for simplicity. Likewise, if you have resource constraints, such as budget, time, or headcount, using a simple design that can be refactored later is a better approach than adopting a complex design pattern.

# Choose the right frameworks, libraries, and protocols

The choice of frameworks and libraries is closely intertwined with application design. They affect performance, scalability, and maintainability. However, design requirements can limit your framework choices. For example, use of the Semantic Kernel SDK often encourages a microservices-based design where each agent or function is encapsulated within its own service. Consider the following factors when you choose frameworks and libraries:

- **Application requirements:** The requirements of the application, like real-time processing or batch processing, might limit the choice of framework. For example, if the application requires low latency, you might need to use a framework that has asynchronous capabilities.
- **Integration needs:** The design might require specific integrations with other systems or services. If a framework doesn't support the necessary protocols or data formats, you might need to reconsider the design or choose a different framework.
- **Team expertise:** The skill set of the development team can limit framework choices. A design that relies on a less familiar framework might lead to increased development time and complexity, so consider using a more familiar tool.
- **Community and support:** Consider the maturity and support ecosystem of the framework. Active communities and comprehensive documentation reduce implementation risks.
- **Performance characteristics:** Evaluate framework performance for your specific use case, including memory usage, startup time, and inference speed.

Adopt standardized tool protocols to improve governance and enable flexibility:

- **Use MCP-style servers:** Use MCP or another suitable standard to wrap business capabilities as discoverable tool servers.
- **Use specifications that have broad SDK support.** For example, define tools by using OpenAPI specifications for consistent interface documentation and validation.
- **Advertise capabilities:** Design tools to advertise their capabilities, which allows orchestrators to discover and route requests appropriately. For example, wrap enterprise resource planning (ERP) read and write operations as a tool server that advertises capabilities. This approach allows you to tune the ERP interactions without changing the agent logic.

## Protocol preferences

When you select protocols and standards, prefer open, documented interfaces over proprietary formats. There are a lot of emerging protocols in the industry. As you consider your agent-to-agent and control use cases, consider technical debt that can come from protocols that are under rapid evolution or deprecation. Layers of abstraction can help keep your design interoperable across different AI frameworks and providers.

# Design a security strategy for your workload's AI components

AI components require security considerations beyond traditional application security. Implement security measures across all application layers while using standard identity providers like Microsoft Entra ID for foundational authentication and authorization.

## Core security principles

Implement the following foundational security practices:

- **Standard authentication and authorization:** Use established identity providers and role-based access control (RBAC) systems.
- **Data lineage security with user context propagation:** Ensure that access controls are maintained throughout the data pipeline. Pass forward user identity by using on-behalf-of flows so that users can only access results based on data that they're authorized to see.
- **Audit trails:** Implement detailed logging of AI interactions for compliance and security monitoring.

## AI-specific security measures

Address security challenges that are unique to AI applications:

- **Prompt filtering and injection prevention:** Implement safeguards against prompt injection attacks that can manipulate AI behavior or extract sensitive information.
- **Safe tool usage controls:** When you use agents with tool access, implement controls to prevent harmful actions and validate tool usage before invocation.
- **Agent behavior monitoring:** Monitor agent actions and decisions to detect unusual or potentially harmful behavior patterns.

- **Model access control:** Implement fine-grained permissions for different models and capabilities within your AI system.
- **Response filtering and safety checks:** Filter AI outputs to prevent the generation of harmful, inappropriate, or sensitive content.

## Security-trimmed grounding

Go beyond preventing direct database access, and implement security-aware knowledge retrieval:

- **Identity propagation:** Pass Microsoft Entra group claims or equivalent identity information into knowledge tool calls so that grounding services never retrieve unauthorized data or media.
- **Group and access control list (ACL)-based trimming:** Implement filtering based on security groups and ACLs at the knowledge layer.
- **Auditable denials:** Log and audit logging request denials caused by insufficient permissions.
- **Authorization enforcement:** Require that code sits between agents and knowledge sources to enforce authorization policies.

For example, you might pass Microsoft Entra group claims into the knowledge tool call to ensure that grounding never retrieves unauthorized documents. This configuration guarantees cached results and ensures that AI responses respect user permissions.

# Consider nonfunctional requirements

Your workload might have nonfunctional requirements that pose challenges because of factors that are inherent to AI technologies.

- **Latency of model inferencing or timeouts:** AI applications often require real-time or near-real-time responses. Designing for low latency is crucial. It involves optimizing model architecture, data processing pipelines, and hardware resources. Implementing caching strategies and ensuring efficient model loading are also essential to avoid timeouts and provide timely responses.
- **Token or request throughput limitations:** Many AI services impose limits on the number of tokens or the throughput of requests, especially with cloud-based models. Designing for these limitations requires careful management of input sizes, batching of requests

when necessary, and potentially implementing rate-limiting or queuing mechanisms to manage user expectations and prevent service disruptions.

- **Cost and chargeback scenarios:** Designing for cost transparency involves implementing usage tracking and reporting features that facilitate chargeback models. These features enable your organization to allocate costs accurately across departments. Chargeback management is typically handled by an API gateway, like API Management.
- **Model accuracy and drift:** AI models can degrade over time as data patterns change. Design monitoring systems to detect accuracy drift, and implement automated retraining pipelines when necessary.
- **Compliance and explainability:** Some industries require explainable AI decisions. Design your system to capture and provide reasoning for AI-generated outputs when required.
- **Data privacy and residency:** Ensure that your design meets data protection requirements, including the ability to delete user data and comply with geographic data residency rules.

## Next step

Design area: Application platform

# Application platform for AI workloads on Azure

Article • 11/18/2024

You must carefully consider the application hosting platform that your AI workload is deployed on to ensure that you can maximize efficiency, the security of operations, and reliability.

This design area covers several types of applications that might be relevant to your AI workload:

- Exploratory data analysis (EDA)
- Model training and fine-tuning
- Inferencing

This article provides guidance for selecting the best platform for each of these functions to meet your business needs. There are also general recommendations that you can apply to all of these functions.

## Recommendations

Here's the summary of recommendations provided in this article.

Expand table

| Recommendation | Description |
| --- | --- |
| **Reuse tools**. | Start by evaluating the tools you already use to understand whether they can be reused for your AI workload. If they support the required functionality and can meet your requirements for reliability, security, cost, and performance, bringing in a new tool might not be worth the cost and effort. |
| **Consider compliance requirements for your data and the regions that you plan to deploy in**. | You might need to limit the regions you deploy in or isolate parts of your workload from each other to meet compliance requirements. Going into your designing phase with this information can help protect you from needing to redesign later. |
| **Minimize building**. | Consider platform as a service (PaaS) or software as a service (SaaS) solutions to minimize the operational burden that building your own solution introduces, like patching and other maintenance. Minimizing the Day-2 burden required for the new |

| Recommendation | Description |
| --- | --- |
| | technology simplifies your adoption. Many AI functions are complex, so we don't recommend building your own platform. |
| **Understand your quotas and limits**. | When you design for the use of PaaS or SaaS solutions, understand any quotas or limits that apply. Your ability to scale out to meet high traffic demands might be affected by quotas or limits, so you might need to adjust your design to minimize that risk. |
| **Deploy in the same region**. | Try to deploy all related resources in the same region to reduce latency and to simplify the design. |
| **Practice safe deployment**. | In general, you should treat the APIs for your AI workload the same as any other API in your environment. All APIs should be placed behind a gateway and all code should be handled with the same safe deployment practices as every other code asset. |
| **Establish performance benchmarks through experimentation**. | Every AI workload is different, and the amount of compute that you need depends on your use case. Determine the amount and types of compute that are optimal for your workload by conducting thorough benchmark testing. This guide helps you choose a platform, but you'll only know which SKUs are appropriate for your workload after benchmark testing. |

# Considerations for the EDA platform

EDA is a common preliminary function that data scientists perform before modeling or statistical analysis. It can therefore be considered a development phase, which means that targets for reliability and performance might be significantly lower than those for production resources and maintaining productivity is the more important factor.

This section provides guidance on capabilities to consider when you select an EDA platform solution.

## Functional requirements

When you evaluate an EDA platform, consider the following questions:

- **Does the platform support transient usage?**

  The platform should support transient workspaces and compute, which means that you should be able to stop the necessary resources when they aren't being used. This capability helps control costs. EDA jobs are typically interactive, so users need to be able to start VMs and stop them as they run jobs.

- **Does the platform support compute optionality?**

  The platform should enable on-demand access to GPUs as needed and provide various compute options to help right-size the platform.

- **Does the platform support MLflow?**

  Your EDA platform should make it possible to choose a technology that enables integration with MLflow for tracking your experiments. We recommend MLflow as a model development, deployment, and management protocol because it provides the following benefits:
  - *Experiment tracking*. MLflow allows you to track experiments by recording parameters, metrics, and artifacts. This capability is essential during EDA so that you can keep track of different data preprocessing steps and feature engineering techniques and their impacts on model performance.
  - *Reproducibility*. Because it logs all the details of your experiments, MLflow helps ensure that you can reproduce your results, which is critical for validating findings.
  - *Data and model versioning*. MLflow helps with versioning datasets and models, which makes it easier to manage different versions of data transformations and tested models.
  - *Collaborative work*. MLflow provides a centralized platform where data scientists can share their experiments and results, which facilitates collaboration and knowledge sharing.

## Nonfunctional requirements

Consider these questions as well:

- **How can the platform help control costs?**

  The platform should enable data scientists to perform their work according to their schedule requirements, but it should be right-sized to ensure that cost expectations are met.

- **What security requirements must be followed for the platform?**

  The data used during your EDA phase will probably be production data, which requires you to follow production practices to secure that data and monitor the platform. To that end, your platform should support all necessary security controls, including:
  - *Access and authorization*.
  - *Encryption at rest and in transit*.

- *Regional data protection requirements*.
- *Robust monitoring and alerting functionality, including logging and auditability*.
- *Private networking access to centralized repositories for container images, data, and code assets*.

### Tools

Use an Azure Machine Learning compute instance with team-level file shares as your EDA platform. One exception to this is if your team or organization is already using a suitable hosting platform, like GPU-enabled compute clusters in Databricks, for example. In that case, it might be more appropriate to stay on that platform.

> **Note**
>
> Don't build a full EDA platform unless you need to. GPU-optimized compute is expensive and isn't appropriate if your use case doesn't require it.

## Considerations for the model training and fine-tuning platform

When you move to model training and fine-tuning, you'll probably need high-performance GPU-optimized compute for the compute-intensive work that's required by those activities. Reliability typically isn't as important as performance because most of this work occurs behind the scenes. If high reliability is a requirement, evaluate whether spreading the workload across availability zones or regions is necessary. High reliability becomes more important when model freshness is updated frequently, which requires training to be completed on a tighter schedule. Your RTO should determine the reliability design that you choose.

The guidance in this section applies to both model training and fine-tuning. Unless you're forced to use separate platforms for these functions, you should use a single platform.

### Functional requirements

When you evaluate platforms for model training and fine-tuning, consider the following questions:

- **Does the platform support transient usage?**

Like EDA activities, model training and fine-tuning are typically not run full-time, so you should use a platform that can be stopped when it's not in use to help control costs. Unlike EDA, however, model training is typically a batch process, so the compute is only needed when the batch runs and can be shut down until the next run.

- **Does the platform provide orchestration?**

  Because of the complexity required in managing the compute for model training and fine-tuning, we recommend an orchestrator.

- **Can existing technologies in your environment be part of the solution?**

  If your existing data platform has machine learning capabilities, like Azure Databricks does, you can use it for certain steps, like data transformation and feature engineering, training, fine-tuning, and other steps in Machine Learning. Combining technologies can help you minimize the cost and complexity involved in using a data platform for functions it might not be ideally suited for.

## Nonfunctional requirements

Consider this question as well:

- **What's the tolerable tradeoff between cost and performance?**

  Given the high-performance, GPU-optimized compute requirements, ensure that you test and benchmark your training and fine-tuning extensively to determine the ideal SKU that balances performance against costs.

## Tools

We recommend Azure Machine Learning for the model training and fine-tuning platform because it provides orchestration functionality with support for batch compute. There are two compute options to evaluate:

- Serverless compute is ideal for short, infrequent runs that can tolerate noisy neighbor effects. You can choose either standard pricing or spot pricing. Spot pricing is only recommended for highly interruptible training. Don't use serverless compute for full-time operations. The costs can escalate quickly.
- Compute clusters give you significant control over available hardware and are tuned for parallel or distributed training.

> **Note**

For foundation models, your choice of model hosting platform might limit your fine-tuning options. For example, using Azure OpenAI Service for model hosting limits your fine-tuning options to the built in Azure OpenAI fine-tuning functionality.

# Considerations for the model hosting and inferencing platform

Model hosting and inferencing functions make up the serve layer of the AI workload. These functions are performed with endpoints that are specific to the software that you use. Model-serving software solutions, like NVIDIA Triton, TorchServe, and TensorFlow Serving, are essentially Python SDKs that front a model with an API and add functionality that's specific to the solution. You can choose your hosting platform based on your choice of software or choose your software based on your choice of hosting platform.

When you use SaaS or PaaS solutions with prepackaged models, like the large language models that are available in Azure OpenAI, you have few or no opportunities to select a serving software. Instead, the service that you're consuming provides an API. This reduces flexibility in the process for creating a model deployment, which can provide advantages and disadvantages. For example, it can streamline the development process of your workload. On the other hand, it reduces flexibility in how your application can call and interact with the model.

Fundamentally, the APIs for the serve layer are microservices, so you should follow the same practices for these APIs that you follow for other microservices in your environment. They should be containerized, bulkheaded from other services, and have their own lifecycles that are independent of other services and APIs. Keep in mind, however, that serve layer APIs generally require significantly more GPU-based compute power and larger container images than traditional APIs.

This section provides guidance on capabilities to consider when you select a model hosting and inferencing platform.

## Functional requirements

When you evaluate platforms for model hosting and inferencing, consider these questions:

- **Does your workload require batch or online inferencing?**

The inferencing endpoints are used either for batch or online inferencing processes, and the inferencing method helps determine the right hosting platform. Batch inferencing is best hosted on a platform that supports transient usage and allows the compute to be shut down when it's not being used. Online inferencing is best hosted on a platform that supports elastic compute utilization, which scales automatically based on the load at any given time.

- **Does the platform support traceability?**

  Traceability is critical for maintaining the integrity of the models used in your workload. It's important to know information about the model, like the current version, who deployed it, when it was deployed, and the model's data lineage.

  Apply meaningful tags to images in your container registry to ensure that your model hosting service pulls a specific version that the team can easily identify. This approach helps with data governance by reducing the risk of outdated or incorrect models being used in production.

- **Will your hosting platform be a centralized resource?**

  Many organizations use a centralized model hosting platform that's used by different teams for their own workloads. If your hosting platform is centralized, you should consider whether you need support for chargeback. This functionality allows you to track platform utilization by team and workload.

## Nonfunctional requirements

Consider these questions as well:

- **What are the reliability requirements for the platform?**

  Serve layer APIs are production resources, so you should apply the same reliability requirements to them that you apply to other workload flows that match their criticality rating. If their criticality requires high availability, your hosting platform should support availability zones or a multiregion design.

- **What networking controls are required for the platform?**

  Determine whether you need private networking or an egress firewall to provide protection for the platform.

- **What are the identity and access security requirements for the platform?**

  Determine the identity and access controls that are required for your endpoints. Consider whether you need native role-based access control (RBAC) or built-in

support for your identity and access platform, for example, Microsoft Entra ID.

- **What monitoring capabilities does the platform support?**

  Determine the required monitoring capabilities for your endpoints. Depending on the platform, you might have limited access to logs and metrics, which might limit your ability to audit activities or detect malfunctions.

- **What are the performance requirements for the platform?**

  Inference latency is a common concern, and different platforms have different performance profiles. Serverless and PaaS services that use a utility model can be affected by the noisy neighbor problem and often have no throughput guarantees. On the other hand, the same platforms might offer a self-hosted option that provides guaranteed throughput with a pre-purchasing model. You can also consider self-hosting on Kubernetes for more predictable latency behavior.

  Be aware of service limits and quotas that might affect your performance, like those for Azure OpenAI. Often these quotas and limits are aggressively set to meet capacity demands, so if your choice of platform doesn't provide the performance that you require, you might need to adopt strategies to spread the compute demand across instances.

  Advanced architectures can combine multiple deployments to achieve fixed throughput for the bulk of the workload and bursting capabilities for a more flexible compute.

## Tools

### Batch inferencing

- If you're performing inferencing on data that resides in a platform that supports model hosting, like Databricks, consider using that platform for inferencing. Be sure to isolate the inferencing compute from other functions performed by the data platform.
- We recommend Azure OpenAI Batch API for foundation models.
- For non-foundation models, consider these recommendations:
  - Consider using Azure Machine Learning batch endpoints for the following scenarios:

    - You need to perform inferencing on a large dataset that's distributed in multiple files and you don't require low latency.
    - You need to perform long-running batch operations over large datasets and can take advantage of parallelization.
    - You need to deploy pipeline components for batch processing.
  - If you need to run Spark jobs for distributed data processing, consider using Azure Synapse Analytics, Databricks, or Machine Learning serverless Spark compute.
  - If none of these scenarios apply, we recommend Machine Learning batch endpoints.

## Online inferencing

- Evaluate platform PaaS and serverless solutions as a first step. These services are typically the easiest to adopt and manage because they simplify your design and minimize operational burden. For example, Azure OpenAI is a good choice for foundation models.
  - Consider using the Azure Machine Learning Serverless API to aggregate endpoint access even if you use Azure OpenAI or another foundation model hosting solution.
- Consider Machine Learning with managed compute clusters when PaaS or serverless solutions aren't the best fit. Compute that's managed by Machine Learning supports traffic splitting and mirroring for A/B testing, debugging, and robust auditing. Because the compute is managed by the service, Day-2 operations are easier when you self-host your model. Managed compute also offers a wide selection of compute configurations and scaling capabilities.
- If you choose to self-host your model on an Azure Kubernetes Service (AKS) cluster that's attached to Machine Learning or another container-based platform, be sure that the node pool is isolated from other APIs or any other workloads on the cluster to achieve predictable performance and to optimize security. Avoid using GPU-based or GPU-optimized compute for anything other than your AI workload functions to reduce costs. Instead, establish your performance baseline through testing and right-size your compute to meet your performance requirements without over-provisioning.
- You can also self-host your model by using infrastructure as a service (IaaS) solutions, like Azure Data Science Virtual Machine.

# Considerations for the orchestration platform

Orchestration, in the context of AI workload application platforms, refers to tools like prompt flow in Machine Learning and Azure AI Studio. These tools are designed to streamline the entire development cycle of AI applications by automating many common workflow functions.

## Nonfunctional requirements

As with all other production workloads in your cloud estate, when you evaluate orchestration tools, you need to consider:

- **Reliability, security, and monitoring.** Orchestration tools should adhere to reliability, security, and monitoring standards for production workloads.
- **Performance.** Orchestration tools don't require GPU-optimized or GPU-based compute, consider general-purpose SKUs.
- **Cost optimization.** Orchestration tools are *always-on*, consider elastic compute options to minimize utilization costs.

## Tools

- Prefer an off-the-shelf solution like prompt flow. Determine whether its capabilities match your orchestration needs before you look into custom hosting with tools like LangChain or Semantic Kernel.
- Host endpoints for solutions like prompt flow on Machine Learning with compute instances or on AKS with self-hosting.

# Next steps

Design area: Training data design

# Feedback

Was this page helpful? Yes No

# Design training data for AI workloads on Azure

When you design data for AI functionality in applications, consider both non-functional requirements, such as operability, cost, and security, and functional requirements that are related to data ingestion, preparation, and validation.

Data design and application design can't be decoupled. Application design requires that you understand use cases, query patterns, and freshness requirements. To address business requirements that drive the need for using AI, the application might need output from discriminative models, generative models, or a combination of model types.

To produce meaningful results, AI models need to be trained. Model training involves teaching a model to classify or predict new or unseen situations. The training data must be tailored to the specific problem and workload context.

Supervised training involves providing the model with labeled samples. This type of training is useful when the desired outcome is clear. In contrast, unsupervised learning allows the model to identify patterns and relationships within the data without guidance on the expected output. During training, the algorithm type and its parameters are adjusted to control how the model learns. The approach varies depending on the type of model, which can include neural networks, decision trees, and others.

For example, image detection models are typically trained on tasks like object detection, facial recognition, or scene understanding. They learn from annotated images to identify specific objects or features. Other common examples include fraud detection algorithms and price-point prediction models. These models learn from historical financial data to make informed decisions.

This article focuses primarily on the preceding use case, where models are trained *before* they can give meaningful input to the application. The article includes guidance on data collection, processing, storing, testing, and maintenance. Data design for exploratory data science or business intelligence via AI isn't covered. The goal is to support training needs through strategies that are aligned with workload requirements by providing recommendations on the training data pipeline of an AI workload.

For information about data design for AI models that require context *during* inferencing, see Grounding data design.

> **Important**

Expect data design to be an iterative process that's based on statistical experimentation. To reach an acceptable quality level, adjust training data, its processing, model feature development, and model hyperparameters (when possible). This experimentation loop typically happens both during initial model training and during ongoing refinement efforts to address data and model drift over the lifespan of the feature in the workload.

# Recommendations

Here's the summary of recommendations provided in this article.

Expand table

| Recommendation | Description |
| --- | --- |
| **Select data sources based on workload requirements.** | Factor in available resources and whether the data source can help you reach the acceptable data quality for model training. Cover both positive and negative examples. Combine diverse data types to achieve adequate completeness for analysis and modeling. Consider techniques like Synthetic Minority Oversampling Technique (SMOTE) for data scarcity or imbalance.<br><br>▪ Data ingestion and analysis |
| **Conduct data analysis on the collected data early.** | Perform analysis processes, such as Exploratory Data Analysis (EDA), offline. Consider the costs and security implications. For small datasets without resource constraints, you can consider performing analysis at the source.<br><br>▪ Data collection store |
| **Maintain data segmentation, if business and technical requirements call for it.** | If you use data sources that have distinct security requirements, create separate pipelines for each model. Establish access controls to limit interaction with specific data subsets.<br><br>▪ Data segmentation |
| **Preprocess data to make it meaningful against training goals.** | Refine the quality of ingested data by filtering noise, rescoping the data, addressing duplicates, and standardizing diverse formats.<br><br>▪ Data preprocessing |
| **Avoid training on stale data.** | Monitor for data drift and concept drift as part of your inner and outer operational loops to maintain the accuracy and reliability of models over time. Regularly update training data with new observations. Define conditions that trigger model retraining and determine update frequency. |

| Recommendation | Description |
| --- | --- |
| | ▪ Data maintenance |

# Types of data

To build predictive power in models, you need to collect data, process it, and feed it to the model. This process is usually conceptualized as a pipeline that's broken into stages. Each stage of the pipeline might deal with the same data set, but it might serve different purposes. Typically, you handle data of these types:

- *Source data* is point-in-time observation data. It can also be data that can be labeled to serve as a potential input to the data pipeline.

  This data is usually obtained from production or from an external source. These data sources can be in storage accounts, databases, APIs, or other sources. The data can be in various data formats, like OLTP databases, unstructured documents, or log files. This data serves as a potential input to the data pipeline.

- *Training data* is a subset of source data that's used for providing samples to the model. The samples are descriptive precalculated data that help the model learn patterns and relationships. Without this data, the model can't generate relevant output.

- *Evaluation data* is a subset of the source data that's used to monitor and validate the performance of a machine learning model during training. It's distinct from training and test data and is used to periodically evaluate the model's performance during the training phase and guide hyperparameter tuning. For more information, see Model evaluation.

- *Testing data* is used to validate the predictive power of a trained model. This data is sampled from source data that wasn't used for training. It contains observations from production so that the testing process is conclusive. From a data design perspective, you need to store this data. For information about testing models, see the Testing design area.

Data can be structured (databases, spreadsheets), semi-structured (JSON, XML), or unstructured (text documents, images). *Multi-modal data* combines different data types like text, images, audio, or video within the same training dataset. When working with multi-modal data, preserve cross-modal relationships and maintain semantic connections between different data types throughout processing and feature engineering. Ensure synchronized data processing so that related information across different modalities remains aligned during preprocessing and training.

In some cases, information that's provided by users during interactions with the application can eventually become source data. In general, we recommend that user input used this manner is of high quality. Otherwise, the need to continuously handle quality issues downstream can become problematic. Guidance about handling user data isn't covered in this article.

# Data ingestion and analysis

Training data is collected within a predetermined window that has sufficient representations for training the type of model that you select. For example, when you train a binary classification model, training data must include representations of what is the case (positive examples) and what isn't the case (negative examples). For training data to be meaningful, conduct EDA early during feature design.

EDA helps analyze source data to identify characteristics, relationships, patterns, and quality issues. You can conduct EDA directly at the source data store or replicate data into centralized stores, like a data lake or data warehouse. The outcome of the process is to inform data collection and processing for effective model training.

> ⓘ **Note**
>
> Although EDA is a pre-production process, it uses data that's sourced from production. Apply the same level of control to this process as you would for production.

Following are some considerations for collecting data in preparation for model training.

## Data sources

Data can be collected from these sources:

- **Proprietary data** is created or owned by the organization. It's not intended for public consumption. It serves internal purposes.
- **Public sources** are accessible to anyone. These sources include websites, research papers, and publicly shared databases. It might be specific to a niche area. For example, content from Wikipedia and PubMed are considered publicly accessible.
- **User-generated data** includes information captured from user interactions, expert feedback, and collaborative workflows. This proprietary data supports continuous learning architectures where are updated incrementally as new information becomes available. Design your data pipeline to accommodate feedback loops while maintaining data quality standards and preventing degradation from low-quality inputs.

Your choice of data sources depends on workload requirements, available resources, and the quality of the data that's acceptable for training the model. Imbalanced datasets can lead to biased models, so you need to design data collection to get sufficient samples of representative data. You might need to oversample minority data or undersample majority data. If the data is scarce or imbalanced, consider techniques like SMOTE and Synthetic data generation.

## Data collection store

There are two main options for collecting source data:

- Querying the data at the data source
- Copying the data to a localized data store and then querying that store

The choice depends on workload requirements and the volume of data. If you have a relatively small amount of data, the source system might handle your raw queries directly. However, the common practice is to query and analyze from the localized store.

>  **Tradeoff.** Although localized data stores might facilitate analysis and the training process, you also need to balance costs, security, and model requirements.
>
> Duplicating data incurs storage and compute costs. Maintaining a separate copy necessitates additional resources. Local copies might contain sensitive information. If it does, you need to protect the data by using regular security measures.
>
> If you use production data for training data, it must be subject to all the original data classification constraints of that data.

Data can be provided to the training process (push mode) or the process itself can query the data source (pull mode). The choice depends on ownership, efficiency, and resource constraints.

When data is pushed to the workload, it's the responsibility of the data source owner to provide fresh data. The workload owner provides a suitable location in their localized data store to store the data. This approach applies to proprietary data that's owned by the organization, not to public sources.

There are two approaches that you can use for pulling data. In one approach, the workload queries against the data store, retrieves necessary data, and places it in the localized store. Another way is to perform real-time queries in memory. The decision depends on data volume and available compute resources. For smaller datasets, in-memory retrieval might be sufficient for model training.

Regardless of whether you use push or pull mode, avoid training models on stale data. The frequency of data updates should align with workload requirements.

### Data segmentation

Workload-specific requirements might necessitate data segmentation. Here are some potential use cases:

- **Security requirements** often drive segmentation decisions. For instance, regulatory constraints might prevent exporting data across geopolitical regions. If your application design allows the use of separate models, data design incorporates separate data pipelines for each model.

  However, if a single model is used, segmented data sources feed into that model. You need to train the model on data from both geographies, which potentially adds complexity.

  Whether the application is using a single model or multiple models, preserve security measures on each data segment so that it's protected with the same level of rigor as data at the origin.

- **Data freshness rate** can be a factor for separating data. Data from different sources might refresh at varying time intervals. If the data changes, retraining becomes necessary. Segmentation enables granular control of the data lifecycle. Consider using separate tables or pipelines for different data segments.

Regardless of the use case, when data is segmented, access controls are key. Data professionals, like data engineers and data scientists, explore available source data to understand patterns and relationships. Their insights contribute to training models that predict outcomes. Establish access controls to ensure that only authorized users can interact with specific data subsets. Apply least privilege on data that's considered to be relevant. Collaborate with data owners to set up appropriate permissions.

## Data preprocessing

In a real-world scenario, source data isn't simply stored for AI scenarios. There's an intermediate process that prepares data for training. During this stage, data is stripped of noise, making it useful for consumption. When dealing with source data, data scientists engage in a process of exploration, experimentation, and decision making. Their primary goal is to identify and extract parts of the source data that holds predictive power.

The preprocessing logic depends on the problem, data type, and desired outcomes. Following are some common techniques for preprocessing. This list isn't exhaustive. The actual criteria for your workload will be driven by business requirements.

- **Quality**. Preprocessing can help you ensure that training data is stripped of noise. The goal is to ensure that every row in your training data represents a clear observation or a good example that's relevant to your use case and to eliminate observations that lack quality or predictive power. For example, if you collate product reviews, you might choose to eliminate data that's too short. You need to discover what data quality produces meaningful predictive results.

- **Rescoping**. Source data fields that are too specific can restrict predictive powers. For example, consider an address field. Broadening the scope from full address (house number and street name) to a higher level, like city, state, or country/region, might be more relevant.

- **Deduplication**. Eliminating redundancy can ensure that your training data remains accurate and representative. In certain cases, the frequency with which an observation is made isn't relevant. For example, when you scan logs, if a log entry appears 1,000 times, that indicates its frequency. It doesn't necessarily imply that it's a more serious error than a log that occurred only once. This type of redundancy can introduce noise.

- **Sensitive data handling**. Eliminate personal data unless it's absolutely vital to the model's predictive power in a way that can't be achieved through anonymization. Training data should be effective without compromising privacy. If the data provides value, you need to be aware of the ethical considerations of handling sensitive data. For more information, see Responsible AI.

- **Standardized transformation**. Domain experts consider the preceding techniques to be a core part of feature engineering. Broad scope and diverse source data eventually need to merge into feature stores where features are organized (for example, into feature tables) for the explicit purpose of training models. After you select predictive data for training, transform the data to a standardized format. Standardization also ensures compatibility with the training model.

  Converting images into text representations is a form of transformation. For instance, you might convert scanned documents or images to machine-readable text.

  To ensure compatibility with models, you might need to adjust orientations or aspect ratios of images to match the model's expectations.

> ⓘ **Note**

Mixing large amounts of structured and unstructured data can increase processing time. Workload teams should measure the impact of processing diverse formats. As the window between retraining efforts becomes shorter, the amount of time spent on preprocessing becomes more critical.

# Feature store design

Feature stores serve as centralized repositories for storing, managing, and serving engineered features for machine learning models. They provide consistency across training and inference phases while enabling feature reuse across multiple models and teams. Organizations can implement multiple approaches to implement the feature stores:

- **Centralized feature stores** provide a single source of truth for all features across an organization. All teams contribute to and consume from a shared feature repository. This approach promotes consistency and reduces duplication but requires strong governance and standardization practices.
- **Distributed feature stores** allow teams to maintain their own feature repositories while providing mechanisms for controlled sharing. This pattern offers greater autonomy for individual teams but requires careful coordination to prevent feature inconsistencies across models.
- **Hybrid approaches** combine centralized governance with distributed implementation. Common features are managed centrally while domain-specific features remain with individual teams.

Workload teams in organizations with strong data governance teams and standardized ML practices will likely follow the centralized approach, or a hybrid model if they need to comply with the organization standards while still having some autonomy. Distributed patterns suit organizations where teams have distinct data requirements and operate with high autonomy.

## Design considerations

- Design clear separation between feature definition, storage, and serving layers
- Organize features into logical groupings that align with business domains and model requirements
- Establish consistent naming conventions and schema evolution practices
- Make features immutable once published, with new versions created for changes
- Integrate with data preprocessing, model training, and model serving infrastructure
- Implement automated checks for data drift, feature distribution changes, and quality degradation

# Foundation model fine-tuning

Training data for foundation model fine-tuning follows different patterns than traditional model training from scratch. Instead of training entire models, you're adapting pre-trained capabilities to specific domains or tasks.

**Domain-specific fine-tuning** requires high-quality, task-relevant training examples that represent the specific patterns you want the model to learn. The volume of training data is typically smaller than traditional training scenarios, but the quality and representativeness become more critical.

For example, fine-tuning a general language model for medical documentation requires training examples that accurately represent medical terminology, clinical reasoning patterns, and documentation standards specific to your organization or domain.

**Task-specific adaptation** focuses training data on particular capabilities or output formats. This approach might involve training the model to follow specific instructions, produce structured outputs, or integrate with particular workflows.

Consider how your training data pipeline can efficiently provide curated, high-quality examples for fine-tuning while maintaining traceability and version control for different model variants.

# Data retention

After you train a model, evaluate whether to delete the data used for training and rebuild the model for the next training window.

If the data remains relatively unchanged, retraining might not be necessary, unless model drift occurs. If the accuracy of prediction decreases, you need to retrain the model. You can choose to ingest the data again, preprocess, and build the model. That course of action is best if there's a significant delta in data since the last training window. If there's large volume of data and it hasn't changed much, you might not need to preprocess and rebuild the model. In that case, retain data, do in-place updates, and retrain the model. Decide how long you want to retain training data.

In general, delete data from feature stores to reduce clutter and storage costs for features that have poor performance and that are no longer relevant to current or future models. If you retain data, expect to manage costs and address security issues, which are typical concerns of data duplication.

# Lineage tracking

*Data lineage* refers to tracking the path of data from its source to its use in model training. Keeping track of data lineage is essential for explainability. Although users might not need detailed information about data origins, that information is crucial for internal data governance teams. Lineage metadata ensures transparency and accountability, even if it's not directly used by the model. This is useful for debugging purposes. It also helps you determine whether biases are introduced during data preprocessing.

Use platform features for lineage tracking when you can. For example, Azure Machine Learning is integrated in Microsoft Purview. This integration gives you access to features for data discovery, lineage tracking, and governance as part of the MLOps lifecycle.

# Data maintenance

All models can become stale over time, which causes a model's predictive power or relevance to decay. Several external changes can cause decay, including shift in user behavior, market dynamics, or other factors. Models trained some time ago might be less relevant because of changing circumstances. To make predictions with better fidelity, you need recent data.

- **Adopting newer models**. To ensure relevance, you need an operational loop that continuously evaluates model performance and considers newer models, which keeps the data pipeline minimally disruptive. You can alternatively prepare for a bigger change that involves redesigning the data lifecycle and pipeline.

  When you choose a new model, you don't necessarily need to start with a new data set. The existing observations used for training might remain valuable even during a model switch. Although new models might reveal narrower scenarios, the fundamental process remains similar. Data management approaches like feature stores and data meshes can streamline the adoption of new machine learning models.

- **Trigger-based vs. routine operations**. Consider whether model retraining should be triggered by specific events or conditions. For example, the availability of new, more relevant data or a drop in relevancy below an established baseline might trigger retraining. The advantages of this approach are responsiveness and timely updates.

  Maintenance can also be scheduled at regular fixed intervals, like daily or weekly. For fail-proof operations, consider both approaches.

- **Data removal**. Remove data that's no longer being used for training to optimize resource use and minimize the risk of using outdated or irrelevant data for model training.

  *The right to be forgotten* refers to an individual's right to have their personal data removed from online platforms or databases. Be sure to have policies in place to remove personal data that's used for training.

- **Data retention**. In some situations, you need to rebuild an existing model. For example, for disaster recovery, a model should be regenerated exactly as it was before the catastrophic event. We recommend that you have a secondary data pipeline that follows the workload requirements of the primary pipeline, like addressing model decay, regular updates via trigger-based or routine operations, and other maintenance tasks.

>  **Tradeoff.** Data maintenance is expensive. It involves copying data, building redundant pipelines, and running routine processes. Keep in mind that regular training might not improve answer quality. It only provides assurance against staleness. Evaluate the importance of data changes as a signal to determine the frequency of updates.

Make sure that data maintenance is done as part of model operations. You should establish processes to handle changes via automation as much as possible and use the right set of tools. For more information, see MLOps and GenAIOps for AI workloads on Azure.

## Next steps

Design area: Grounding data design

---

Last updated on 10/30/2025

# Grounding Data Design for AI Workloads on Azure

05/28/2025

For AI applications, the Well-Architected Framework approach to data design must address nonfunctional requirements like operability, cost, and security and adhere to the core principles of Azure Well-Architected Framework pillars. It should also consider functional requirements such as data ingestion, preparation, and validation.

The AI model that you choose affects subsequent data design decisions. This article discusses key architectural considerations for foundation models that need augmentation to enhance result relevance. These models are typically generative.

Generative AI models are prebuilt or pretrained, which allows you to immediately use them without making modifications. However, out-of-the-box models often don't meet specific workload requirements. To address this issue, models are augmented with context-specific data to improve their performance. For example, you can use the GPT model in various use cases. These applications include retrieving information from documents, providing IT Helpdesk support, and summarizing complex information. To use foundation models to meet your specific needs, it's important to understand these considerations.

> **Important**
>
> Data design is an iterative process that's based on statistical experimentation. Generative AI applications send queries to the model that includes the prompt and context data. To refine data design, the prompt and context data should both be iterated. The iterative process should include preprocessing, selecting embeddings, and chunking. These steps help create data that's suitable for an index. For more information, see Designing and developing a retrieval-augmented generation (RAG) solution.
>
> As you experiment and iterate, keep the consumption use cases in mind. Adjust the data design based on actual query patterns. Determine what's acceptable through refinement and testing.

In a solution, you can use a combination of generative AI and discriminative AI models to fulfill the workload requirements. For more information about training data, see Training data design.

## Recommendations

Here's the summary of recommendations provided in this article.

Expand table

| Recommendation | Description |
| --- | --- |
| **Anticipate user queries**. | Understand the expected types of questions related to your source data and their expectations of freshness. This understanding helps you design your data pipelines and indexes to provide relevant grounding data. |
| **Externalize data to a search index**. | Instead of querying directly from the source system, use a search index. Evaluate different index technologies based on workload requirements. Create a capability matrix to assess the best fit for your needs. Consider powerful search index technologies like Elasticsearch or AI Search.<br><br>▪ Indexing |
| **Develop an ingestion strategy**. | Develop a comprehensive index management strategy that covers data ingestion and preprocessing. Remove noisy or irrelevant data by addressing inconsistencies and duplicates and standardizing to a common schema. Convert source formats and types to data types that facilitate querying and analysis.<br><br>▪ Data preparation<br>▪ Data volume rescoping |
| **Design your index for maximum relevancy**. | Enable features like filtering, sorting, and metadata handling on specific fields to improve query efficiency. For instance, label fields as searchable only if you intend to search on them. To avoid unnecessary storage costs, don't make every field retrievable without a specific use case.<br><br>▪ Schema design<br>▪ Index capabilities<br>▪ Efficient querying |
| **Update your index to prevent inferencing on stale data**. | When you update an index, consider adopting a side-by-side deployment strategy for maintenance. Rebuilding the index ensures that deletions and updates are handled because the index becomes a fresh data set. This approach allows thorough testing of the data before making the index live. When you make changes to indexes, coordinate schema modifications with code updates. This practice ensures seamless transitions.<br><br>▪ Index maintenance |

# Types of data

You can augment generative AI models by using context data during inference, or optimize them further through a fine-tuning process. Both approaches need supplementary data that provides the model with more context. The model uses that context to answer the user query

and forms the answer according to the expectations. Typically, you use the following data types:

- **Source data** is existing data in production. This data can be structured, such as data in databases, or semi-structured, like JSON files. It can also be unstructured, like documents, images, and audio files.

- **Grounding data** is information provided to a language model at inference time to help it generate responses that are more accurate and relevant to a user's query. This process of *grounding the model* involves supplementing the model with data that was not part of its original training. The grounding data can come from internal systems, such as databases, or from external sources. It's combined with the user's query to construct the prompt that is sent to the model during a specific inference call.

  This data should be quickly searchable and retrievable by the application. You can store it many ways. You can use a separate index that's accessed in real time. Or, if your source data platform supports built-in indexes, you can run queries directly.

- **Behavioral instructions** is information that's used to influence the model so that it can adapt to specific tasks, domains, or response styles for future inferencing requests. For example, if the model is expected to provide answers in a specific grammatical style, that style guide would serve as fine tuning data.

- **User data** includes information provided by users during interactions with the application. When you interact with generative models, stateful interactions occur. These models lack inherent memory and treat each interaction as atomic.

  When you manage stateful interactions, also known as *TURN data* in chat applications, it's important to store data for the shortest time necessary. Ideally, this data should be destroyed after the session ends. However, there might be operational or compliance reasons that require you to retain certain data, such as the original question or the model's response, beyond the session's duration. When possible, avoid storing this data past the session.

# Source data querying

Grounding data can be sourced from the source data platform if it supports advanced querying capabilties. For example, built-in vector indexes, that allows you to run semantic queries directly. Another option is graph databases, which makes it easier to model and query relationships between entities. For example, you can use a graph database to find products preferred by users aged 20–30 by linking user profiles, orders, and product data. This

relationship-based querying is more intuitive and efficient in graph databases compared to traditional databases.

Querying the source database directly simplifies security measures, when compared to indexes where techniques like security trimming can be complex and error-prone. However, using the live system for data queries has performance tradeoffs as it can impact current users due to added load. To mitigate this, one option is to create a replica of the database for querying purposes.

# Indexing

The core of data design includes efficiently storing and managing foundational data. This approach ensures that data can be augmented to achieve the highest level of relevance.

A simple AI strategy might involve querying the source data for each user interaction. However, this approach isn't practical because of the high costs and complexities of direct data source interactions. Instead, you should repurpose source data as a copy in an index that's optimized for search and retrieval. The goal of this approach is to improve the model's comprehension and its ability to generate relevant responses.

Consider a banking workload that stores details related to user bank accounts and preferences and financial transactions in a data store. In a generative AI scenario that uses a RAG pattern, grounding data is created and indexed with context so that the model can give relevant responses. For instance, by providing relevant data about user transactions for context during inferencing, the model can answer questions related to the user's spending patterns in the last quarter.

## Specialized index technology

Consider externalizing the grounding data to a search index. Use this approach instead of querying directly from the source system.

There are benefits of using the search index. You can model and transform the copy of the data according to the expected queries. Direct queries to the primary source are problematic because there's a possibility that source data isn't accessible. An index ensures that the data remains available as long you deem it relevant to the application. You also avoid stressing the source data system. This strategy ensures that AI-related queries don't affect its primary use case.

Some technology options have self-indexing capabilities. Indexes can reach out to data sources and incorporate their data. For this option, network considerations are key. If the index needs to connect to databases, there are potential issues, such as network latency and reliability.

There's an initial cost to import data. After the data is in your index, you won't need to move it again unless there are changes or updates. Data management over time is a crucial aspect of index design. For more information, see Index maintenance.

## Default or custom index

Certain technologies support automatically building a default index for your data. This index is generated on data ingestion with minimal input. The index has out-of-the-box capabilities. A default index might be acceptable for proof of concepts and some production scenarios.

Some scenarios might require you to have a custom index schema to improve relevancy based on specific workload requirements. Those requirements dictate how you design the schema, enable index capabilities, and include relevant metadata.

## Schema design

You can think of indexes as structures that organize and optimize data for retrieval. More specifically, they organize data in documents and fields of a table. Consider the following points:

- **Index topology**. Evaluate whether to colocate all data in a single index or distribute across multiple indexes. This decision significantly affects query performance, index maintenance, query simplicity, and dissimilar field configuration (or schema) between documents.

  For example, consider user queries that request content in a specific language. The simplest data design choice is possibly translating all languages into one language and storing it in a single index. Or, data can be stored in all languages in a single index. This choice results in multiple documents for each language. The index's filtering capability could be used to restrict the results to the desired language. Alternatively, each index could contain the translated versions for a given language as expected in the query.

  In some situations, you might need multiple search indexes. This approach allows you to independently optimize each index for maximum relevancy from your search queries. For example, an HR employee handbook and a product maintenance manual serve different purposes and audiences. By indexing them separately, you can tailor the schema and search queries for each, which improves user experience. This approach can be complex to implement and requires an orchestrator to facilitate calls to each index. The orchestration component is described in Application design for AI workloads on Azure.

  > ⓘ **Note**

The choice between the two topologies and the data segmentation strategy depends on workload requirements, use cases, and user expectations.

Doing cross-index queries can be challenging and can affect search relevancy. In worst case scenarios, there might be manual sifting through results, deciding which ones fit the criteria. This process introduces latency and adds complexity. In contrast, a single index approach is simpler and more straightforward. Relevancy can be improved by using index capabilities such as filtering.

In some cases, compliance considerations lead to the need for separate indexes. For instance, if business requirements demand that data is isolated between Europe and America, multiple indexes might be inevitable.

- **Document design**. Align your data design with expected user queries to optimize relevancy. Consider how each document should serve queries. For search indexes, prioritize relevant documents and refine the results to a concise set that's densely packed with relevant information.

- **Field design**. Configure your index fields to support search performance and relevance. Your index fields should map to the document attributes you want to make searchable, retrievable, filterable, and sortable. They include embeddings, IDs, or any other data that can boost searching.

## Index capabilities

Configure the search index fields to return the most relevant set of documents. The decision depends on the capabilities that the search index technology and workload requirements support.

- **Filter, search, and sort options**. Consider these options because they're directly related to use cases for augmentation. For example, *filterable* determines true or false against a value provided in the query and returns relevant documents. For *searchability*, the attribute indicates whether the search query can reference the field. For instance, you might check if a text field contains specific text or if it's mathematically related to another vector. You can optionally assign a relative weight to that field as part of the search query. You can also make result sets *sortable*, which lists the results by relevance.

  One potential use case for filtering is performing **security trimming**. Security trimming ensures that users only see the data they're authorized to access, even when using a shared index. For example, a single index might include a region field, and queries are filtered based on the user's region or role. The application accesses a shared index of documents but only returns results the user has permission to view.

> **Tradeoff.** Enabling capabilities to index fields increases space requirements, affecting costs. Only add capabilities that you intend to use.

- **Metadata**. Indexes typically have metadata associated with index fields. Metadata helps us understand and manage data by providing relevant details about it. When designing indexes, consider whether metadata are retrievable or only used for relevance determination. The decision affects compute costs because the underlying indexing process is different. Excessive metadata can unnecessarily increase the size of the index.

There are many technology choices for indexing. Many share similar characteristics, such as the ones listed previously. Some indexes might have extra features, like processing text and language analysis during indexing. To make text more suitable for indexing and searching, break text into tokens, convert it to lowercase, or remove stop words.

## Efficient querying

Grounding data is used in generative AI applications to increase the accuracy and relevance of the responses to user queries. Consider user queries upfront. Understand what questions could be asked, who asks them, and how frequently they're asked. This information helps the application form context and understand what result might be relevant.

Typical types of searches are:

- **Vector queries** search for similar items based on their vector representations or data points in a high-dimensional space.
- **Keyword search** searches within the entire content of textual documents. It indexes and queries large volumes of text data and is commonly used in search engines, databases, and document management systems.
- **Semantic ranking** improves the relevance of search results by reordering them based on their semantic relevance to the query, promoting the most semantically relevant matches to the top of the list.
- **Hybrid search** combines different search types, like vector search, full text search, and semantic ranking, to further improve the relevancy of the search results.

To further improve model performance, combine search types.

The manner in which data is stored and processed effects query efficiency. Each time data is added to an index, compute cycles are needed for indexing. If indexing and responding to queries are done on the same compute resources, there could be contention. Ideally, an index

should focus on the primary goal of answering queries efficiently and finding relevant documents rather than excessive indexing.

Cost and performance are key drivers of index design. Techniques like creating shadow copies can speed up querying. However, data duplication occurs through indexes, which incurs costs.

> **Tradeoff.** Index design should consider both cost and performance. Strike a balance by optimizing storage and prioritizing efficient query answering and relevant document retrieval over excessive indexing.

For technology choices for the data store, search indexes, such as Elasticsearch or AI Search, provide powerful search capabilities, including vectorized and relevancy-based searches. Alternatively, consider database options that support the type of data that you have and the types of queries you need because they're optimized for querying. Ultimately, it's about the capabilities offered by the options and the investment of building new skill sets on the team.

# Data preparation

Grounding data is based on existing data, which needs to be made suitable for semantic querying. Some queries to find relevant documents in the index can be literal matching. Other queries require fuzzy matching.

Before contextual data is ready to support inferencing requests to the model, there's an upfront preprocessing step that aims at cleaning, transforming, and structuring data. The goal is to reduce noise and bias, search efficiently, and maximize relevancy of the index searches. The choice tools or logic for preprocessing depends on the workload team, but there are some broad considerations.

## Data volume rescoping

Data volume rescoping involves adjusting the scope of data by expanding or narrowing it to create a tight index so that relevance is increased. Query efficiency is another significant concern. Storing unnecessary data negatively affects both of those goals. For example, consider location data for a user. If only the city portion is relevant, optimize by storing just the city text instead of the full text that represents the address.

Here are some broad considerations.

- **Data elimination**. Retain only what's essential for the product's functionality, discarding unnecessary details. Here are some common examples.

- **Qualitative elimination**. One way to transition from a broad scope to a narrower more relative one is to eliminate low-quality data by selectively only choosing to index relevant source data. The challenge lies in programmatically identifying content that's not relevant for AI scenarios. While the content might be useful for other intents, such as auditing or completeness, including it in the AI workload risks reducing relevance. One way to flag such content is by using metadata that can be used at index population time if the content must be added to the index.

- **Sensitive data**. Copying data from source data to an index might also bring over sensitive information. Respect the data classification labels applied at source and maintain the same level of sensitivity to this data set. When you deal with data that has personal information, don't store personal data unless you need it to respond to the query. For example, apply data classification when indexing emails. If an email is flagged as sensitive, avoid storing it in a general sensitivity data store.

- **Normalizing and standardizing text**. Addressing typos and standardizing text is crucial for keyword-based indexes. A potential use case is translations, especially when dealing with multilingual content.

  This type of preprocessing is also needed for embeddings, which enables you to compare words based on their context and significance. However, one challenge occurs from the case sensitivity of words. Context matters, and there can be nuances, such as the semantic differences between the adjective "civic" and the proper noun "(Honda) Civic."

- **Data addition**. Augmenting context often relies on a process called *enriching*. This adds extra information, which makes it more meaningful. For instance, consider a text snippet. A human in the loop or AI creates relevant questions that can be answered using the context of the snippet. When you store these questions alongside the grounding data, user queries can be compared to the generated queries to evaluate document relevancy. Colocating this new data with grounding data can make chunked data more useful. Another example of enrichment is tagging content with metadata. This can uncover natural breaks in meaning, or semantic boundaries, which can be used to define chunks. These chunks may be different sizes, but each one is meaningful and can stand alone.

  Consider maintaining data lineage. It's important for AI workloads to track the source of data because that information can get lost when a system aggregates various components into one index. This information might not ever be exposed to users, but information about data origins is crucial for internal data governance teams. This metadata isn't necessarily for the model. It helps maintain transparency and accountability.

> **Tradeoff.** On one hand, adding new data increases the chances of finding relevancy within the data set. However, this benefit comes at a cost. Specifically, the computational resources required to process and manage that field. The time spent collecting and storing data can be substantial. Be aware that overloading with unnecessary fields can strain resources.

- **Processing text data**. Consider techniques such as synonyms, stemming, and semantic proximity to enhance relevancy. Delegate these techniques to tooling if possible. Some technologies, such as Elasticsearch or AI search, offer such features for preprocessing data during index creation.

## Data type morphing

Index fields in a datastore are data-typed to serve a specific purpose. Numeric fields facilitate efficient querying, text fields allow for text-based searches, and Boolean fields handle binary information.

Source data typically exists in various types of data, like text, images, and tables, and processing that data might be complex. You might have to extract key-value pairs, identify section headings for semantic chunking, recognize specific identifiers, and so on.

For example, if your source data contains images, they aren't inherently searchable. They need to be converted into vector representations to enable efficient semantic searches and comparisons. If relevancy is tied to the data behind these formats, invest in extraction of the data. Transform source data types to functional data types that help in querying and analysis.

## Chunking and the context window size

A context window refers to the maximum amount of text that the model can handle at one time when generating a response. The context window size limits how much input (user query, grounding data, system instructions) the model can process in a single interaction.

Grounding data often contains a large volume of information. Models only support up to a certain amount of tokens in one request, beyond which text can be truncated or ignored. If the context window is large, the entire text can be fed directly to the model. This strategy called *full-document grounding* can improve accuracy, because the model has access to the full content. However, there's a trade-off. Sending large documents consumes a lot of tokens, which can increase costs.

Alternatively, if the context window is smaller, strategies like *chunking* can be efficient. It involves dividing text into smaller pieces that can be individually processed and indexed. This

allows for efficient search and retrieval despite token limitations. Check the maximum number of tokens that your choice of model can handle. Your chunks shouldn't exceed that limit.

> **Tradeoff.** Full-document grounding offers high accuracy and simpler implementation by allowing the model to process the entire context at once. However, it comes with high token costs, increased latency, and limited scalability due to context window constraints. In contrast, chunking is more complex to implement and maintain but is more flexible. It enables targeted responses and faster processing though it may be less accurate if the chunks aren't well designed.

There are many techniques for implementing chunking. For more information, see Chunking approaches.

*Embeddings* is also another design strategy that enables vector search capabilities. Embeddings is a mathematical representation of an object that's generated by AI models based on grounding data. They're stored in the index and add more context that helps complex queries yield results with better relevancy. For more information, see Generate embeddings.

# Index maintenance

Maintenance over time is a crucial aspect of index design. For static data, where documents remain unchanged, index maintenance is straightforward. But, most indexes are dynamic. Over time there might be new data added and the index schema might need new fields. Conversely, some data and fields might need to be deleted if they're no longer relevant. Commonly used technology options for indexers have features to handle updates automatically. For information about the recommended index characteristics, see Considerations for a search index.

## Maintenance criteria

- **Functionality updates**. The index might need to be updated if there's a change in application functionality. This situation happens when new questions are asked. To accommodate these changes, you might need to add new fields to the index or modify filtering, searching, or text-processing options on existing fields.
- **Data deletion**. Data deletion is challenging because you need to analyze available and missing data to determine what's irrelevant. To exclude outdated content from an index, consider the use of metadata that prevents search engines from indexing specific pages or content. Also, when you choose storage options, select a technology that supports deletions efficiently. For example, blob storage supports soft deletes. If you use AI search and loading documents from storage, blob storage can detect removed documents and

delete corresponding entries. This approach isn't ideal but it's necessary when reindexing is costly because of a large index size.

The concept of the *right to be forgotten* refers to an individual's right to have their personal data removed from online platforms or databases. Ensure that you have policies in place to remove personal data if it was used for training. You can address this requirement by reindexing your data set. If data is deleted from the transactional database, subsequent index updates reflect those changes.

- **Maintaining compatibility**. Applications often require specific data structures, and any deviation can disrupt their functionality. For example, if a field is removed and the application requests that field, then a failure condition might occur. As you would for a traditional database, adopt a forward compatibility mindset for indexes and maintain a level of rigor. When you make changes to the index, like adding or removing fields, coordinate schema changes with code updates.

  >  **Tradeoff.** Add, update, and delete actions against an index are expensive. Consider the frequency of updates and the cost to performance based on data store size and efficiency. Keeping obsolete documents in the index incurs storage, maintenance, and querying costs.

## Deployment strategy

**Deployment strategy**. There are two main strategies for updating the index.

- **Side-by-side deployments**. In this approach, a new index that has updates lives alongside the existing one. After the new index is tested and fully operational, queries are switched to use the updated index. The application remains unaware of this switch because it only interacts with the new index. If you discover other issues after the new index is deployed to production use, you can revert to the old index. This approach minimizes downtime and ensures continuous availability.

  Side-by-side updates work well when the cost of rebuilding the index is reasonable and can be completed in a reasonable time frame. In general, strive to keep indexes as efficient as possible because larger indexes consume more resources. Regularly monitor and maintain indexes to avoid unnecessary growth.

> **Tip**
>
> When you perform resource-intensive data pre-processing tasks like entity recognition, lookups, and calculations, consider saving a copy of the results. This approach ensures that when you need to rebuild the index, you can avoid redoing all the computations.

Some calculations might no longer apply because of deletions or updates, but many will remain relevant.

- **In-place update deployments**. This approach directly modifies the existing index. Saving the cost of duplication can be beneficial, but it also introduces risks because of potential downtime and resource-intensive operations. If your index is large and rebuilding it from scratch exceeds your desired update frequency, you can consider using in-place updates. However, this approach is challenging and carries the risk of breaching your service-level objective (SLO).

  > **Tradeoff.** Evaluate the cost of doing side-by-side deployments of indexes against doing in-place updates that deploy additions, updates, and deletions. In most cases, you should use side-by-side updates instead of in-place updates. When an index is rebuilt, the process effectively handles deletions and updates because it creates a completely new data set. This strategy provides the opportunity to test data. Even though side-by-side deployments temporarily duplicate data and incur additional costs, the benefits in testing and performance evaluation often justify this storage requirement. Before you make an index live, examine the data to ensure that it aligns with your expectations.

- **Scheduled updates**. Instead of maintaining continuous real-time communication with data sources, you can refresh grounding data periodically. This approach ensures that the data stays relevant through scheduled updates, which eliminates the need for constant interaction.

- **Emergency updates**. Unexpected situations can occur, such as unwanted data inadvertently leaking into the search index. If this issue occurs, you might need to take immediate action, such as removing specific documents or adjusting data within the index. Regardless of the deployment strategy you choose, such as side-by-side updates or in-place updates, always plan for the possibility of emergency operations.

- **Self-updating index**. If your indexing technology supports automatically updating the index to keep it synchronized with an external data source, it might be able to automatically process changes in the data. Data changes include additions or deletions, without manual intervention. Keep in mind that every change triggers an operation in the index, which consumes resources. The index might remain responsive to queries, but its capacity for handling them could be reduced during the update process.

## Freshness operations

Measure the window of time between source data creation or modification and its addition to the index as an indicator, and track it against SLOs. This indicator drives data decisions around updating your data pipeline design to ensure that data is available in your index when you need it. An index should only be as fresh as is required.

To maintain freshness, you can either rebuild the index entirely, or incrementally update it to stay synchronized with the original data sources. Both methods ensure that the index remains current and accurate.

Upfront investment in fine tuning the model might be less expensive than implementing a RAG pattern, prompt engineering, and data augmentation methods.

## Next steps

Design area: Data platform

# Data platform for AI workloads on Azure

Choosing a data platform involves understanding the unique data challenges that these solutions bring. GenAI solutions, especially those built with foundation models, depend on diverse, high-quality data, fast access to scalable data stores that supports vector search. The goal is to meet these needs without adding unnecessary complexity to your architecture. Understanding the principles of effective data pipeline design is essential before evaluating platform options.

When evaluating platform choices, start by asking whether you truly need additional components. Simpler architectures are often faster to deploy, easier to manage, and more cost-effective. Ask yourself:

- Can the model achieve its expected performance using data from a single source?
- Does the source data store already provide the analytics or search capabilities you need?
- Is the source data already structured and indexed for AI or vector search?

If the answer is yes to most of these questions, a complex architecture might not be necessary. For example, databases such as Azure Cosmos DB and Azure SQL Database already support vector data types and vector search natively but they need to be enabled and configured. These capabilities can reduce the need for separate indexing or specialized vector databases, minimizing data movement while improving performance.

As your workload grows and data comes from multiple sources, the platform decision becomes more complex. You may need to consider solutions that support ETL or ELT pipelines, specialized search indexes, and scalable storage for large datasets. Each added capability should serve a clear purpose rather than simply expanding the technology stack.

This article provides guidance on choosing a data platform for workloads where data needs to be stored, processed, or analyzed. The focus is on solutions that support generative AI (GenAI). We highly recommend that you understand the principles of good data pipeline design before you explore the technological capabilities that this article describes. For more information, see Grounding data design.

For recommendations specific to discriminative model training and fine-tuning, see Training data platform considerations.

## Considerations for data storage platform

In AI workloads, data usually moves through several stages of storage and processing, guided by pipelines that connect each step. One important stage is the data store that holds

information collected and combined from multiple sources. This store allows you to process and refine the data until it's ready for the next stage.

> **Note**
>
> You might not need this component in your architecture. In some cases, you could access data directly from the source systems. However, doing so can lead to performance issues and may overload those systems with AI queries. It can also cause access or reliability challenges. To prevent these issues, it's usually better to copy the data into a dedicated store for aggregation and processing.

When you choose a platform for this store, make sure it follows the same security standards as your source systems, is cost-effective, and works well with ETL, ELT, or EL processing tasks. Your options can range from simple storage solutions to large-scale data platforms, depending on your data volume and performance needs. Look for a storage option that's reliable, scalable, and provides good value for your workload.

Here are a few questions to help guide your choice of data store technology.

### Can the platform handle different data formats?

Your data store should be able to store a variety of data formats and, when needed, convert data between them.

For example, if your ingestion pipeline brings in data from both a relational database and a JSON file, it must support structured and semi-structured data. You might want to convert your data into Delta format to enable the richer functionality Delta Lake technology provides. The platform should provide built-in tools for this kind of transformation so that you do not need to write custom code.

### Do you expect to store multiple versions of the data?

Data changes over time in both values and structure, and source systems usually store only the current state. If you need historical context, choose a data platform that supports versioning. Without it, you might have to duplicate datasets, which adds complexity.

Versioning has other benefits. In some cases, you may need separate copies of data for different use cases. Each copy can evolve independently, and the platform should manage versioning across all copies to preserve context for your AI models.

### Does the platform have built-in data lifecycle management capabilities?

Data lifecycle management (DLM) helps control growth from creation to deletion. Your platform should automatically remove intermediate copies, manage archived data, and support regulatory retention when needed. Without that, data can grow uncontrollably and that unnecessary volume can make processing difficult. For instance, you might need to rerun preprocessing steps several times to improve data quality. The platform should automatically remove intermediate copies when they are no longer needed.

In other cases, you may need to retain data for compliance or audits. Look for storage options that support cold or archived tiers for rarely accessed data at lower cost.

## Does the platform support data governance features?

Auditability is an important aspect for AI workloads. Your platform should maintain audit trails to track data access, ensure privacy, and document data origins. It should also support a data dictionary or catalog that manages metadata, data types, purpose, and lineage, especially when data comes from multiple sources.

## How much data do you expect to store?

AI workloads generate large volumes of data, which can grow further with multiple versions and extra metadata. Your data platform should scale efficiently for both storage and throughput, handling high ingestion rates, concurrent writes, and intensive processing without performance degradation.

When choosing a platform, consider the entire workflow, as ingestion and processing often happen at the same time. The system should support parallel processing and frequent data movement, and provide telemetry to give clear insight into read and write performance.

## Is this data store critical to your workload's reliability?

Choose a platform that supports reliability and scalability through replication or multiple instances. Many big data stores use controllers that distribute processing automatically and provide failover when an instance becomes unavailable.

Data also needs to be both durable and accessible. Make sure the platform guarantees data integrity, provides accessible APIs, and supports backup or restore capabilities if rebuilding data from scratch would be costly.

## Do you have any cost constraints?

After reliability and performance requirements are met, consider how to optimize costs. For many AI workloads, a write-once, read-many pattern is sufficient and helps control expenses. Grounding data should be cost-effective to store and retrieve, even if it doesn't require the same level of responsiveness as a production database. The goal is to balance cost, efficiency, and performance.

### Do you need to support data sovereignty or regional compliance requirements?

For workloads that handle regulated or sensitive data, consider deploying in a sovereign cloud, such as Azure Government, Microsoft Azure operated by 21Vianet, or other National Partner Clouds. These environments are designed to meet strict data residency, privacy, and compliance requirements by ensuring that data storage, processing, and access remain within specific jurisdictions.

Sovereign clouds provide greater control and independence over your data, which is often a requirement for sectors such as government, defense, or banking. However, keep in mind that some advanced AI and data platform features might not yet be available in these regions. Review service availability before designing your architecture.

Use Microsoft Purview to maintain data cataloging, classification, and lineage tracking across these environments. For highly confidential workloads, consider using confidential computing and customer-managed keys to strengthen data protection. You must verify that your deployment aligns with regional regulations.

## Technology options

Expand table

| Function | Recommended Technologies | Alternatives / Complementary Tools |
|---|---|---|
| **Multi-format data storage** | Azure Data Lake Storage Gen2, Microsoft Fabric Lakehouse, Azure Databricks Lakehouse | Azure Blob Storage, Azure Synapse Analytics, on-premises data warehouse |
| **Data versioning and lineage** | Microsoft Fabric Lakehouse, Azure Data Lake Storage Gen2 (with Delta Lake), Azure Databricks (Delta Lake) | Git LFS, DVC (Data Version Control), Apache Iceberg |
| **Data lifecycle management (DLM)** | Azure Data Lake Storage Gen2 (lifecycle policies), Azure Blob Storage (tiering), Azure Databricks (table optimization) | Amazon S3 (lifecycle policies), Google Cloud Storage |

| Function | Recommended Technologies | Alternatives / Complementary Tools |
|---|---|---|
| **Data governance and cataloging** | Microsoft Purview, Azure Databricks Unity Catalog | Apache Atlas, DataHub, Collibra |
| **High-volume data storage** | Azure Data Lake Storage Gen2, Azure Synapse Analytics, Azure Databricks Lakehouse | Azure Blob Storage, Hadoop HDFS, Amazon S3 |

# Considerations for data processing platform

The data processing platform plays a key role in preparing and transforming data so it's ready for downstream use, whether that's RAG indexing, analytics, or other use case.

> **Note**
>
> For GenAI and retrieval-augmented generation (RAG), it's useful to understand the difference between ETL, ELT, and EL processes.
>
> - ETL: Extract, transform, then load, typical for traditional data warehousing.
> - ELT: Extract, load, then transform, common for data lakes and big data tools like PySpark.
> - EL: Extract and load, used in RAG scenarios where you store documents first, then perform transformations like text chunking or image extraction later.

There are two places where processing can happen:

- Ingestion Layer. The ingestion pipeline collects data from various sources and moves it into your aggregate data store. Along the way, it often performs basic preprocessing or formatting so the data is queryable. To reduce the need for custom code, it's best to use a data platform that handles as much of this as possible. When evaluating tools, consider the ETL or ELT features required to support your AI workloads, such as data augmentation.

- Processing Layer. After data lands in the aggregate store, it typically needs deeper processing before it's ready for indexing or use in AI models. These pipelines should offer similar levels of reliability and scalability as your ingestion layer, but the focus shifts to transforming and reshaping the data.

Typical tasks include:

- Entity recognition and enrichment

- Integrating additional data sources
- Performing lookups and transformations
- Cleaning or deleting irrelevant data

A strong data platform helps automate and orchestrate these operations efficiently.

## What's the support for connecting to data sources?

The platform should easily connect to data sources you expect ingest from, whether that's relational databases, big data sources, or blob storage.

Look for prebuilt connectors and low-code integrations. Ideally, you want drag-and-drop or configuration-based connectors that support lookups, data copying, and governance.

## Can the platform process various data formats?

Data comes in many shapes: structured (SQL, relational tables), semi-structured (JSON, XML, Parquet), and unstructured (documents, images), and streaming (IoT data). Choose a platform that can handle the formats your use case requires considering immediate and longer term requirements.

## Does the platform offer features for data preparation and rescoping?

Before your data is ready for indexing or model consumption, it needs to be cleaned, enriched, and reshaped. Your data design strategies should explicitly outline the requirements. A good platform should:

- Remove duplicates and fill missing values
- Handle stemming, normalization and other basic cleanup tasks when planning to support keyword or hybrid (keyword+vector) search
- Support advanced transformations like chunking, enrichment, and document analysis

If your data store supports these operations natively, you can process data in place without moving it. Otherwise, use external tools like Azure Databricks or Azure Data Factory for heavy transformations.

In some cases, you may choose to externalize part of this responsibility to the platform that supports the next stage. A common example of this approach is RAG implementation. During processing, documents are divided into smaller chunks, with each chunk stored as a separate row in the index. These chunks are then paired with embeddings, often generated through an OpenAI service. In Azure AI Search, this process is orchestrated as part of the enrichment pipeline during indexing, where documents are processed by an embedding model (such as an

OpenAI embedding model) to generate vector representations that are then stored in the index.

## Is there a built-in orchestrator for managing workflows?

Data processing typically happens as modular jobs that need complex coordination. Your platform should include an orchestrator to define, schedule, and monitor these workflows. Look for:

- Support for job dependencies and checks that validate sequence of execution
- Flexible modification of workflows that allows for easy adjustments without rewriting large portions of code.
- Monitoring and logging capabilities

Popular tools include Azure Data Factory for its rich feature set for workflow management, or Azure Databricks for more complex orchestration. If cost is a concern, Apache NiFi or Airflow can be more economical alternatives.

## How much data do you expect to ingest?

Estimate how much data you'll ingest and the frequency of ingestion. For example, if you expect to load 10 terabytes of data daily into an index, the platform should support strong parallelization and distributed execution. For smaller workloads, simpler tools like Logic Apps might work, but for higher volumes, Data Factory or Databricks are more suitable. For scalability and throughput, consider:

- Data volume and frequency
- Tolerable latency requirements
- Job complexity

For instance, data cleansing involves validating and potentially replacing invalid fields or masking sensitive information. These tasks, though basic, require significant resources because each row is processed individually, which adds to the overall time.

## What monitoring capabilities do you need?

Data processing pipelines should have monitoring capabilities and provide insights into the pipeline's performance and status of jobs. Your platform should provide:

- Job progress tracking
- Logs, metrics, and alerts to understand pipeline behavior
- Integration with your broader monitoring stack

Identify any gaps in the built-in telemetry, and determine what additional monitoring you need to implement. This monitoring might involve adding custom logging or metrics to capture specific details about the job steps.

### How much reliability do you expect of the data processing platform?

Choose a platform that minimizes single points of failure and supports retries for failed tasks. For example, hosting custom processing logic invoked from Data Factory on Azure Kubernetes Service (AKS) typically offers stronger reliability than hosting it in Azure Logic Apps.

If your data updates infrequently and you handle processing through weekly batch processing, occasional failures may be acceptable. But for real-time AI scenarios, you'll need higher reliability.

### Are there any cost constraints?

The goal is to avoid over-engineering and pick a platform that fits your needs today while leaving room to scale. For example, if you don't need Databricks' advanced features, Data Factory might offer a more affordable option. Open-source tools like Airflow or NiFi can further reduce costs.

### What are the security requirements on the workflows and on the data that you process?

Security, privacy, and data residency requirements should guide your choice. Ideally, the platform should provide built-in support for such isolation that enables efficient and secure data management. At the very least, make sure the platform:

- Meets regional data residency laws. You might need to run separate pipelines for different regions, such as one for Europe and another for America, to meet local compliance regulations.
- Supports identity and access management (IAM) to ensure that only authorized identities have access to specific jobs or steps within workflows.
- Allows fine-grained access control at the workflow or step level.

## Technology options

Expand table

| Function | Recommended Technologies | Alternatives / Complementary Tools |
|---|---|---|
| **Data cleansing** | Azure Data Factory, Azure Databricks, Microsoft Fabric Dataflows | Apache NiFi, Apache Airflow |
| **Data transformation** | Azure Databricks, Azure Synapse Analytics, Microsoft Fabric Data Engineering | Azure Data Factory Pipelines |
| **Data enrichment** | Azure AI Document Intelligence, Azure OpenAI Service, Azure AI Search | Custom Python APIs or third-party AI services |
| **Workflow orchestration** | Azure Data Factory Pipelines, Databricks Jobs | Apache Airflow, Apache NiFi |
| **RAG Workflows** | Azure OpenAI Service, Azure AI Search, Azure Databricks | Microsoft Fabric Data Science |

# Considerations for a search index

A search index stores the contextual or grounding data that's sent to a model's inference endpoint along with the prompt. Index queries are a critical component in preparing the data sent to the model in the inference requests and must deliver low-latency performance.

Unlike batch-oriented ETL pipelines, this index must support real-time inferencing, meaning high performance and reliability are non-negotiable. It's purpose-built for AI workloads and supports capabilities like keyword indexing, filtering, and vector-based search, which go beyond what traditional data stores provide.

The ideal design is a high-performance data store, optimized for reads, that can handle imprecise or fuzzy queries while still returning relevant results. Choose the index technology keeping those points in mind.

## What types of search does the search index support?

Every request to the system may result into one or more queries to the index. For retrieval-augmented generation (RAG) and other AI-driven workloads, vector search is a must-have. Vector search allows the system to find semantically similar data points by using embeddings rather than exact keyword matches.

However, combining vector search with full-text search, filtering, and special data types (such as geo-location) makes the index far more powerful.

Your data design should clearly specify which search types are required and how they should work together. For more information, see Efficient querying in the data design.

## How does the index handle multimodal data?

AI workloads often deal with data that includes not just text, but also images, audio, or video. The index itself can't directly understand images. So before adding images to the index, they need to either be converted into a text-based representation (using OCR or image captioning), from which embeddings are generated, or vector embeddings can be generated directly from the image using vision models. The index can then perform vector search, allowing semantic queries.

In this use case, the search index should have:

- Vector search support to store and query embeddings (numeric vectors) derived from the image.
- Integration with external APIs and AI services for extracting or enriching data during the indexing process.
- Ability to store extracted fields (text, tags, captions, embeddings) in appropriate schema fields as metadata for search and filtering.

## Does the index support automatic update capabilities when the data in the data sources changes?

Automation is key for maintaining data freshness. Select an index that supports automatic updates or incremental refreshes when the underlying data changes.

If the platform doesn't offer this natively, you'll need to implement a custom process to detect and push updates. Offloading this responsibility to the platform can reduce operational overhead and simplify maintenance, especially as data volumes grow.

## Can the index perform with large volumes of data?

The index must scale efficiently as data volume grows. For workloads that implement RAG, each document is often split into multiple chunks, which significantly increases the amount of data stored.

Your chosen platform should be able to:

- Scale horizontally as data grows
- Maintain query performance under heavy load
- Store both raw data and related metadata, enrichments, and entities

## Does the index have built-in reliability features?

Reliability for the search index should mirror that of your inference endpoint as both are part of the same real-time processing path.

Each step must meet similar uptime and performance expectations. To achieve this, when you choose the data platform, look for:

- High availability and zone redundancy capabilities to survive zonal and regional outages.
- Automatic recovery and easy index rebuilding to prevent the use of a corrupted index for inferencing.
- Index aliasing or swap capabilities to enable updates with zero downtime.

Also, understand the system's failure modes or stress indicators, such as throttling. For example, during background reindexing, throughput may drop. The system might normally handle 50 concurrent users but only 30 during that job. Plan job timing and capacity accordingly, accounting for both front-end queries and back-end maintenance tasks.

## What are the main cost drivers of this technology?

Search index costs are typically usage-based, so it's important to model your expected data volume, query rate, and throughput.

Most index platforms, such as Azure AI Search, are Platform as a Service (PaaS) offerings, where pricing is abstracted and presented in units of capacity, storage, and feature usage.

Be mindful of:

- Tier pricing and scaling limits
- Extra costs from advanced features (for example, image extraction or skillset enrichment)
- Unused capacity in overprovisioned tiers
- Index complexity (number of indexes and concurrent query limits)

To understand the costs associated with AI Search, see Plan and manage costs of an AI Search service.

## Do the security features of the index fulfill your security data design?

Your data design should clearly specify the security and privacy requirements, and your index must fully support them. When working in dev or test environments that use real data, ensure the index complies with access control and traceability policies. Look for features such as:

- Data masking and PII removal
- Client identity management through Microsoft Entra ID
- Document-level access controls for filtering results based on user identity

If the platform doesn't natively support these, consider implementing query-level filters as a fallback. For more information, see Security filters for trimming results in AI Search.

From a network security perspective, the index should:

- Support egress control and network segmentation
- Integrate with private networks when compute runs in a virtual network
- Use managed identities for authentication through Microsoft Entra ID
- Avoid exposing components directly to the public internet

Embeddings can still expose sensitive information if not properly secured. Risks include embedding inversion (reconstructing original text from vectors), data poisoning (inserting malicious vectors), and unauthorized access to embedding stores or backups. To mitigate these risks, apply security measures like:

- Encryption at rest and in transit
- Strict access controls
- Private network connectivity discussed above
- Monitor embedding endpoints for anomalies or tampering

Similar to other types of data, have processes to remove sensitive or personal data. Treat vector indexes as sensitive data stores that require the same level of security and governance as other production systems.

## Technology options

Expand table

| Function | Recommended Technologies | Alternatives / Complementary Tools |
|---|---|---|
| **Vector search and semantic search** | Azure AI Search, Azure Cosmos DB (vector search), Azure Database for PostgreSQL (pgvector) | Pinecone, Weaviate, Chroma, Qdrant |
| **Full-text search and keyword indexing** | Azure AI Search | Elasticsearch, Apache Solr, Azure SQL Database Full-Text Search |
| **Multimodal data processing** | Azure AI Search (with skillsets), Azure AI Document Intelligence, Azure AI Vision | Custom processing with OpenAI APIs, Amazon Textract |
| **Automatic data refresh and indexing** | Azure AI Search (with indexers), Azure Data Factory triggers | Custom polling solutions, Apache NiFi, change data capture |
| **High-availability and reliability** | Azure AI Search (zone redundancy), Azure Cosmos DB (global distribution) | Multi-region deployments, load balancers, Azure Traffic Manager |

| Function | Recommended Technologies | Alternatives / Complementary Tools |
| --- | --- | --- |
| **Index aliasing and zero-downtime updates** | Azure AI Search (index aliases), Azure Cosmos DB | Blue-green deployment patterns, custom routing logic |
| **Document-level security and access control** | Azure AI Search (security filters), Microsoft Entra ID integration | Custom authorization layers, row-level security in databases |
| **Network security and private access** | Azure Private Link, Virtual Network integration, Managed Identities | VPN gateways, Azure Firewall, custom network security groups |

# Training and fine-tuning considerations

When designing your data platform for traditional machine learning (ML) or non-GenAI workloads, your focus shifts from real-time inference to data quality, reproducibility, and environment separation. These workloads rely on well-structured aggregated data and often involve additional layers, such as feature stores and batch inference data stores, to optimize model performance and cost efficiency.

We highly recommend that you understand the principles of good data pipeline design before you explore the technological capabilities that this article describes. For more information, see Training data design.

## Do you plan to conduct training with production data?

How you deploy your models determines how tightly production data is coupled with your development environment. There are two main deployment approaches:

- **Model deployment**. The model is trained or tuned using production data during development. This approach can improve model relevance but demands strong security controls, since sensitive data is being used outside production.
- **Code deployment**. The model is trained using non-production data in development and only interacts with real data once it's deployed to production. This method simplifies development security but can increase compute and storage costs, since training may need to be repeated in multiple environments.

Regardless of the approach, your data platform should clearly separate development and production environments, ensuring proper isolation and access control.

## Are you prioritizing convenience over functionality?

When choosing a data platform for ML, don't make the decision based solely on notebook support.

Notebooks are great for exploratory data analysis, but they're not a deciding factor for selecting a production-grade data platform. Notebook compute resources usually sit outside the aggregation data store and are integrated with external tools such as Azure Machine Learning or Databricks Workspaces.

Prioritize core capabilities, such as data versioning, governance, scalability, and security,— over convenience features.

## How will you process and prepare your data?

In ML workloads, the data processing pattern you choose has a big impact on flexibility and performance.

- ETL (Extract, Transform, Load) – Common in traditional data warehousing, where schema constraints require you to transform data before loading it into the target system.
- ELT (Extract, Load, Transform) – Typical for data lakes or lakehouse architecture, where raw data is loaded first, then transformed later using tools like Python or PySpark.
- EL (Extract, Load) – Common in GenAI and RAG patterns, where you store documents or media first and perform downstream transformations (like text chunking or image extraction) later.

ELT is often preferred since it preserves raw data and allows more flexible transformations during model preparation.

## Do you need a feature store?

It's often beneficial to introduce a feature store as an intermediate data layer between your aggregated data store and training environment.

A feature store acts as a catalog of curated features, complete with metadata such as feature lineage, generation time, and source. It's the perfect place to maintain "golden" training data that can be reused across multiple models or experiments.

Managed feature stores, such as the one in Azure Machine Learning, integrate directly with MLflow and other ML lifecycle tools. They enable reproducibility, governance, and version control for your features.

Treat the feature store as a sensitive data store in its own right with proper access controls, encryption, and auditing.

### Should you use a batch inference data store?

In some cases, you can improve performance and reduce costs by performing inferencing in batch, that is, pre-computing inference results and storing them for later use instead of calling the model in real time.

This approach can be highly effective when the same queries or predictions are requested repeatedly (for example, generating FAQs or standard recommendations).

Key benefits include:

- Reduced latency and improved user experience, results are served instantly.
- Easier scalability because inference can be batched and distributed offline.
- Enhanced reliability that avoids putting real-time load on the inference endpoint.
- Lower compute costs resulting from batch processing can use lower-tier hardware.
- Built-in pre-validation where results can be verified for accuracy before being exposed to users.

However, this approach works best when a significant percentage of predictions are reused. If your workload involves mostly unique queries, maintaining a batch inference store may not be worth the complexity.

Your batch inference data store should be optimized for read operations, scalable enough to handle large datasets, and integrated with your aggregate data store.

Technologies that fit this pattern include Azure Cosmos DB for fast, globally distributed access, or Azure Table Storage for simpler, lower-cost read-heavy workloads.

## Technology options

Expand table

| Function | Recommended technologies | Alternatives / complementary tools |
| --- | --- | --- |
| **Aggregated data storage** | Azure Data Lake Storage Gen2, Microsoft Fabric Lakehouse, Azure Synapse Analytics | Azure Blob Storage, SQL Database, on-premises data warehouse |
| **Data processing and transformation (ETL/ELT)** | Azure Data Factory, Azure Databricks (PySpark, SQL), Microsoft Fabric Data | Apache Airflow, Apache NiFi, Synapse Pipelines |

| Function | Recommended technologies | Alternatives / complementary tools |
| --- | --- | --- |
| | Engineering | |
| **Development and training environment** | Azure Machine Learning (with MLflow integration), Azure Databricks Workspaces | JupyterHub, Kubeflow, Amazon SageMaker |
| **Feature store** | Azure Machine Learning Feature Store, Databricks Feature Store | Feast (open source), Tecton |
| **Batch inference** | Azure Cosmos DB, Azure Table Storage | Azure SQL Database, PostgreSQL, Redis Cache |
| **Model registry and experiment tracking** | MLflow (integrated in Azure Machine Learning or Databricks) | Weights & Biases, Neptune.ai, DVC |
| **Orchestration and automation** | Azure Data Factory Pipelines, Azure Machine Learning Pipelines | Apache Airflow, Prefect |
| **Security and access control** | Microsoft Entra ID (Azure AD), Azure Key Vault, Managed Identities | HashiCorp Vault, AWS IAM |

# Next steps

Design area: MLOps and GenAIOps

Last updated on 11/20/2025

# MLOps and GenAIOps for AI workloads on Azure

Article • 11/18/2024

AI workload operations are centered on curation of data and consumption of that data. Operations ensure efficiency in achieving and maintaining the quality, reliability, security, ethics, and other standards that you prioritize for the workload.

The workload tasks can be categorized into three main areas: application development, data handling, and AI model management. Each category should adopt industry-proven operational methodologies, like **DevOps, DataOps, MLOps, and GenAIOps**.

DevOps activities cover the entire **application development lifecycle management** through automated continuous integration and continuous deployment (CI/CD) pipelines and monitoring. However, for AI workloads, the data pipeline is one of the core components. DataOps, a specialization of DevOps, focuses on **managing the data lifecycle** by streamlining processes like data extraction, transformation, and loading (ETL/ELT). DataOps practitioners typically measure data flow performance and the efficacy of data cleansing and monitor the pipeline for anomalies.

AI workloads are inherently nondeterministic. Many AI models are prone to producing different answers for the same inquiry during inference. These workloads need processes that can manage and adapt to the unpredictability of AI outputs. DataOps extends into MLOps, which **operationalizes machine learning workflows** for model training and testing. GenAIOps, a specialized subset of MLOps, targets generative AI solutions. It involves tasks like **model discovery and refining pretrained models** with enriched data.

Operational activities often overlap, and the different methodologies apply to varying degrees. For instance, in discriminative AI, DataOps plays a major role, while DevOps activities are less prominent. Conversely, in generative AI, operational excellence relies more heavily on DevOps than DataOps.

Regardless, the overall goal is capability delivery with efficient operations throughout the development lifecycle. The expected outcomes are:

- Repeatable processes with consistent results.
- Sustained accuracy of the models over time.
- Effective governance that minimizes risks.
- Change management processes for adapting to model drift.

**Automation and monitoring are your key operational strategies** to achieve those goals.

You also need to **establish standardized processes for AI components**, for routine, unplanned, and emergency operations, and have proper tracking mechanisms in place. Without those processes, you run the risk of:

- Repeated errors and nonreproducibility in data processing, model hosting, grounding data management, and other tasks.
- Low quality or stale data used for model training and refinement.
- Affecting end user confidence in the system, in the worst case leading to legal, compliance, or security issues.

You must implement established processes with the **right set of tools**. Specialized tools are available for managing AI / machine learning workflows across environments.

This article focuses on the design strategy for operations and provides tool recommendations.

# Recommendations

Here's the summary of recommendations provided in this article.

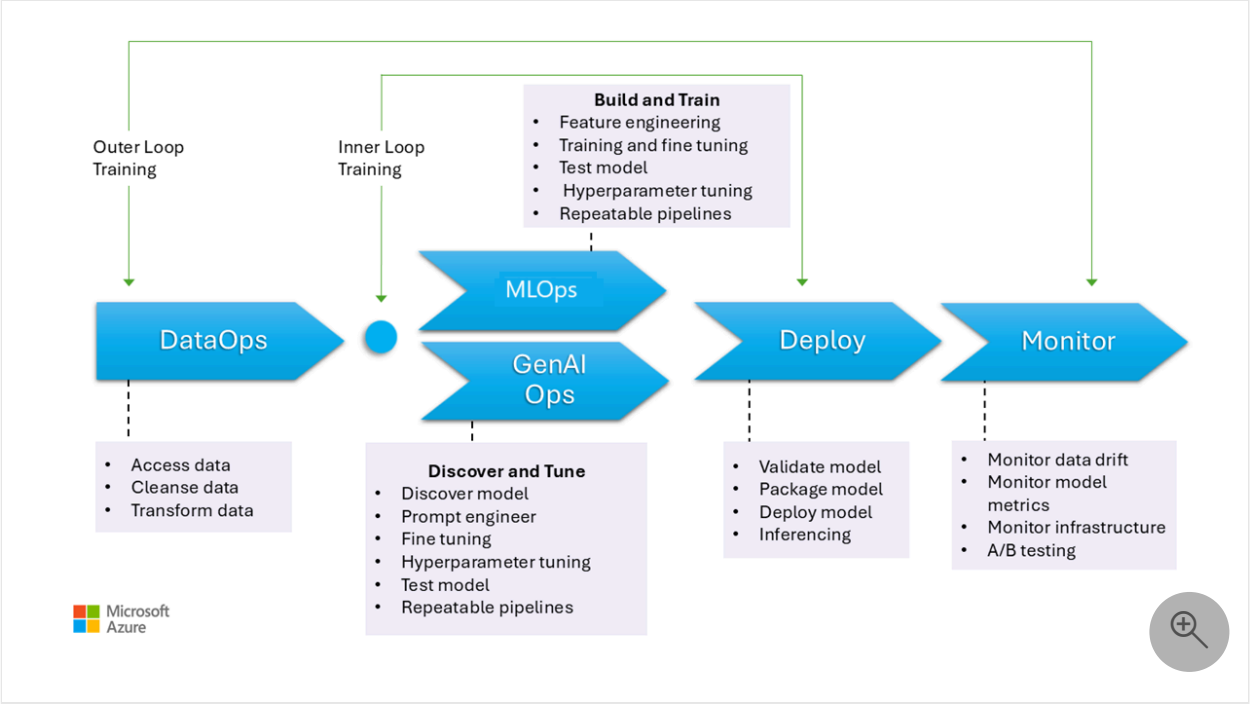
Expand table

| Recommendation | Description |
| --- | --- |
| **Design an efficient workload operations lifecycle**. | Based on the components of your AI workload different operational stages will apply to its lifecycle. It's important that you understand the best practices relevant to your scenario and the tools that are available to you to implement them. Take the the time to learn about and implement the key recommendations for all components of your workload.<br><br>▪ DataOps<br>▪ MLOps<br>▪ GenAIOps<br>▪ Monitoring |
| **Automate everything**. | Automation ensures repeatability and efficiency in a workload's lifecycle. While DevOps processes are a key contributor to this, there are additional steps that you need to take to build, test, validate, and deploy your models efficiently.<br><br>▪ Automation |

| Recommendation | Description |
| --- | --- |
| **Utilize deployment pipelines where possible.** | Deployment pipelines can help you deliver repeatable infrastructure deployments, or integrate code continuously. They're also a great tool for creating and/or validating models before promoting them to production. Implementing deployment pipelines helps you improve reliability as well as the overall user experience of your workload.<br><br>▪ Deployment Pipelines |
| **Prevent drift and decay in your models.** | You must guard against model decay and drift while also having a structured process that helps you accept new model changes in a controlled way. Following the recommendations around model maintenance helps you remain compliant, avoid unexpected user experiences, and provide a more up-to-date service.<br><br>▪ Model Maintenance |

# Workload operations lifecycle

This image illustrates the operational stages for AI models, including data gathering, cleaning data to eliminate inconsistencies or errors, and transforming data into an analyzable format. These stages are relevant for training both discriminative models and grounding data in generative models. However, the specific use case of training generative models isn't depicted in this image. That use case falls outside the scope of this article.




The stages of MLOps and GenAIOps are similar. The main difference is that, in the case of GenAIOps, the focus shifts from training to selecting the right model, prompt

engineering, and incorporating domain-specific knowledge through fine-tuning or implementing Retrieval Augmented Generation (RAG).

Even the stages for deploying and monitoring are fairly similar.

The following sections describe the common operational practices. They cover every stage of the lifecycle, from preproduction to production.

## DataOps

Data is aggregated from various production data sources and then preprocessed to remove errors and inconsistencies and to handle missing values. Finally, it's transformed and normalized into a suitable format for training or enrichment. Aspects of design are described in the Training data and Grounding data articles.

The data operations processes during this stage should be efficient because **handling large volumes of data** from multiple sources and complex data pipelines can be challenging. The approaches that you take need to ensure that this phase **produces high quality data**. Monitor this stage to track progress toward the acceptable quality bar.

You also need to ensure that data is secure, given that the data is from production. Make sure that lower environments, like Dev/Test, are as secure as production to help prevent any security issues.

> **Note**
>
> Address poor quality data by investing in extensive data cleaning during the initial phases. Take advantage of the well-known analytics patterns, like medallion, data mesh, and feature stores, to do upstream processing tasks. If the upstream phases are ineffective, you need to improve quality during the downstream phases, which leads to increased workload costs because data preparation occurs during each stage.
>
> For information about data processing tasks, see these articles:
>
> - Data preprocessing for training data
> - Data preparation for grounding data

### Tools

We recommend that you standardize your workload's data orchestration tools. The tools should be able to provide a data pipeline that can group activities and that has built-in automation.

An Azure Data Factory pipeline can be an initial choice. It can connect and process many data sources efficiently. You can also consider Azure Synapse Analytics, which combines big data and data warehousing and supports data lakes, Apache Spark, and Azure Synapse SQL. It also integrates with Data Factory for ETL.

For preparing training data, Azure Machine Learning pipelines provide specialized features that can automate tasks like data collection and processing.

Open-source technologies like Pandas (for data preparation) and Scrapy are popular choices.

## MLOps

Model training is the process of selecting appropriate algorithms and providing them with preprocessed historical data and observations to enable the model to learn patterns and make predictions.

Training the model (feature engineering) and hyperparameter tuning are iterative processes, and they're expensive. During each iteration, data scientists track results with combinations of data, code, and parameters. **Use repeatable pipelines to track experiments** with minimum manual effort until the right level of accuracy is achieved.

Another operational challenge is provisioning and scaling **specialized compute resources** where experiments are conducted. Also, you should **package and publish models** efficiently.

Teams can start with UI-based development to reduce challenges, and, as they become more confident, transition to a code-based approach.

## Tools

We recommend that you use tools that can **track machine learning experiments** by capturing details like code versions, environments, parameters, runs, and results. MLflow is one such open-source framework. Consider using Azure Machine Learning workspaces, which are MLflow-compatible and provide a streamlined workflow that enables data scientists to manage productivity and reproducibility in their projects. To manage code development with source control tracking, integrate the machine learning pipeline with source control like GitHub, or use file shares.

The hosting compute can also influence your choice of a workflow orchestrator. If your application is hosted on Azure Kubernetes Service (AKS), consider using Kubeflow.

If you're considering Azure Machine Learning, we recommend that you start with the Azure Well-Architected Framework perspective on Machine Learning to make sure you understand how the product can help you with the Well-Architected Framework quality concerns of the workload.

Part of the benefit of the process is to optimize personal time. Data scientists typically need specific tools and SDKs to effectively conduct exploratory data analysis (EDA) and experimentation from their workstations. Evaluate whether prebuilt options in Azure Machine Learning are suitable. If not, store workstation configuration or maintain approved VM images for this work. One example of an image that you can use as a starting point is the Data Science Virtual Machine (DSVM).

In certain cases, the use of VMs might be disallowed by policy. Look for alternatives, like adding Microsoft Dev Box and Azure Virtual Desktop. You can also consider using Docker to bootstrap machines that contain prebuilt images.

However, as this stage matures and you need extended experiments, move to managed compute instances and prefer options that are integrated as part of the workflow. Evaluate whether you can use Azure Machine Learning compute instances for training and inferencing for development and testing purposes. The compute clusters can handle large datasets and complex models.

Azure Machine Learning provides both code-based solutions through the SDK and low-code options like automated machine learning and the visual designer. The Python SDK offers multiple ways to train models, each with distinct features. Machine Learning also supports advanced optimization and distributed computing technologies like ONNX Runtime Training's ORTModule, DeepSpeed, and LoRA to speed up the training process.

## GenAIOps

Key activities during this stage begin with **discovering and evaluating existing models** to identify one that's pretrained for the specific use case. This is an iterative process. After you find a suitable model, it might benefit from being refined for domain-specific grounding, which also involves iterative steps and requires a certain level of orchestration.

Integrating and deploying models requires specialized tools and practices that go beyond traditional MLOps capabilities, including orchestrating models, vector indexes, prompts, and code blocks.

### Tools

To address discovery tasks, take advantage of model catalogs that include models from various providers. The model Catalog in Azure AI Studio allows you to evaluate from among curated collections and deploy models efficiently.

Azure Machine Learning prompt flow can help with the development of orchestration code, enabling prototyping, experimenting, iterating, and prompt engineering. These flows can be deployed to Azure Machine Learning managed endpoints. Evaluate whether you can run and deploy the flows with your existing CI/CD pipeline technology.

## Deployment

During this stage, the model is deployed to a hosting and inferencing platform or the serve layer of AI workloads. The APIs need to be packaged as a scalable container. The container platform can be a managed compute or custom hosting platform. Operational practices should ensure safe deployment and enable rollbacks.

Start with platform as a service (PaaS) and serverless solutions like Azure OpenAI Service to simplify adoption and management. Consider using the Azure Machine Learning Serverless API for aggregating endpoint access. Managed compute clusters are a viable option for advanced needs. Self-hosting on AKS is another option. Make sure you right-size your compute and maintain proper isolation from other workloads. You can also consider options like fully hosting your model as infrastructure as a service (IaaS). IaaS provides flexibility but can add operational burden. These options are described in Application platform.

This stage presents the last chance to catch issues before you move the model to production. Test processes should include validation steps to make sure the model is configured to provide predictions as expected.

You should integrate the model into the existing production environment by following **progressive exposure processes and using side-by-side deployments**. The canary model is a common way to roll out new models. With this method, the user base is increased gradually. Blue-green deployment is another method.

### Tools

You can use Azure Machine Learning pipelines or Azure Pipelines to deploy your models for inference. Machine Learning provides several features for simplified operations, including node provisioning, OS updates, autoscaling, monitoring, and isolated virtual networks.

Machine Learning also supports blue-green deployment, which allows a single endpoint to contain multiple deployments.

If you use other hosting platforms, like Azure Container Apps or Azure App Service, you're responsible for the operations, including provisioning and scaling. Use Azure DevOps, GitHub pipelines, or your choice of CI/CD technology in those cases.

## Monitoring

Monitoring is a key strategy and is applied at all stages. It's an ongoing process and serves as an input to quality gates that ensure AI workloads are rigorously tested to maintain consistency and reliability throughout the development lifecycle. Models must be monitored from both operational and data science perspectives.

We strongly recommend that you have a DataOps inner loop monitoring process that measures proximity to an acceptance quality bar and checks for anomalies.

For pretrained models, it also important to monitor data drift and performance, with a primary focus on relevancy. Evaluate the inputs (prompts) and the outputs (completions) to ensure they are relevant and accurate. Additionally, be aware of security risks, such as attempts to manipulate the model's behavior through malicious prompts. Make sure there's thorough content moderation that inspects data in both directions and filters out inappropriate content. These considerations are described in the ResponsibleAI design area.

After deployment, monitoring operations are necessary to address issues like model decay. Models can become stale because of changes in data or external changes that can cause the model to produce irrelevant results. As a proactive measure, use automated processes for continuous monitoring, and evaluate and retrain to maintain accuracy and relevance. Additionally, you need to monitor infrastructure and workload metrics, as you would with any other workload, to help ensure optimal performance and reliability. For more information, see Testing for model decay.

### Tools

Invest in tools that make it easier to collect metrics from inference endpoints, like Azure Machine Learning Data collector.

You also need observability of model performance, data drift, and safety and quality for generative AI.

For more information, see these articles:

- Data collection from models in production
- Model monitoring with Azure Machine Learning
- Content filtering in Azure AI Studio

# Automation

AI workloads are complex because the overall lifecycle involves many roles, frequent changes, and interrelated steps. Manual processes can be prone to errors and inconsistencies. Automation in data processing model hosting helps ensure **repeatability and efficiency**. Automation isn't always required, but it's an effective way to manage these complexities. Here are some use cases in which automation can mitigate risks:

- Unlike traditional code deployment, nondeterministic models and solutions in AI / machine learning require iterative experimentation and training. When multiple teams collaborate, automation, as a way to enforce standardized processes, can help maintain consistency, reproducibility, and effective collaboration among data scientists, engineers, and operations teams.

  The model lifecycle involves two main types of training:

  - Online training incorporates recent data into the model frequently, sometimes daily, to ensure that decisions are based on the latest information. This training is integrated into the workload so that the model is continuously updated as part of the regular process.
  - Offline training trains the model less frequently, allowing a longer gap between updates. The training process is separate from the main workload and is done asynchronously. After the new model is ready, it's then integrated into the system.

  **Reliability can be compromised if updates are infrequent**. If an update is missed, it can be postponed without major issues. This concept also applies to grounding data. For example, if you use RAG, you must decide whether you need to use recent data or if slightly older data is sufficient. Both scenarios involve balancing the need for up-to-date information with the practicality of update frequency. You should perform online training via automation because of the frequency and reliability that's required. For offline training, because of the required frequency, you need to justify automation by performing a cost-benefit analysis. Additionally, you can perform offline training by using less expensive resources, like offline hardware.

- Traditional DevOps processes are typically affected by structural changes. However, in AI and machine learning, models are trained on production data. Model decay poses a significant risk and can lead to decreased performance over time if it's not monitored. Automated collection and analysis of performance metrics, alerts, and model retraining are required to maintain model efficacy. Use automation in a way that can help you **detect changes in data and model dependencies** to get a clear understanding of the current state at any given time.

- Models can be trained with two distinct approaches.
  - **The models is trained in the development environment with full production data** and only the artifact is promoted through environments. This approach can lower computational costs but requires tighter security to handle production data in the lower environments and may not be possible in all organizations.
  - **The model is trained in each environment**. Code promotion may help with stability because training code is reviewed and tested in the lower environments, but increases the cost of compute.

  There are pros and cons to both approaches. Choosing the right approach depends on your organization's priorities and workload's Software Development Life Cycle (SDLC) practices. Regardless of the method, thorough testing and evaluation of the model before production deployment are essential

- Your automation code should **incorporate data lineage to support auditability** by providing a clear record of data processing stages. This record helps you manage expectations and enables you to demonstrate how decisions were made so you can address any concerns about outcomes.

# Deployment pipelines

In AI / machine learning workloads, model development involves **creating, validating, and promoting models** to model hosting platforms. It's important to have deployment pipelines that streamline complex workflows related to data processing, feature engineering, model training or augmentation, and deployment to production. Given the non-deterministic nature of AI, which makes processes opaque, you need to **incorporate qualitative tests in release pipelines** and monitoring systems.

Although MLOps and GenAIOps might require distinct AI activities and core technologies might differ, the underlying concepts remain similar to those of DevOps. We recommend that you apply best practices from your existing DevOps processes. **Integrate AI activities into your workload's existing pipelines**.

Typically, AI workloads involve traditional code deployments. You can choose to handle model deployment alongside the code or separately in its own lifecycle. The former approach is preferable. Be prepared to package models and inference endpoints with workload deployment to keep AI operations primarily focused on data preparation, training/fine-tuning, grounding data management, and monitoring.

Reevaluate how the following assets can be tailored to cover the entire MLOps and GenAIOps lifecycle, from preproduction to production:

- Infrastructure-as-code (IaC) tools
- CI/CD pipelines
- The observability stack for tracking and identifying issues

### Tools

You can extend Azure Pipelines and GitHub Actions workflows, commonly used for CI/CD, to machine learning models. They help deploy machine learning infrastructure, custom workload components, orchestration code, and models. Combine Azure Machine Learning pipelines with Azure DevOps or GitHub pipelines. For more information, see Use Azure Pipelines with Azure Machine Learning.

Two main factors influence your choice of the right combination of tools: use case and capabilities. For example, Azure Machine Learning pipelines are great for the orchestration that's performed by data scientists. It has a rich feature set that supports reuse, caching, and more. For tooling choices, see Which Azure pipeline technology should I use?.

## Model maintenance

The AI/ML landscape is competitive with ongoing innovation. New models emerge frequently, new use cases are discovered, and new data sources become available. As a result, model decay is a common challenge.

To prevent model performance degradation or drift over time, you need to implement automated processes for continuous monitoring, evaluation, and retraining. For example:

- **Maintain a model catalog**. Automate the process of discovering new models and updating the catalog.
- **Adapt to new use cases**. As new use cases are added to the workload requirements, anticipate the queries and adjust your data-processing logic

accordingly.

- **Incorporate new data sources**. If new data sources can potentially enhance the predictive power or relevance of your model, update your data ingestion pipeline to connect to and pull data from those sources.
- **Assess compliance with regulatory requirements**. When you adapt to new capabilities, ensure that changes remain valid within the constraints of organizational or external compliance standards.

Implement a formal process for tracking continuous improvement, and incorporate self-improvement as a subprocess within that cycle.

## Continuous evolution

Regularly review and improve operations and encourage innovation.

The MLOps maturity model progresses from manual processes to full automation. Start with manual builds and monitoring, and incorporate automated application builds, training environments, and deployment in phases as justified by comprehensive metrics. For more information, see MLOps maturity model.

The GenAIOps maturity levels move from basic models to structured deployment, progressively using automated optimization techniques. For more information, see Advance your maturity level for GenAIOps.

## Next steps

Design area: Operations

## Feedback

Was this page helpful? Yes No

# AI workload operations on Azure

Article • 11/18/2024

When you build and transition your AI workloads to production, it's important that your operations teams are fully equipped to support these workloads like they would any other production workload. Because your operations teams might have limited experience with AI technologies, it's essential to train them on these technologies and integrate the AI workloads into their workflows early in the process. Bring operations and data teams together early in the development of the AI workload to foster mutual understanding of each team's processes. This early collaboration is crucial because both teams need to work closely to support the AI workload effectively. Data teams depend on operations teams to provide reliable health signals and actionable alerts. Operations teams depend on data teams to help diagnose potential problems and resolve actual problems according to operational standards. This partnership helps ensure smooth and efficient system performance.

This guide provides recommendations for how to develop operational mechanisms and practices to improve support for AI workloads. It emphasizes efficient collaboration between operations and data teams.

## Recommendations

Here's the summary of recommendations provided in this article.

Expand table

| Recommendation | Description |
| --- | --- |
| **Monitor all aspects of your workload**. | While many general monitoring and observability concerns also apply to AI workloads, there are specific considerations that you need to work through to ensure that the whole of your workload is appropriately monitored at all times. To build your monitoring and observability strategy, you might need to work across different teams to get the right expertise and cover all relevant modes and metrics.<br><br>▪ Extend the observability platform |
| **Apply safe deployment practices to your AI workload**. | Take steps to ensure the highest level of safety around sensitive production data and align deployment approaches with zero-downtime requirements. Use the appropriate tools where necessary, and put an emphasis on not reinventing tools and processes that already exist. Often you'll be able to use established services to achieve a high level of efficiency while also enabling safe deployments. |

| Recommendation | Description |
| --- | --- |
| | ▪ Include AI workload components in your safe deployment practices |
| **Embrace DevOps practices on testing and automation**. | Apply DevOps practices when building, deploying, and operating your AI workload in production. Your workload should allow for observability and testing with real user input in production. This can only be provided in a safe manner if a strong DevOps process and streamlined automation allow for swift deployments, error correction, and A/B testing.<br><br>▪ Support testing in production<br>▪ Automate operational practices when possible<br>▪ Embrace DevOps practices |
| **Document your progress**. | Build good documentation habits from the start to allow the capturing of strategic decisions, change history, and key information about the data that your workload uses.<br><br>▪ Adopt good documentation practices |

# Extend the observability platform

To achieve operational excellence, robust observability is essential. When your organization adopts AI technologies, it's crucial that you enhance your observability platform to help ensure comprehensive monitoring of workload health. Organizations new to AI might lack big data, data science, and DataOps expertise within their operations team. Therefore, training on the operational best practices is an important first step to improve your observability platform. For this reason, operations and data teams should collaborate to determine the correct modes of monitoring and metrics to capture and analyze.

To evaluate the health of a model, a comprehensive overview of its specific quality metrics is required. Quality measurements typically include metrics such as model freshness, output correctness, and response latency. However, you should collaborate with data scientists and engineers to establish the specific metrics that define your workload's quality. The nondeterministic nature of AI workloads makes vigilant monitoring of quality especially important because those measurements can change unexpectedly at any time after deployment. Recommendations for observability include:

- Work with data scientists and engineers to determine quality metrics.
- Build or extend dashboards to assess the overall health of the workload. This approach should include component availability metrics and quality metrics.

- Implement well-designed availability and quality alerts that the operations teams can understand and take action on.
- Codify standard operating procedures that define how operations teams respond to quality alerts, like working with data teams to investigate and remediate potential malfunctions.
- Pay close attention to utilization metrics because it can be cost-intensive to run AI workloads. Costs can grow quickly if the workload team doesn't shut down, scale down, or deallocate resources when they're not in use. Operations can help ensure that costs remain within expected parameters by monitoring utilization.

# Include AI workload components in your safe deployment practices

AI workloads depend on production data, which often includes sensitive information. Therefore, it's important to maintain the highest level of safety around these workloads. To help protect your data, extend your safe deployment practices to include all of the code that's related to your workload's AI components. If you have zero-downtime requirements for your workload, design your deployment approach for the AI components accordingly.

- For inferencing endpoints, use blue-green or canary deployments with or without traffic mirroring, depending on your deployment model.
- For index serving, use a side-by-side deployment model with alias updates to cut traffic over.
- For orchestration code, use feature flags or blue-green deployments.

Depending on your application, data platform, and specific network topology, you might need to use a gateway solution, like Azure Application Gateway or Azure Front Door.

## Tools

Azure Machine Learning supports blue-green deployments natively with built-in traffic splitting.

## Don't reinvent the wheel

Because online inferencing endpoints are essentially microservices, they operate as fully self-contained workload components with their own data and code that serve a specific function in the workflow. For this reason, treat online inferencing endpoints like other critical microservices that have their own life cycles. You can update online inferencing endpoints individually. However, like other microservices in a larger workload, they must work together seamlessly. Therefore, you should prioritize integration testing when you deploy updates. Make sure that your deployments don't negatively affect other services, like model serving and the orchestrator. Alternatively, batch inferencing endpoints are often closely coupled with the job processing compute, and are included in the data pipeline. In these cases, treat them as part of a larger solution instead of as microservices.

## Support testing in production

The operations teams will likely not design or perform AI workload testing. However, they must support AI workload testing through monitoring, alerting, and investigation because testing is needed in production. Because of AI's nondeterministic nature, testing in production is necessary to help ensure that the workload is performing as expected over time. The operations team should partner closely with the workload team to capture and diagnose abnormal test results efficiently and according to operational standards. To ensure that operations can support testing and that workload teams can perform testing, both teams need to align on their processes to work together. Training on testing activities in nonproduction environments helps familiarize the team with the order of operations.

## Automate operational practices when possible

Automate all workload-related operational practices, including monitoring, alerting, and testing processes. Automation is a primary strategy to keep processes repeatable, efficient, and consistent. When you design your automation strategies, require human oversight for activities, such as properly diagnosing model inconsistencies and other quality signals. This is particularly true of initial releases. Processes that are related to maintaining the model are new for the operations teams, so the risk of improper responses to quality signals is higher at this stage. As your workload matures, you might be able to use automation to identify false alarms with well-designed quality gates, but getting to that point can be a long and complex process.

When possible, reuse existing automation tools to perform new automation tasks for your AI workload. For example, if you already use Azure Pipelines or GitHub Workflows, continue to use them for deploying orchestration code instead of using a separate tool.

However, avoid over-engineering in an effort to only use one automation tool. If certain tasks or jobs aren't well-suited to your chosen tool, choose a more appropriate tool instead of building a custom solution that adds unnecessary complexity.

> ⓘ **Note**
>
> To fully support the workload, many intersecting roles and technologies are required. Data, infrastructure, security, and operations teams all depend on automation to some extent. Use the fewest possible tools and standardize them to help keep the automation strategy manageable and easier to train on.

## Embrace DevOps practices

Early in your workload development, standardize the use of DevOps practices in your development processes. For all environments besides sandbox environments, strictly defined and enforced standards should govern the entire development life cycle. Enforce these standards by strictly prohibiting any manual deployment activities. All deployments, whether they're updates, patches, or new resource deployments, must be done through a continuous integration and continuous deployment pipeline that adheres to safe deployment practices. Good DevOps hygiene practices should include:

- **Version control:** Use version control for all code assets as much as possible. Version control is a cornerstone tenet of DevOps and good change management practices and is vital for smooth collaboration. Apply version control to library packages, including SDKs and container images. Be consistent in using a specific version of library packages across the software development life cycle (SDLC). Different versions of libraries like XGBoost or scikit-learn in various development environments can cause variations in your workload's behavior. The same is true for model versioning. Ensure that model versions are consistent across the SDLC so that you don't test one version of the model in a preproduction environment and release a different version in production.

- **Simple version naming scheme:** Use a simple version naming scheme to help ensure that you always use the most recently approved version of a given asset. AI-specific assets can include:
  - Notebook code.
  - Orchestrator code.
  - Data processing job code.
  - Machine Learning job code.
  - The model.

- **Tools that you can operationalize:** Some tools are great for experimentation but aren't designed for operationalization. This limitation can make it difficult or impossible to integrate them into your operational automation. For example, notebooks are a good tool for experimentation and exploratory data analysis work, but they aren't a good tool for production pipeline development work. When you complete your experimentation, pull the logic out of those tools and put it into a Python package that you can use for your job code.

# Adopt good documentation practices

Good documentation habits are important for all types of workloads. The complexity of AI workloads makes those habits even more important. Ensure that you have a repository that's specific to your workload team where documentation is securely stored and version controlled. Like with other workloads, you should document standard information. This standard information includes all of the tools that are used in the workload, security configurations, network designs, and setup guides. Consider documenting the following AI-specific workload information:

- **Model training and grounding data index management information:** Document data sources, the owner of data sources, refresh frequency, and bias elimination processes to establish how to train the model and how to handle grounding data.
- **History of the training process:** Detail how you arrived at the current approved configuration by documenting why you chose specific hyperparameters, temperatures, weights, and other parameters. Include what other configurations you tested and the behavioral changes you observed throughout the training process. This information helps prevent the repetition of unsuccessful or inefficient configurations.
- **Feature store information:** Document which features have the best predictive power and how you made that determination.
- **Data scientist workstation configuration:** Document workstation configurations thoroughly to streamline the onboarding process for data scientists. Specify the necessary libraries and dependencies that are required to use conda environments.
- **Data transformation information:** Document the entire process from start to finish when data transformation occurs. Document how the data appears at ingestion and how it's presented after the transformation.

## Tools

Take advantage of automated configuration and history capturing through MLFlow and Machine Learning.

## Next steps

Design area: Testing and evaluation

## Feedback

Was this page helpful? Yes No

# Test and evaluate AI workloads on Azure

09/05/2025

This article focuses on two distinct aspects: evaluating the models and testing the entire system. Evaluation and testing are often used interchangeably, but they should be considered separate processes that use distinct datasets.

*Evaluation* is an iterative activity that you do during the development phase. It focuses on experimentation to find the best model with the right level of tuning. Then, evaluate the model based on various metrics.

*Testing* includes verifying the entire system when a change is introduced, including the tuned model and non-AI components. The goal is to validate whether the workload meets identified targets and fulfills user expectations. It's also a non-negotiable change management strategy that prevents quality regressions.

Both practices are coupled in actual implementation. The entire process includes sending requests to the model, evaluating its responses, and making a go or no-go decision based on the test data. While the process is non-negotiable before production, we recommend that you conduct the process in production using a mix of real and synthetic data.

The primary focus here is on solutions built using generative AI, particularly scenarios using foundation models. For guidance applicable to training and fine-tuning models, skip ahead to Guidance for testing model training and fine-tuning.

## Use quality metrics for model evaluation

Establish a baseline and measure model quality by using metrics that align with your business goals.

Have processes that evaluate and quantify results of the user experience against a set of metrics. For instance, *Groundedness* evaluates whether a generative model's response is supported by the provided context, rather than fabricated. Suppose a legal firm develops an AI assistant that cites statutes. Without proper validation, it could draw from outdated or misclassified documents, resulting in serious consequences. A high groundedness score helps ensure that the model's output remains aligned with trusted source material.

Select and prioritize metrics based on your specific use case, monitor them continuously, and use them as decision gates for model tuning and deployment. Avoid relying on a single metric, use a combination to capture different dimensions of quality. For example, even if a model demonstrates strong groundedness, it may still produce biased outputs. Incorporate fairness evaluations for more balanced and responsible outcomes.

For information about metrics, see Monitoring evaluation metrics descriptions and use cases.

## Use the right data for evaluation

Refine your model through an iterative process using evaluation data. This dataset, often referred to as the *golden dataset* consists of trusted input-output pairs, typically created or validated by humans. It serves as the objective benchmark to assess model performance against defined quality metrics.

Ensure the dataset is representative of the domain or task, with diverse, high-quality examples and minimal noise. A limited sample size can lead to poor evaluation quality, so consider generating synthetic data when the sample data lacks sufficient diversity or coverage to improve balance and completeness.

## Validate agentic workflows

As architectures are evolving to use AI, functionality that was once handled by deterministic code is now offloaded to *AI agents*. These agents make decisions often with dynamic behavior.

Consider an agentic application where the *orchestrator* itself is implemented as an agent. Unlike traditional orchestration, agents can invoke tools, interpret prompts, collaborate with other agents, and adapt in real time, making them more flexible, but harder to validate.

This type of architecture introduces new challenges for testing and evaluation. Because agents operate non-deterministically, traditional static tests are insufficient. Testing strategy should validate the complete flow from user input to final response, including *grounding data* retrieval, tool invocation, and response generation. For example,

- **Verify that agents are calling external tools, APIs, and other agents correctly.** Use mock dependencies to validate that data is passed correctly. Simulate tool or agent failures to test reliability in behavior.
- **Design scenario-based tests using predefined prompts and expected outputs.** Because outputs may vary, evaluate results using automated scoring with another model. Also use human-based review, especially for sensitive or subjective tasks.
- **Integrate content safety tools to detect harmful, biased, or inappropriate outputs.** Include red teaming exercises to identify unexpected behaviors or jailbreak vulnerabilities. Monitor for fairness, transparency, and compliance with ethical standards.

From a tooling perspective, consider Azure AI Evaluation SDK, which supports checks like:

- Intent resolution: Does the agent/orchestrator correctly understand the user's request?

- Tool call accuracy: Are the correct tools called, with the right parameters?
- Task adherence: Does the final output align with the assigned task and prior reasoning steps?

In addition, conduct regular performance and load testing. Assess the agent's ability to scale under concurrent requests, handle long execution paths, and manage interactions across multiple agents. Continuously monitor for regressions in both logic and performance as the system evolves through iterative releases.

## Test the security aspects

Secure agentic workflows by controlling access, validating all inputs, and monitoring agent behavior to prevent misuse or unintended actions.

- **Jailbreak testing.** Always test for jailbreak attempts. Attackers typically target the orchestration layer first, which parses and forwards requests to the model. If malicious inputs aren't filtered, they can compromise model behavior.

- **Content safety.** In chat-based applications, run both user prompts and grounding context through a content safety service.

- **Endpoint security.** For RESTful interfaces, enforce strong authentication and thoroughly test security controls to prevent unauthorized access.

There are other open source libraries available such as Scikit-learn, PyTorch's torch.testing module, FairML for bias and fairness testing, and TensorFlow Model Analysis for model evaluation.

>  **Tradeoff.** Testing this code has cost implications. For example, if you use Azure OpenAI to host your inference endpoint, stress testing is a common practice that can help you determine the system's limits. However, Azure OpenAI charges for every call, which can make extensive stress testing expensive. One way to optimize charges is to use unused PTUs of Azure OpenAI in a test environment. Alternatively, you can simulate the inference endpoint by using tools like Dev Proxy.

## Test the deterministic behavior

In some architectures, you might use deterministic logic to enable orchestration. For example, instead of a non-deterministic agent orchestrator, you can choose to have an orchestrator that uses static code to manage the execution flow, such as interpreting user intent, querying the index for grounding data, and calling the model *inference endpoint*.

From a testing perspective, treat this code like any critical system component: run performance, reliability, and functional tests, especially on its routing logic. Apply unit testing to deterministic components, especially if you're using agent frameworks like Microsoft's Semantic Kernel or LangChain. These tests validate prompt templates, tool selection logic, data formatting, and decision trees, isolated from runtime variability.

## Test the inference endpoint

Inference endpoints expose your generative models through REST APIs and must be tested beyond just model accuracy. Whether you're using PaaS platforms or self-hosted servers, test the endpoint like any other endpoint to ensure reliability, scalability, and security.

- **Functional and integration testing.** Validate request handling, response structure, and integration with other components.
- **Performance and load testing.** Simulate realistic conditions to evaluate throughput, latency, and resource usage. For PaaS inference endpoints, focus on token-level metrics (tokens/sec or tokens/min), which are more meaningful than traditional request sizes of REST APIs.
- **Scaling and GPU optimization.** Test under varying load to determine the right GPU SKU or autoscaling configuration. Avoid overprovisioning by monitoring actual GPU usage.

  > **Trade-off.** GPU SKUs are expensive. It's important to continuously check whether GPU resources are underused and rightsize them, when possible. After you make adjustments, test resource usage to maintain the balance between cost efficiency and performance optimization.

- **Failure handling.** Simulate throttling, like HTTP 429 errors, backend timeouts, and service unavailability. Validate that your client handles retries, backoff, and circuit breaking appropriately.
- **Security and content safety.** For public or self-hosted endpoints, perform penetration tests and validate access controls. Use content moderation tools like Azure AI Content Safety to test and filter unsafe input/output.

## Test the grounding data workflow

The relevance of a generative AI model depends on the quality and integrity of its *grounding data*. Grounding data can be seeded to the model by using data processing pipelines. This data is preprocessed, chunked, and indexed before reaching the model. The model queries the

index in real time during user interaction, making indexing performance and accuracy critical to the user experience. Integrate testing early and maintain it throughout the system lifecycle.

Poorly tested data pipelines can lead to inconsistent results and cross-cutting concerns like security breaches. To ensure a high-quality experience, test the entire data flow, including source documents, preprocessing, orchestration logic, and the index itself. Key testing considerations include:

- **Functional and integration testing.** Validate that all data loads correctly and completely. Ensure the pipeline handles missing, empty, or synthetic data as expected.
- **Index schema compatibility.** Test schema changes to ensure backward compatibility. Any field or document changes must preserve support for older data formats.
- **Preprocessing and orchestration testing.** Grounding data preparation involves preprocessing, chunking, and embedding computation, often orchestrated by tools like Azure AI Search skill sets. Test the orchestration pipeline to ensure all steps execute correctly and the resulting data is accurate and relevant.
- **Data freshness and quality checks.** Include tests for stale data, versioning mismatches, synthetic artifacts, and empty or partial tables. Update queries or index settings as needed to reflect the most current and clean data.
- **Index load testing.** Indexes can behave differently under varying loads. Test query performance against realistic usage scenarios to inform decisions about scaling, compute SKUs, and storage requirements.
- **Security testing.** If documents are partitioned with access controls, rigorously test those controls. Ensure that each user or role only accesses permitted content to maintain confidentiality and compliance.

# Guidance for testing model training and fine-tuning

Similar to generative AI models, use various types of tests at different stages of the development life cycle and across different system components and flows. As much as practical, develop workload assets with testing in mind. For instance, when you perform data manipulation and reshape source data for feature engineering, adhere to good coding practices and ensure that you structure the code to support testing.

## Evaluate the model

Apply a baselining strategy during model training to measure and compare model quality. Evaluate the performance of various combinations of models, parameters, and features by using well-defined metrics. These metrics provide objective, data-driven scores that you can iteratively compare across versions and configurations to identify the best-performing model.

For more information, see Regression/forecasting metrics.

## Data to evaluate and test

Partition source data into three distinct datasets: training, evaluation, and testing. Use the training dataset to build the model, the evaluation dataset to tune it, and the test dataset to validate final performance.

Ensure each dataset contains high-quality data to reduce noise. Use test cases in data pipelines to enforce quality, and supplement with synthetic data when real samples are limited, in domains like fraud detection, where real fraud instances are rare and provide limited data for training reliable models.

Keep all datasets separate and non-overlapping to maintain objectivity and prevent bias in predictions. Don't reuse training data for evaluation or evaluation data for testing.

## Testing the training and fine-tuning workflow

- **Data pipeline technologies.** Combine functional, load, and performance tests using synthetic data to assess scalability and make informed decisions about sizing or product suitability, required SKUs, and system integration.
- **Ingestion workflow.** Test ETL/ELT pipelines end-to-end to make sure that they ingest data reliably and that the data is high quality. Also, test integration with all connected systems and monitor external dependencies. Use synthetic data to validate end-to-end processing, particularly for complex or high-volume workloads.

  Test scheduled jobs to validate that ingestion tasks complete on time and return expected volumes.
- **Data quality on ingestion.** Verify that data cleansing and processing include tests to confirm that data manipulation functions as intended. Include checks for completeness, freshness, schema consistency, uniqueness, and relevance. Also verify that structured data is ingested without duplicates, missing values, or invalid entries.
- **Feature and label integrity.** Validate that features are correctly calculated and labels accurately assigned, especially when using complex rules. Check for data leakage to prevent future or label-derived information from contaminating training data. Also, verify

that data splits are appropriate to avoid biased or overlapping samples because even subtle leakage can harm model performance.

- **Hyperparameter testing.** Hyperparameter testing is an iterative process where model parameters are tuned to meet accuracy goals based on your workload's use case. This involves repeatedly training on selected data and evaluating on test data to validate performance. Start with a smaller dataset to quickly assess model behavior, then scale testing to the full set. Be mindful of the trade-off between model accuracy and the computational cost and time required for repeated training and evaluation.

- **Code quality.** When training models using custom code, like with PyTorch script, run load tests during the design phase to evaluate compute requirements and select appropriate SKUs. Use unit tests to catch regressions during development, and rely on manual testing when automation isn't feasible. Since these scripts run within workflows, add integration tests to verify that scripts are executed reliably within the pipeline.

- **Inference endpoint.** This REST API provides access to the trained machine learning model for making predictions. The model is deployed to an environment with an endpoint that can receive real-time or batch input data, process it, and return predictions. Like any other API, make sure that the inference endpoint undergoes functional, performance, and security testing validate that it returns accurate results, handles expected load, and remains secure against misuse.

- **Live-site testing.** Extend functional testing into the live system. Run scheduled tests to validate data volumes, detect missing or duplicate records, and confirm data freshness. Use synthetic data to safely validate end-to-end transformations and logic under production conditions. Incorporate A/B tests to evaluate new experiences and prevent quality regressions before full deployment. Configure alerting to trigger immediate investigation when tests fail.

Integrate data testing into CI/CD pipelines by automating unit and functional tests, especially during code changes or pipeline updates. Add quality checks before retraining, and use side-by-side deployments to safely test in production. Set up alerting for test failures or ingestion anomalies.

> **Note**
>
> Testing and monitoring serve different purposes. Conduct tests to evaluate potential changes to the system, typically before you implement any changes. Conduct monitoring continuously to assess the overall health of the system.

## Testing for model decay

All models degrade over time due to internal and external changes in the workload. This decline in model quality, known as model decay, can occur in two ways:

- **Data drift** happens when input data changes, making the model outdated. For example, a model that predicts voting patterns becomes irrelevant because of demographic shifts after redistricting.
- **Concept drift** occurs when external conditions change, causing the model's predictions to no longer reflect reality. For example, a model that predicts sales trends becomes irrelevant because there's a change in consumer behavior after a competitor launches a new product.

To detect decay, use automated testing to compare predictions against actual outcomes, and monitor for drift using statistical metrics. User feedback, for example using thumbs up/down, is also a valuable signal for identifying issues. When potential decay is detected, the operations team should alert data scientists to investigate and determine root causes and next steps.

## Test tools

Consider these Azure resources:

- Azure Machine Learning data collector for real-time logging of input/output data for models deployed to managed or Kubernetes online endpoints.
- Azure Machine Learning Model Monitoring for automated monitoring by comparing real-time inference data with reference datasets to track drift and other performance metrics. See the best practices for more recommendations.

# Next steps

Design area: Responsible AI

# Responsible AI in Azure workloads

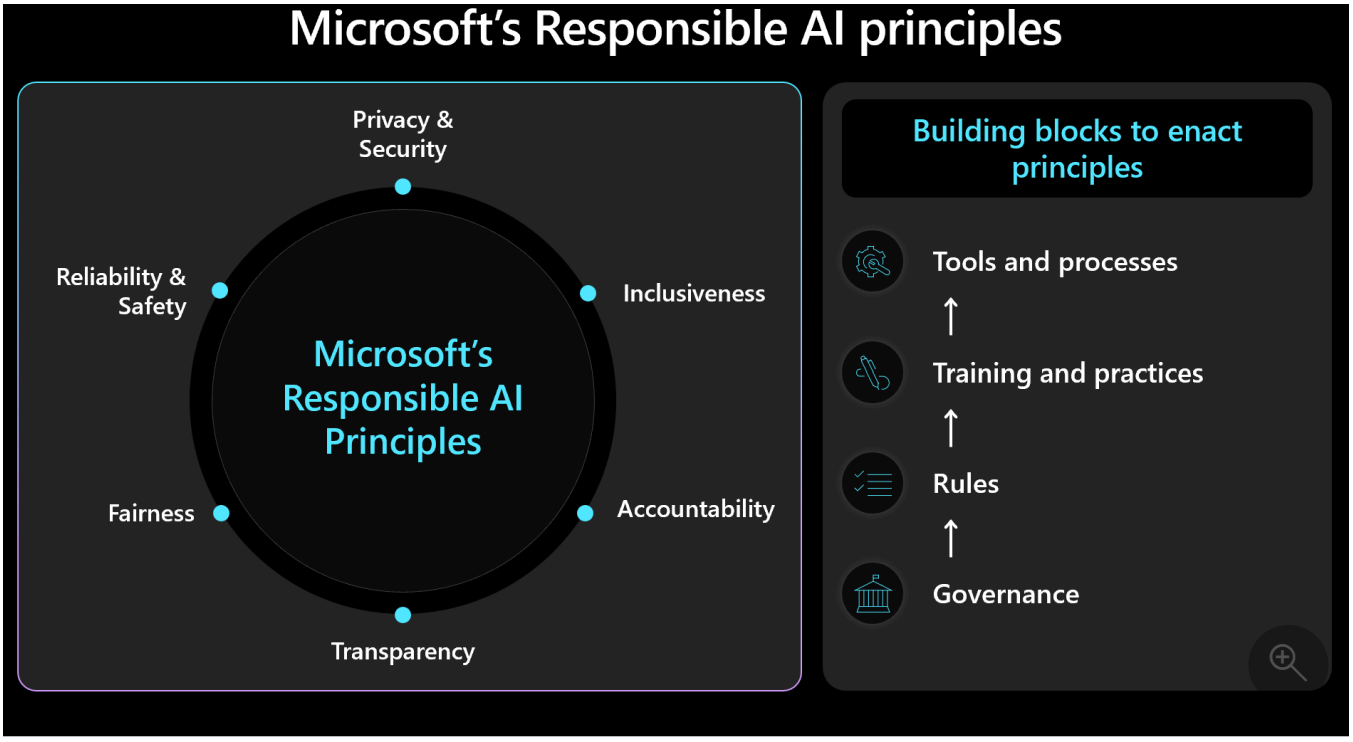
07/14/2025

The goal of responsible AI in workload design is to help ensure that the use of AI algorithms is **fair**, **transparent**, and **inclusive**. Microsoft Azure Well-Architected Framework security principles are interrelated and focus on **confidentiality** and **integrity**. Security measures must be in place to maintain user privacy, protect data, and safeguard the integrity of the design. The design shouldn't be misused for unintended purposes.

In AI workloads, models often use opaque logic to make decisions. **Users should trust the system's functionality and feel confident that models make these decisions responsibly.** Unacceptable behaviors, such as manipulation, content toxicity, IP infringement, and fabricated responses, must be prevented.

Consider a use case in which a media entertainment company wants to provide recommendations by using AI models. If the company doesn't implement responsible AI and proper security protocols, a bad actor might take control of the models. The model might recommend content that causes harm. For the organization, this behavior can result in brand damage, unsafe environments, and legal problems. Therefore, maintaining appropriate vigilance throughout the system's life cycle is essential and nonnegotiable.

You should prioritize security and workload management and keep human outcomes in mind when you make design decisions. Familiarize yourself with the Microsoft framework for responsible AI and ensure that you measure and implement the framework's principles in your design. The following image shows the core concepts of the framework.



> **Important**
>
> The accuracy of predictions and the metrics for responsible AI are typically interconnected. By improving a model's accuracy, you can enhance its fairness and alignment with reality. Responsible AI frequently aligns with accuracy, but accuracy alone doesn't include all safety considerations. It's crucial to validate these principles responsibly.

This article provides recommendations about responsible decision-making, validating user input, and helping to ensure a safe user experience. It also provides guidance about data security to help protect user data.

# Recommendations

The following table summarizes the recommendations in this article.

Expand table

| Recommendation | Description |
| --- | --- |
| **Develop policies that enforce moral practices at each stage of the life cycle.** | Include checklist items that explicitly state safety requirements and are tailored to the workload context. Examples include user data transparency, consent configuration, and procedures for how to handle the right to be forgotten (RTBF).<br><br>▪ Develop your policies for responsible AI<br>▪ Enforce governance on policies for responsible AI |
| **Protect user data with the goal to maximize privacy.** | Collect only what's necessary and with proper user consent. Apply technical controls to protect the profiles of users, their data, and access to that data.<br><br>▪ Handle user data appropriately<br>▪ Inspect incoming and outgoing data |
| **Keep AI decisions clear and understandable.** | Clearly explain how recommendation algorithms work. Provide users with insights into data usage and algorithmic decision-making to help them understand and trust the process.<br><br>▪ Make the user experience safe |
| **Implement responsible practices for agentic AI systems.** | For agents accessing public and private data, executing tasks in internal and external systems, and autonomously driving decisions, implement auditability of agent activities, role-based access control, and circuit breaker functionality. |

| Recommendation | Description |
| --- | --- |
| | ▪ Implement agentic AI safeguards |

# Develop policies for responsible AI

**Document your approach to responsible AI usage.** Explicitly state policies that you apply at each stage of the life cycle so that the workload team understands their responsibilities. Microsoft standards for responsible AI provide guidelines, but you must define what these guidelines mean specifically for your context.

For example, the policies should include checklist items for mechanisms that support user data transparency and consent configuration. Ideally, these mechanisms should allow users to opt out of data inclusion. Data pipelines, analysis, model training, and other stages all must respect that choice. Another example is procedures for handling the RTBF. Consult your organization's ethics department and legal team to make informed decisions.

Create transparent policies for data use and algorithmic decision-making to help users understand and trust the process. Document these decisions to maintain a clear history for potential future litigation.

Responsible AI implementation includes three key roles: the research team, the policy team, and the engineering team. Collaboration among these teams should be operationalized. If your organization has an existing team, use their work. Otherwise, establish these practices yourself.

Each team should have their own responsibilities. For example:

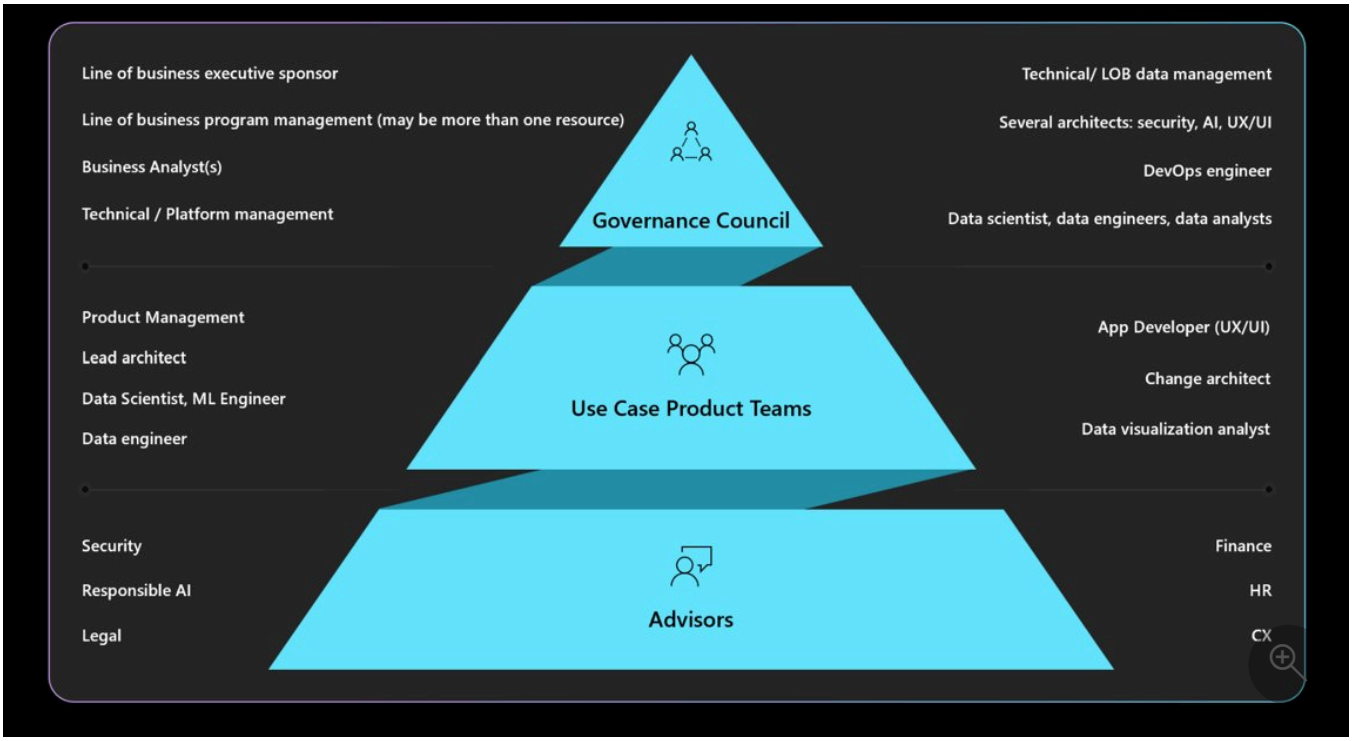
- **The research team conducts risk discovery** by consulting organizational guidelines, industry standards, laws, regulations, and known red-team tactics.
- **The policy team develops policies that are specific to the workload.** They incorporate guidelines from the parent organization and government regulations.
- **The engineering team implements the policies** into their processes and deliverables. The team validates and tests for adherence.

Each team formalizes its guidelines, but the **workload team must be accountable for its own documented practices**. The team should clearly document any extra steps or intentional deviations to make sure that there's no ambiguity about what is allowed. The team should also be transparent about any potential shortcomings or unexpected results in the solution.

# Enforce governance on policies for responsible AI

Design your workload to **comply with organizational and regulatory governance**. For example, if transparency is an organizational requirement, determine how it applies to your workload. Identify areas in your design, life cycle, code, or other components where you should introduce transparency features to meet that standard.

Understand the required governance, accountability, review boards, and reporting mandates. Ensure that your governance council **approves and signs off on workload designs** to avoid redesigns and mitigate safety or privacy concerns. You might need to go through multiple layers of approval. The following diagram outlines a typical governance structure in an organization.




For more information about organizational policies and approvers, see Define a responsible AI strategy.

# Make the user experience safe

User experiences should be based on industry guidelines. Use the Microsoft Human-AI Experiences Design Library, which includes principles and provides implementation guidelines. It also gives examples from Microsoft products and other industry sources.

There are workload responsibilities throughout the life cycle of user interaction. They start with a user's intent to use the system, and they continue throughout a session and during any disruptions that system errors cause. Consider the following practices:

- **Build transparency.** Make users aware of how the system generates responses to their query.

Include links to data sources that the model consults for predictions. This practice increases user confidence by showing them the origins of the information. Data design should include these sources in the metadata. For example, when the orchestrator in a retrieval-augmented application performs a search, it retrieves 20 document chunks and sends the top 10 chunks to the model as context. The top 10 chunks belong to three different documents. The UI can then reference these three source documents when it displays the model's response. This transparency increases user trust.

Transparency becomes more important when you use agents, which act as intermediaries between front-end interfaces and back-end systems. For example, in a ticketing system, the orchestration code interprets user intent and makes application programming interface (API) calls to agents to retrieve necessary information. Exposing these interactions helps make the user aware of the system's actions.

For automated workflows that include multiple agents, create log files that record each step. Log files can help you identify and correct errors. They also give users an explanation for decisions, which operationalizes transparency.

> **⊗ Caution**
>
> When you implement transparency recommendations, avoid overwhelming the user with too much information. Take a gradual approach by using minimally disruptive UI methods.
>
> For example, display a tooltip that shows a confidence score from the model. You can incorporate links, such as links to source documents, that users can select for more details. This user-initiated method keeps the UI nondisruptive and lets users seek more information only if they choose to.

- **Collect feedback.** Implement feedback mechanisms.

  Avoid overwhelming users with extensive questionnaires after each response. Use quick and simple feedback mechanisms instead, like thumbs up or thumbs down, or a rating system for specific aspects of the answer on a scale of 1 to 5. These methods help improve the system over time and allow for granular feedback without being intrusive. Be mindful of potential fairness issues in feedback because there might be secondary reasons behind user responses.

  Implementing a feedback mechanism affects architecture because of the need for data storage. Treat feedback like user data and apply levels of privacy control as needed.

  In addition to response feedback, collect feedback about the efficacy of the user experience. Collect engagement metrics through your monitoring stack of the system.

Ensure that the feedback mechanism provides functionality to allow users **to contest AI decisions**. For example, a financial institution may use AI to automatically approve or reject credit applications, and users should be able to contest rejections. Establish clear procedures for the workload team to review and address contested decisions.

- **Apply ethical safeguards for AI-genereated audio and video.** Users have the right to know when they're interacting with AI-generated media rather than human-created content. Implement clear disclosure mechanisms that inform users when content is AI-generated without disrupting the user experience, and provide options for human interaction when users prefer or require human oversight for sensitive scenarios.

  Consider implementing content watermarking or metadata tagging for AI-generated media to support transparency and traceability. Document your approach to handling requests for content verification and establish clear policies for content authenticity disclosure.

# Implement agentic AI safeguards

For agents accessing public and private data, executing tasks in internal and external systems, and autonomously driving decisions, **implement auditability of agent activities, role-based access control, and circuit breaker functionality to prevent catastrophic outcomes**.

Agentic AI systems require specialized responsible AI practices beyond traditional machine learning models, built on three foundational aspects:

- **Robust data ingress and egress control.** Establish comprehensive governance for data flows into and out of agentic systems. This includes validating data quality and security at system boundaries, ensuring proper authorization for data access, and maintaining clear separation between different agent contexts and external systems.

- **Data validation and integrity assurance.** Maintain data quality and consistency throughout multi-agent interactions. This involves continuous monitoring of data transformations, comprehensive audit trails for decision-making processes, and verification mechanisms to prevent data corruption or manipulation across agent workflows.

- **Autonomous agentic execution with independent guardrails.** Deploy oversight mechanisms that operate independently from the agents themselves. This includes monitoring systems that can detect anomalous behavior, automated controls that can halt problematic executions, and human intervention capabilities that allow operators to override agent decisions at any point.

## Agent complexity considerations

Different types of agents require varying levels of these safeguards because the potential risks and impact of their actions increase with their capabilities and autonomy. As agents move from simple retrieval tasks to complex autonomous decision-making, the need for comprehensive oversight and control mechanisms becomes more critical.

Additionally, the level of complexity increases significantly, making it more difficult to implement responsible AI principles like fairness and transparency:

- **Retrieval agents (read-only):** Focus on data access controls and audit logging.
- **Task-based agents (read and write):** Require comprehensive authorization and transaction monitoring.
- **Fully autonomous agents (multi-turn):** Need all three aspects with the highest level of oversight.

# Operationalize content safety measures

Integrate content safety into every stage of the AI life cycle by using custom solution code, appropriate tools, and effective security practices. Consider the following strategies:

- **Anonymize data.** As data moves from ingestion to training or evaluation, implement checks along the way to minimize the risk of personal information leakage and avoid raw user data exposure. Use intelligent tooling like Azure AI Language Personally Identifiable Information (PII) detection to automatically detect and redact personal information like phone numbers, email addresses, and other types of personal information.
- **Moderate content.** Use the content safety API that evaluates requests and responses in real time. Ensure that these APIs are reachable.
- **Identify and mitigate threats.** Apply well-known security practices to your AI scenarios. For example, conduct threat modeling and then document the threats and how you mitigated them. Typical security practices like red team exercises apply to AI workloads. Red teams can test whether models can be manipulated to generate harmful content. These activities should be integrated into AI operations.

  For information, see Plan red teaming for large language models and their applications.
- **Use the right metrics.** Use metrics that effectively measure the behavior of the model. **Metrics vary depending on the type of AI model.** In some cases, measurement of generative models might not apply to regression models. For example, a model predicts life expectancy, and the results affect insurance rates. Fairness issues in this model can

result in fairness-related harms. This problem stems from deviations in core metric testing because fairness and accuracy metrics are typically interconnected. Improve accuracy to help reduce fairness-related harms.

- **Add appropriate instrumentation.** AI model results must be explainable. **You need to justify and trace how inferences are made**, including the training data, the calculated features, and the grounding data. In discriminative AI, you can justify decisions step by step. However, for generative models, explaining outcomes can be complex. **Document the decision-making process** to address potential legal implications and provide transparency.

  You should implement this explainability aspect throughout the entire AI life cycle. Data cleaning, lineage, selection criteria, and processing are critical stages where decisions should be tracked.

- **Build escape hatches into your agentic design.** For agentic solutions, implement mechanisms that allow human intervention or override at critical decision points. This approach ensures that risky behaviors can be avoided or corrected early in the execution process. Escape hatches are particularly important in complex agentic workflows where multiple agents interact and make autonomous decisions that can have significant business or safety implications.

  Design your architecture to enable continuous tracing and monitoring of agent interactions across the digital ecosystem. Implement governance agents that operate across different system components to identify and flag concerns before they escalate. Consider these essential patterns:
  - **Coordinator agents** that monitor task execution and escalate anomalies to human operators when predefined thresholds are exceeded or unusual patterns are detected.
  - **Human-in-the-loop checkpoints** where agents pause execution for validation before proceeding with high-risk or high-impact actions.

  Build interception points at all critical junctures in the execution path, including routing decisions, resource allocation choices, and external system integrations. This multi-layered approach provides multiple opportunities to prevent unintended consequences while maintaining system efficiency.

- **Make technical decisions about the AI system auditable.** Ensure that decisions made about AI technologies are recorded and auditable. You might include the record of these decisions in a larger audit trail for the workload, but ensure that necessary details are included to meet governance and compliance standards and requirements. Decisions for AI technologies can include:
  - Model selections
  - Model updates

- Algorithm adjustments and hyper-parameters
- Changes to your data processing design

### Tools

Integrate tools for content safety and data traceability, like Microsoft Purview. Azure AI Content Safety APIs can be called from your testing to facilitate content safety testing.

Azure AI Foundry provides metrics that evaluate the behavior of the model. For more information, see Evaluation and monitoring metrics for generative AI.

For training models, see the metrics that Azure Machine Learning provides.

## Inspect incoming and outgoing data

Prompt injection attacks, such as jailbreaking, are a common concern for AI workloads. In this case, some users might attempt to misuse the model for unintended purposes. To help ensure safety, **inspect data to prevent attacks and filter out inappropriate content**. Apply this analysis to both the user's input and the system's responses to help ensure thorough content moderation in incoming flows and outgoing flows.

In some cases, you need to make multiple model invocations, such as through Azure OpenAI in Foundry Models, to serve a single client request. In these scenarios, applying content safety checks to every invocation can be costly and unnecessary. Consider **centralizing that work in the architecture while keeping security as a server-side responsibility**. Suppose an architecture has a gateway in front of the model inference endpoint to offload specific back-end capabilities. You can design that gateway to handle content safety checks for requests and responses that the back end might not support natively. Although a gateway is a common solution, an orchestration layer can handle these tasks effectively in simpler architectures. In both cases, you can selectively apply these checks when needed, which optimizes performance and cost.

**Inspections should be multimodal** and cover various formats. When you use multimodal inputs, such as images, it's important to analyze them for hidden messages that might be harmful or violent. These messages might not be immediately visible, so they require careful inspection. Use tools like Content Safety APIs for this purpose.

To help enforce privacy and data security policies, inspect user data and grounding data for compliance with privacy regulations. Make sure that the data is sanitized or filtered as it flows through the system. For example, data from prior customer support conversations can serve as grounding data. This data should be sanitized before reuse.

> **Important**
>
> **Content safety validation is essential for all AI scenarios**, regardless of architecture complexity. Implement content safety checks using services like Azure AI Content Safety to evaluate both requests and responses before they reach or leave your AI models.

# Handle user data appropriately

Responsible practices involve careful handling of user data management. This management includes knowing when to use data and when to avoid relying on user data.

- **Practice inference without sharing user data.** To securely share user data with other organizations for insights, **use a clearinghouse model**. In this scenario, organizations provide data to a trusted partner that trains the model by using the aggregated data. Then all institutions can use this model and share insights without exposing individual datasets. The goal is to use the model's inference capabilities without sharing detailed training data.

- **Promote diversity and inclusivity.** When user data is necessary, use a diverse range of data, including underrepresented genres and creators, to mitigate fairness-related harms. Implement features that encourage users to explore new and varied content. Monitor usage on an ongoing basis and adjust recommendations to avoid overrepresenting any single content type.

- **Respect the RTBF.** Avoid using user data when possible. Help ensure compliance with the RTBF by having the necessary measures in place to make sure that user data is deleted diligently.

  To help ensure compliance, there might be requests to remove user data from the system. For smaller models, you can remove user data by retraining the model by using data that excludes personal information. For larger models, which can consist of several smaller, independently trained models, the process is more complex and the cost and effort is significant. Seek legal and ethical guidance about handling these situations and make sure to include the guidance in your policies for responsible AI.

- **Retain data responsibly.** When data deletion isn't possible, **obtain explicit user consent for data collection** and provide clear privacy policies. **Collect and retain data only when absolutely necessary.** Have operations in place to remove data aggressively when it's no longer needed. For example, clear chat history as soon as practical, and anonymize sensitive data before retention. Use advanced encryption methods for this data at rest.

- **Support explainability.** Trace decisions in the system to support explainability requirements. Develop clear explanations of how recommendation algorithms work. Provide users with insights into why specific content is recommended to them. The goal is to ensure that AI workloads and their results are transparent and justifiable by detailing how they make decisions, what data they use, and how models were trained.

- **Encrypt user data.** Input data must be encrypted at every stage of the data processing pipeline from the moment that the user enters data. These stages include data as it moves from one point to another, data that's stored, and data that's inferenced, if necessary. Balance security and functionality and aim to **keep data private throughout its life cycle**.

- **Provide robust access controls.** Several types of identities can potentially access user data. Implement role-based access control for both the control plane and data plane so that it covers user and system-to-system communication.

  Maintain proper user segmentation to protect privacy. For example, Microsoft 365 Copilot can search and provide answers based on a user's specific documents and emails, while ensuring that only content relevant to that user is accessed.

- **Reduce surface area.** A fundamental strategy of the Well-Architected Framework Security pillar is to minimize the attack surface and harden resources. You should apply this strategy to standard endpoint security practices by tightly controlling API endpoints, exposing only essential data, and avoiding extraneous information in responses. Balance the design choice between flexibility and control.

  Make sure that there aren't any anonymous endpoints. In general, avoid giving users more control than necessary. In most scenarios, users don't need to adjust hyperparameters except in experimental environments. For typical use cases, such as interacting with a virtual agent, users should only control essential aspects to help ensure security by limiting unnecessary control.

For more information, see Application design for AI workloads on Azure.

# Next step

Workload team personas for AI workloads

# Workload team personas for AI workloads

08/18/2025

In the context of building AI workloads, in contrast to traditional code deployment, nondeterministic models require iterative experimentation and collaboration across multiple roles and teams. Early integration of operations, application development, and data teams is essential to foster mutual understanding. This collaboration demands diverse skills and continuous learning to keep pace with technological advancements.

Effective collaboration hinges on *integrating tools, processes, and people* and is driven by workload needs and specific goals. The recommended strategies include:

- Establishing clear roles and accountabilities.
- Taking advantage of your team's skill set for appropriate tasks.
- Standardizing processes and subprocesses, such as tracking work as part of a shared backlog.
- Relying on automation to achieve consistency and reproducibility.

Personas can be an effective tool for materializing those strategies and standardizing responsibilities. This article describes the personas for AI workloads and their benefits in workload design. It also provides examples and tools for defining and using these team-level personas effectively.

## What are personas?

Personas represent subsets of humans and processes that are involved in the creation and operation of a workload. Personas capture both the roles and the real behaviors and accountabilities of these individuals and processes. An individual can embody one or multiple personas, depending on context. A persona doesn't have to be a person. It can also be an unattended process, such as an agent process in the architecture.

Your workload might have user personas that drive feature development. Those personas aren't in scope for this article.

Unlike roles, which are relatively static functions or positions within an organization, personas are dynamic and goal oriented. They can be used to map skill requirements to processes and tools, like architectural components. Personas primarily help define the scope of responsibility and set context within a project. They provide several other benefits, such as:

- **Identification of resource gaps.** Identifying gaps helps you decide whether to recruit or train resources or redesign the solution. If your workload team lacks individuals that fit a necessary persona, you might need to adjust the architecture, modify the process, or

onboard new personnel. For example, if a senior data science persona is missing, you can redesign the architecture to rely more on general-purpose software as a service (SaaS) AI solutions or incorporate non-Microsoft AI solutions.

- **Enhanced skills.** Mapping personas to specific architectural components also facilitates educational opportunities, like sessions and online courses to enhance skills.
- **Ensuring appropriate levels of access.** You should use personas to define security and access needs by mapping personas to processes, architectures, and services. This mapping helps to ensure appropriate access levels.
- **Facilitating project planning and communication.** In project planning, personas help identify key interactions to facilitate the setup of sync meetings and overall planning. Typically, personas are integrated into the hierarchy of tracking user stories, features, and requirements to streamline project management.

# Agentic personas

Agentic team roles need different persona management than traditional human-centered or traditional process-centered roles because they work fundamentally differently from deterministic systems. Unlike traditional systems where you know exactly which services will be called and in what order, agentic systems make decisions on their own at runtime. These decisions depend on the data they receive and how well the AI models perform. This creates complexity in managing authentication, authorization, and controlling what tools and agents can access.

Agentic workloads require specialized persona management built on key areas:

- **Multi-cloud and cross-system persona definition**. Set up persona governance that works across different cloud platforms, SaaS services, and Model Context Protocol (MCP) servers, and not just a single cloud environment. This means defining agent personas with the right access permissions, dynamic authentication methods like agent-to-agent (A2A), and clear rules for interactions across different platforms.

- **Dynamic persona permissions**. Create persona models that support flexible, workflow-aware access control. This includes providing permissions just when they're needed, granting tool-specific access during different phases, and being able to increase or restrict agent capabilities based on the situation and risk level. Unlike traditional managed identities that follow predictable patterns, agent personas must handle unpredictable decision-making and runtime tool selection.

- **Autonomous agent persona accountability**. Set up persona governance that keeps clear accountability chains for what agents do, since they make their own decisions at runtime. This includes audit trails that connect agent decisions back to responsible human personas, ways for humans to step in and override agent decisions, and clear escalation

paths when agents go beyond their defined boundaries. Because agent behavior is non-deterministic, calling different tools and agents based on data and model performance, you need stronger monitoring and governance than traditional automation systems.

Consider a CI/CD pipeline for an AI workload with agentic personas for code review, testing orchestration, and deployment validation. When a developer submits code changes, the Dev Router agent analyzes the code changes and routes the request dynamically to the right testing agent, based on the type of changes. Unlike traditional pipelines with fixed routing rules, this approach allows agent personas to join or leave the system dynamically, creating non-deterministic behavior that needs specialized persona management for governance, security, and accountability.

Each agent persona needs different permissions. For example, the unit testing agent needs access to the repo where the source is located, but a load testing agent may also need access to Azure AI Foundry, Azure Monitor and other resources. Because agent interactions are dynamic, you need just-in-time permission provisioning and audit trails that connect autonomous agent decisions back to responsible human personas. This ensures accountability even when agents make unexpected tool choices at runtime.

# How to define personas

Identify your team members' specializations and align them with the appropriate roles in your AI operations or design. Create a template to document personas' skill expectations, team information, and the processes in which they'll be involved.

Here's an example baseline template:

Expand table

| Persona template |
| --- |
| 🔷Persona name: [Name]<br>🔷Team: [Team responsible for the persona]<br>🔷Primary interaction: [Other teams or agents the persona interacts with]<br>🔷Component access: [Security and access requirements for processes and system components]<br>🔷Processes: [Processes the persona is responsible for or contributes to]<br>🔷Skills: [Skills required to complete the tasks, including domain and technology specifics like model training or search index optimization] |

## Tools

You can use a table to organize and visualize information for each persona. One advantage to this method is that you can create and link to other tables that provide more specific

information. For example, you can link architecture components to another table where identity-based access control is specified for each service and environment (Dev, Stage, Production).

> **Tradeoff.** Having too few personas can make it difficult to implement role-based access control with least privileged access and to distribute work responsibilities effectively. Conversely, having too many personas adds management overhead. Starting with between 5 and 10 personas is a good balance, and you should only add personas that are necessary for your operations.

You can also use cards to define personas. These cards contain the same information as the table, or a quick summary. You can use PowerPoint or make a set of Markdown files to create these cards.

In certain cases, you can use a combination of tools. For example, each architecture component in a persona card can open a Markdown file that includes a table that maps security and role-based access control for each service and environment. For an example, see MLOps accelerator: Identity RBAC.

## Example personas

The example personas represent the teams that participate in developing and operating AI workload solutions. These include both human personas and automated agent personas that help build and manage the solution, which are distinct from the agents that implement the actual business functionality. Even in non-agentic solutions, automated processes run under specific identities to support development and operations workflows.

You can use cards to define the services a persona needs to be able to access within a process and outline the skills required for each persona (whether it's a person or an agent).

> **Important**
>
> Although the personas defined here serve as baseline examples, we recommend that you create your own personas by using tools like tables, persona template cards, and graphs.
>
> It's important that these personas align with your processes, organization, and users.

### Development personas

Expand table

| GenAI Data Scientist (P006) |
| --- |
| Team: AI Team<br>◆ Primary interaction: Data Ingestion Team, DevOps Team<br>◆ Component access: Azure AI Foundry, Azure OpenAI Service, Azure AI Search, Azure Storage, Azure Key Vault<br>◆ Processes: GenAIOps, inner loop development<br>◆ Skills: Azure AI Foundry, Azure OpenAI Service, Python, model knowledge (LLM, SLM), fine-tuning, RAG, agentic solutions |

Expand table

| GenAI Chat Developer (P007) |
| --- |
| Team: Engineering Team<br>◆ Primary interaction: AI Team<br>◆ Component access: Azure AI Foundry, Azure Web Apps, Azure API Management, Azure Cosmos DB, Azure Container Apps, Azure Functions<br>◆ Processes: DevOps, event-driven processing, microservices, inner loop development<br>◆ Skills: Web application architecture (front end/back end), React, Node.js, HTML, CSS, agentic solutions |

Expand table

| Dev Router Agent (P010) |
| --- |
| Team: Engineering Team (automation)<br>◆ Primary interaction: Unit Test Agent, Load Test Agent<br>◆ Component access: Azure DevOps, GitHub, Azure AI Foundry, Azure Container Registry<br>◆ Processes: Automated agent routing, DevOps<br>◆ Skills: Python, Agent-2-Agent |

Expand table

| Dev Unit Test Agent (P011) |
| --- |
| Team: Engineering Team (automated)<br>◆ Primary interaction: Dev Router Agent, AI Team, Engineering Team<br>◆ Component access: Azure DevOps, GitHub, Azure AI Foundry, Azure Container Apps, MCP Server Tools<br>◆ DevOps<br>◆ Skills: Test automation, code coverage analysis, performance benchmarking, mcp-testing frameworks |

## Operations personas

Expand table

**AI Data Engineer (P001)**

Team: Data Ingestion Team
- Primary interaction: AI Development Team, Operations Team
- Component access: Azure Data Factory, Azure Databricks, Azure AI Foundry, Azure SQL Database, Azure Storage
- Processes: DataOps, ETL, ELT
- Skills: SQL, Python, PySpark

Expand table

**BI Analyst (P003)**

Team: Analytics Team
- Primary interaction: Data Ingestion Team
- Component access: Power BI, Azure Data Explorer, Azure AI Foundry, Azure Storage
- Processes: Data analysis, data warehousing
- Skills: SQL, Python, PySpark

Expand table

**Discriminative AI Data Scientist (P004)**

Team: AI Team
- Primary interaction: Data Ingestion Team, DevOps Team, Operations Team
- Component access: Azure Machine Learning (for training scenarios), Azure AI Foundry, Azure Databricks, Azure Storage, Azure Key Vault
- Processes: MLOps, MLflow
- Skills: Azure AI Foundry, Azure Machine Learning, Python, model training, production model monitoring

Expand table

**Build Agent MLOps (P009)**

Team: Engineering Team (automated)
- Primary interaction: AI Team, Operations Team
- Component access: Azure AI Foundry, Azure Machine Learning (for training scenarios), Azure DevOps, GitHub
- Processes: Processing and serving of Lambda, outer loop MLOps, automated deployment
- Skills: Python, PySpark, model versioning

Expand table

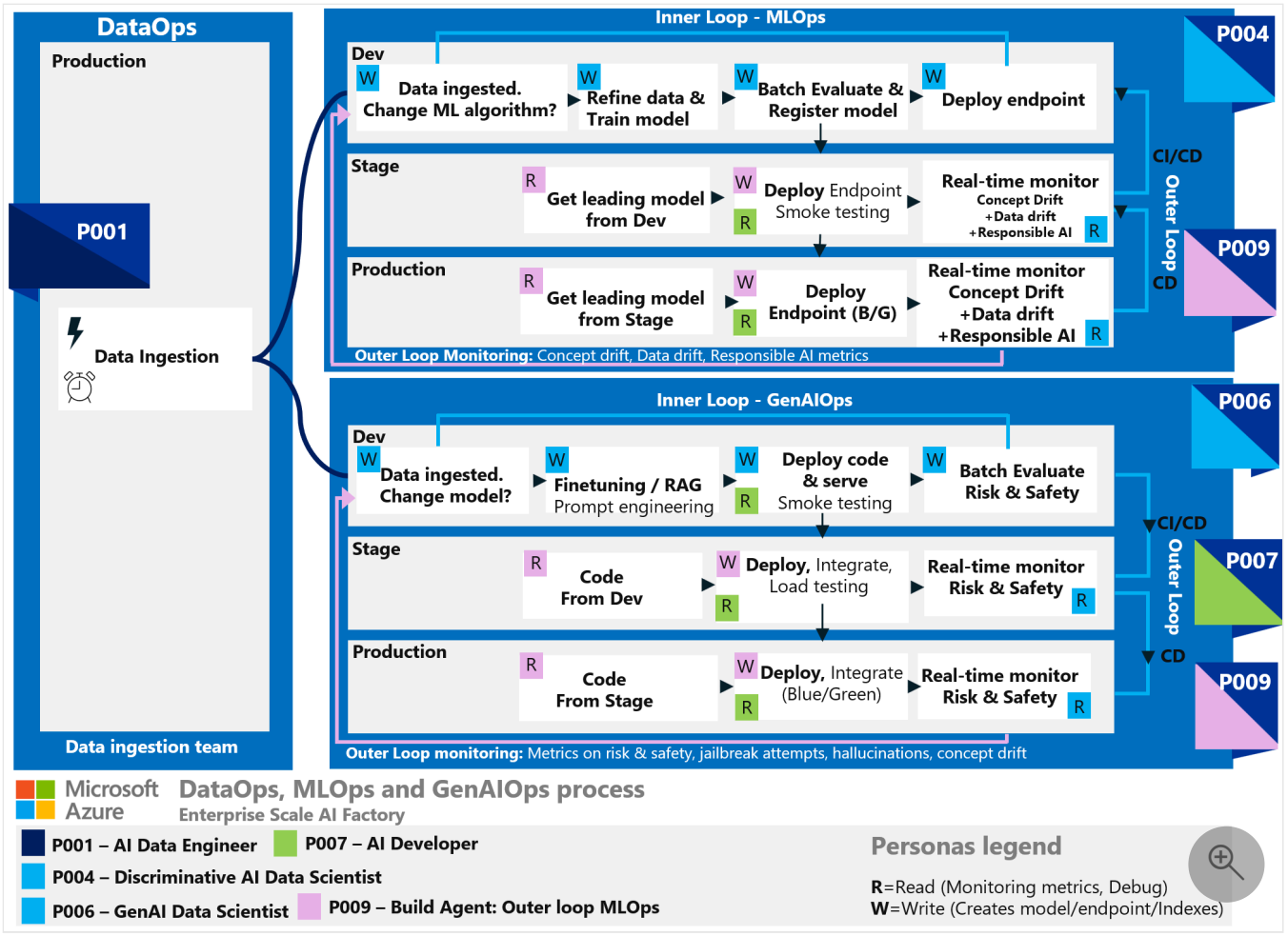
| **Production Monitoring Agent (P012)** |
| --- |
| Team: Operations Team (automated)<br>🔷 Primary interaction: AI Team, Engineering Team, Operations Team<br>🔷 Component access: Azure Monitor, Azure Application Insights, Azure Log Analytics, Azure AI Foundry<br>🔷 Processes: Continuous monitoring, anomaly detection, performance tracking, automated alerting<br>🔷 Skills: Monitoring automation, log analysis, performance metrics, alerting workflows |

# Use case: Personas for AI processes

These main processes are used in AI workloads:

- DataOps is the ingestion and preparation of data.
- MLOps is the operationalization of machine learning models.
- GenAIOps is the discovery and evaluation of existing models and the refinement of these models to the workload context.
- Inner loop is the refinement of solutions in the development environment, either during research or as triggered by outer loop monitoring.
- Outer loop is the movement of solutions from development to production. This loop uses continuous monitoring and evaluation to identify necessary improvements.

Mapping personas to those processes provides context for each persona. This step can help identify the processes where a persona might need upskilling.



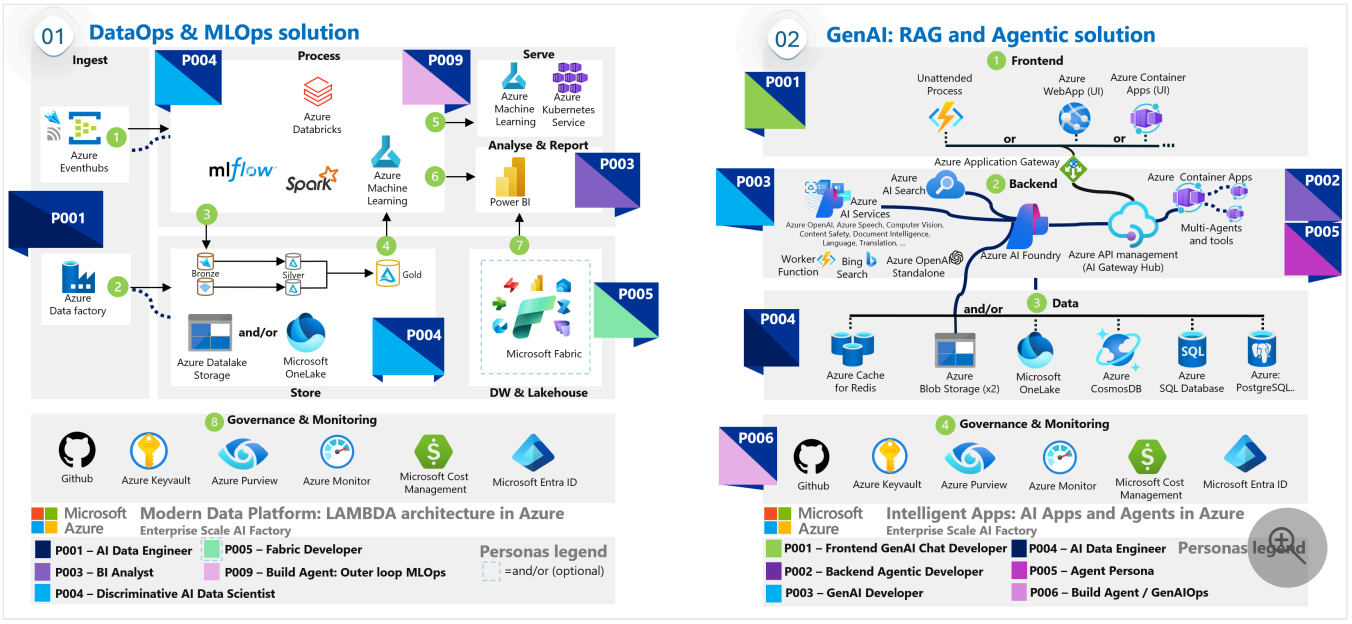

The image shows the workflow for DataOps, MLOps, and GenAIOps within a production environment. Data flows from ingestion to model deployment and evaluation. The workflow uses continuous integration and continuous delivery (CI/CD) practices. Key tasks include refining data models, evaluating batches, deploying endpoints, evaluating models in real-time, and fine-tuning models. The example personas participate in the entire workflow.

# Use case: Personas for architecture design

Connecting processes to the supporting architecture helps you identify the services that a persona needs to interact with and highlights areas for potential upskilling.

To visualize this connection, create a graphical image that shows how architecture components are connected. This visual aid can illustrate data flow and interactions between services and how flows are automated in deployment. It helps stakeholders understand the architecture and the roles of different personas within it.

The following image shows a Lambda architecture for modern analytics on Azure.




# Next step

Next, proceed to the assessment tool to evaluate your design.

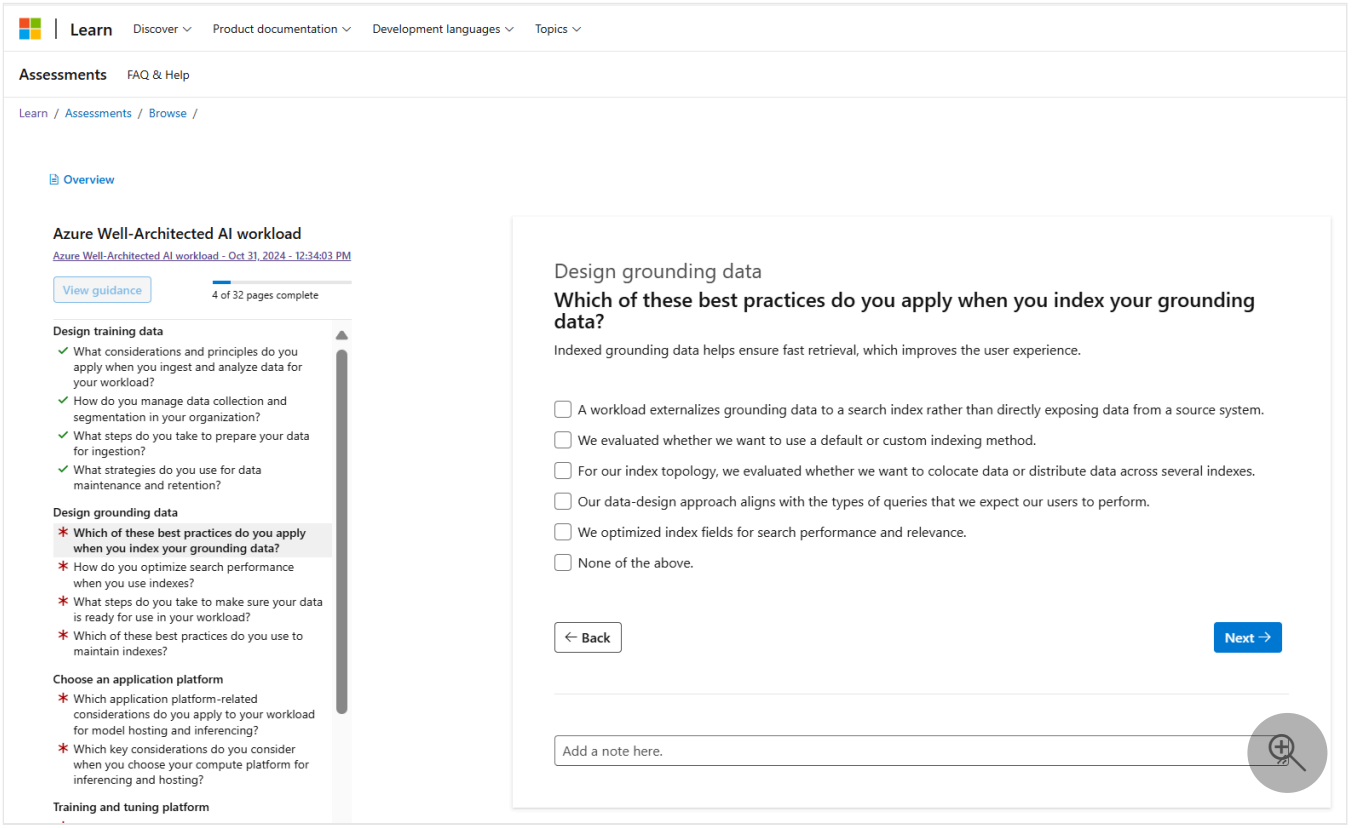
AI workload assessment

# Well-Architected Framework AI workload assessment

The Azure Well-Architected Framework AI workload assessment is a review tool that you can use to self-assess the readiness of your AI workload in production. Running an AI workload on Azure can be a complex process. The assessment is organized so that you can methodically check your workload's alignment with the best practices of the Well-Architected Framework pillars.

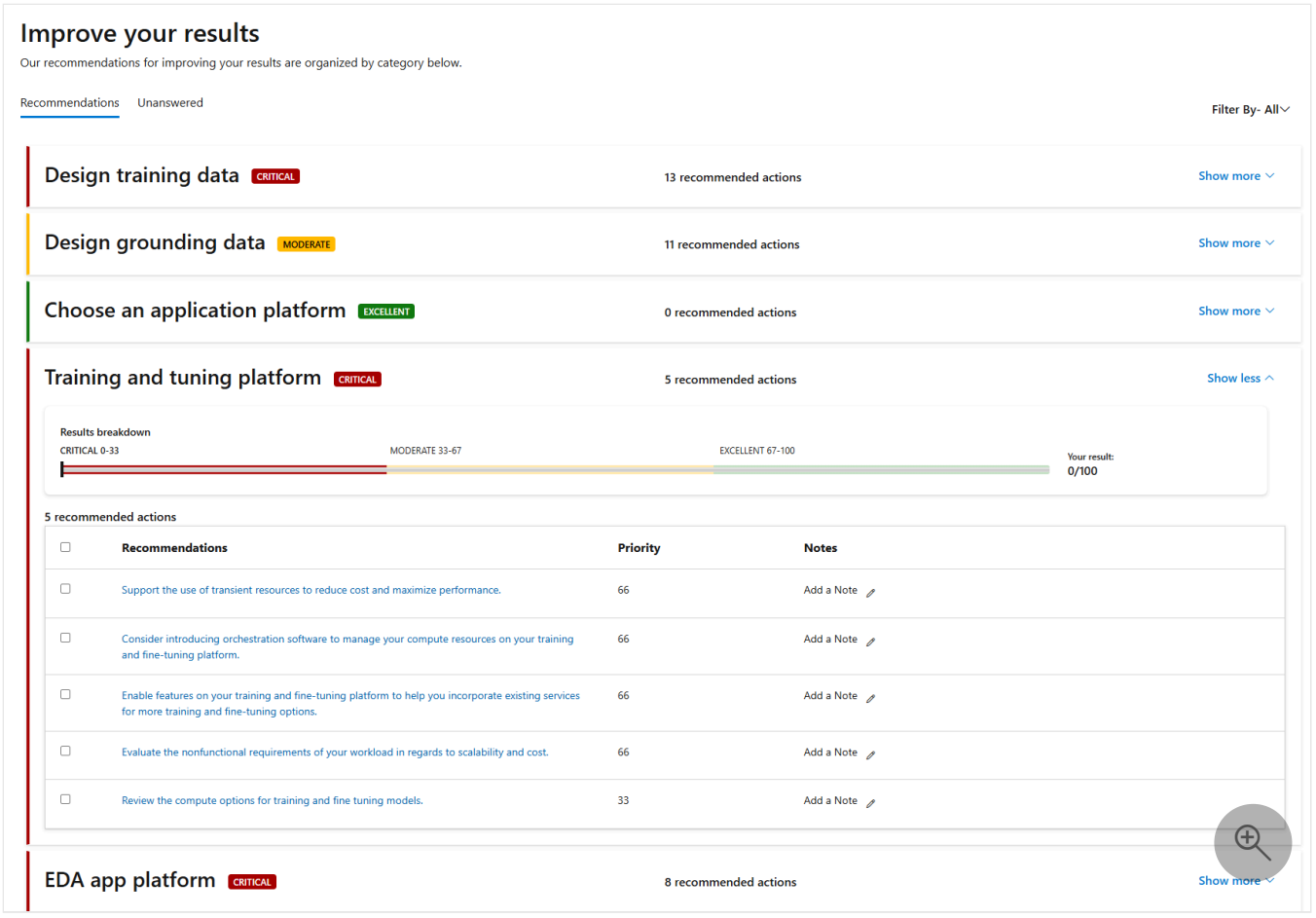
For best results, the team that completes the assessment should be well-versed in the architecture of your workload. That team should also have a strong understanding of cloud principles and patterns. These roles include but aren't limited to cloud architects, operators, and DevOps engineers.

The assessment is a set of questions that are based on the AI workload design areas. It's a method for checking the foundational design choices of your workload's architecture and your end-to-end operational approach.




These questions are designed to help you benchmark your workload's maturity and alignment with the recommended approach for operating an AI workload. The outcome of the

assessment is a set of technical recommendations and documentation that provides guidance about how to implement a highly reliable solution on Azure.




# Related content

See the following reference architecture, which describes design choices for a production-ready implementation:

- Baseline Microsoft Foundry chat reference architecture

---

Last updated on 04/01/2026